Statistical Labs

MED5652

Introduction to
Epidemiology & Statistics

Fred Ho

9 September 2026

Table of contents

Introduction

This is a companion ebook for the statistical labs of the course MED5652 Introduction to Epidemiology and Statistics for MPH students in the University of Glasgow. The principles and theories of statistics are taught in the seminars. This ebook, and the lab sessions, are to apply that knowledge in epidemiology and statistics to real data using R.

The first two preparatory sessions help us ease into the analysis environment. R is primarily a programming language, so it has a steeper learning curve than software like SPSS: we communicate with the computer using text commands instead of a mouse. It pays that curve back, though. R is versatile enough to find a software implementation for almost any statistical method we might need, and it's a skill employers actively look for. This ebook is set up for complete beginners regardless, with no prior data analysis or programming experience expected. From Chapter 3 on, every chapter picks up from the theories taught in the seminars, but it doesn't replace the seminar's content. Think of the seminar as where an idea is introduced, and the lab as where we actually do it, in R, on real data.

Course map

Each chapter of this ebook is one teaching week. The week number is in the chapter title, and the chapters run in week order. Week 6 is reading week and there is no teaching. The two preparatory chapters come before the statistics seminars begin, so they have no seminar of their own to link to.

The table below is the whole course in one place: what each week covers, where that week's seminar slides live, and which chapter in this ebook is the matching lab.

Week	What it covers	Seminar slides	Chapter
4	R and R Studio	None	Chapter 1
5	Data and data handling	None	Chapter 2
7	Describing and visualising data	Slides	Chapter 3
8	Estimation, precision, and sample size	Slides	Chapter 4

Week	What it covers	Seminar slides	Chapter
9	Hypothesis tests and measures of association	To follow	Chapter 5
10	Correlation and simple linear regression	To follow	Chapter 6
11	Multiple linear and logistic regression	To follow	Chapter 7

Course intended learning outcomes

MED5652 as a whole has four intended learning outcomes. By the end of the course, you will be able to:

1. Critically assess the role of epidemiology and statistics within public health.
2. Critically appraise key concepts in epidemiological design.
3. Critically appraise and apply key statistical methods.
4. Critically appraise and evaluate the concept of causality within public health literature.

This ebook is built primarily to get us through ILO 3. Along the way, it also touches on ILO 1, to understand how statistics can inform public health, and ILO 4, to explore how we can use statistics to explore causality.

Structure of the ebook

The ebook is in two parts: two preparatory chapters worked through before seminars on statistics begin, then five weekly chapters, one per statistics week, each following that week's seminar.

All weeks are interconnected, so please attend all classes. Missing one class would mean you lose the foundation of the next, with a knock-on effect on your overall learning. If you really can't attend a class, please make sure you read and work through the relevant chapter in this ebook before the next class.

An appendix at the end of the ebook, the Glossary, collects every package, function, operator, and statistical term used across all seven chapters, grouped by package, with the chapter each one first appears in.

Structure of a chapter

Every chapter has its intended learning outcomes, and all contents are centred around them. At the end of each chapter, there are exercises: most are for you to practise your R skills, but many also require you to understand and interpret the output. This is tailored towards helping you to complete the final assessment, where reporting and interpreting the results correctly are as important as arriving at the right numbers. These end-of-chapter exercises have a folded solution in the ebook. There are also occasional 'Try it yourself' boxes, a light-weight exercise that changes small parts of the code, with no provided solutions.

The two datasets

Most of this ebook runs on one of two datasets. `nhanes_mph` is a curated extract of real health survey data in the US, linked to mortality records. Chapter 2 reads it in for the first time and gets us comfortable with it as data; Chapter 3 introduces it properly and is where it becomes the dataset the rest of the ebook runs on. `strep_tb`, from the `medicaldata` package, is the 1948 Medical Research Council (MRC) streptomycin trial, a historic randomised controlled trial, used from Chapter 4 onward wherever a binary outcome and a genuine trial design are useful. Both get introduced properly, in context, the first time that matters.

How to use the ebook

The best thing to do is to work through the corresponding chapter before you come to the lab. Very likely you will encounter some errors you can't fix, or exercises you can't complete. The labs will be particularly useful to help you with these. The labs will also include demonstration of some key commands but will not be able to cover all the details in this ebook.

When you work on the ebook, attempt the exercises yourself before you open the solutions. Work along in a script of your own as you go, as taught in Chapter 2. The ebook can stay as a reference whenever you need a reminder. If a function's name is only half-remembered, the Glossary at the end is the place to look it up.

1 R and R Studio (Week 4)

We all know how to work with a spreadsheet, but a spreadsheet is not the best way to record, manage, and analyse data. A copy-and-paste error in an Excel-based model contributed to JPMorgan Chase's $6 billion loss in 2012. Public Health England failed to log over 15,000 positive coronavirus test results in 2020, because they used an older Excel format template. A spreadsheet is useful for looking at data and for some simple calculations, but it isn't designed for reproducible work: the work done in one doesn't normally get tracked, and there's no way to tell what has been done to a file just by looking at it. Statistical software like R can take information from spreadsheets and handles it in a different way, keeping a clearer record and making the process more reproducible, which is why public health and statistics use it for data management and analysis. We will examine how to properly interact with R and RStudio, and use it to build an example clinic record.

You need to save R, RStudio and your clinic record on your computer, so first you will work on an installation and setting up a project folder. After that, the chapter is mostly about learning R's vocabulary: objects hold values, values have types, functions act on them, and an error message tells us which of the three we got wrong. Running underneath all of it is one habit that matters more than any of the vocabulary. A saved script can be run again; a line typed into the console is lost and cannot be run again. Reproducibility, replicability and transparancy are key scientific principles: saving scripts is a method for us to make sure our public health science adheres to these principles. Everything we learn goes into a small clinic visit log, built one column at a time until it's a dataset we can ask questions of. Nothing in this chapter is statistics yet; all of it is learning the language we will use to run statistics.

1.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Install R and RStudio on your own device, and verify the installation by running code and loading a package
2. Use an RStudio Project and a saved script, and justify scripting instead of working in the console
3. Assign values to objects, and identify R's basic data types and structures (numeric, character, logical, factor, vector, data frame)

4. Call a function using named arguments, explain what a default value is, and find its help page
5. Write and read a short piped sequence that includes a logical condition and extracts a column from a data frame using `$` and `dplyr`'s `select()`
6. Diagnose a failed line of code from its error message
7. Build a project folder, following sensible file and folder naming conventions

1.2 Setup

There's no data to read yet, so there's nothing to set up in this chapter.

1.3 Installing R and RStudio

R is the programming language itself, and is the software program that actually runs our code. RStudio is a separate, free program that sits on top of R and makes it nicer to work in: a proper editor, a console, and the panes we look at next. Install R first, then RStudio, in that order. We can use R without RStudio, though it's harder for a beginner.

Installing R. Go to https://cran.r-project.org and choose the download link for your operating system, 'Download R for Windows' or 'Download R for macOS.' For most people, you can run the installer and accept every default option.

Installing RStudio. Go to https://posit.co/download/rstudio-desktop/ and download the free RStudio Desktop installer for your operating system. Run it the same way, accepting the defaults throughout.

Verification. Once installed, check it is working properly. Open RStudio, type `2 + 2` into the console, the panel where typed commands run, covered properly in a moment, and press Enter. You should see `[1] 4`. Then confirm a package installs and loads too: run `install.packages('dplyr')` once, then `library(dplyr)`. Both get a proper explanation later in this chapter, under Packages; for now, just check that neither throws an error. If anything here fails, get it fixed before going on, since nothing later in this chapter works without it.

1.4 Finding our way around RStudio

1.4.1 The four panes

RStudio's window is split into four panes, and where each one sits stays consistent no matter what we're doing.

- **Top-left: the source pane.** This only shows when a script is open, so it likely won't appear in RStudio fresh out of the box. Click File > New File > R Script and the pane should appear like in the image below. This is where we write and save scripts, the `.R` files that keep a record of all the statistics we ask R to conduct. Code typed here doesn't run on its own; we have to tell RStudio to run it, which we cover in section 1.4.4.
- **Bottom-left: the console.** This is where code actually executes, one line at a time, and where its output appears. We can type directly into the console, but anything typed there is gone once RStudio closes. We can use the console to test if a line of code works (and check whether we want to keep it or change it), we use the source pane to keep a record of the code we want to keep.
- **Top-right: the environment pane.** This lists every object that currently exists in our R session, its type, and a preview of its contents. Every time we create something, it appears here. When something we expected to see isn't listed, that's usually the first clue something went wrong.
- **Bottom-right: a tabbed pane.** This holds the file browser, the plot viewer, the list of installed packages, and the help viewer, one tab each.

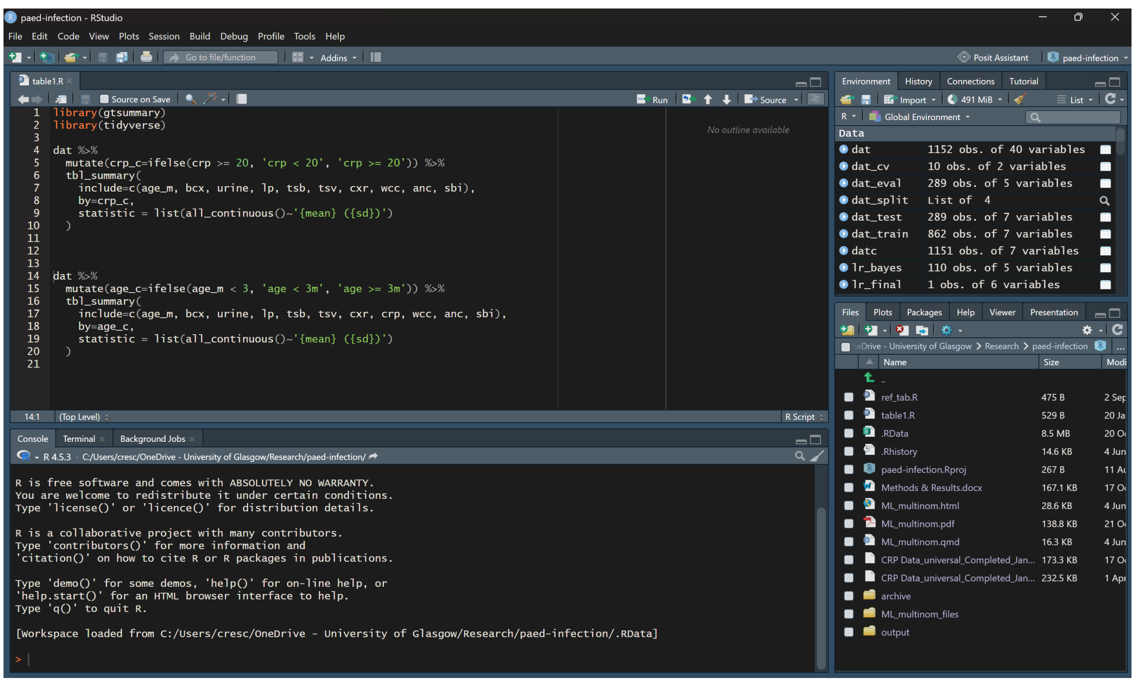


Figure 1.1: RStudio's four panes: source (top-left), console (bottom-left), environment (top-right), and a tabbed pane for files, plots, packages, and help (bottom-right). This particular window is shown in a dark colour theme instead of RStudio's own light default; it's the layout and position of the four panes that matters here, not the colours.

1.4.2 Building a project folder

Working with R means managing at least a couple of files, most likely some data files and some R script files. It is usually a good idea to put them into a "project folder". Pick somewhere sensible on your machine, for example inside Documents. Create a new folder there and give it a short, sensible name, for example `ies_mph`. We will follow a format for naming all files and folders in this course from here on. These naming conventions helps ensure the names can be easily read by computers, and reduces errors in how humans write the code. The conventions are: no spaces, no `&`, `#`, or `%`, consistent case, and a date written as `YYYY-MM-DD`. This date format ensures files are listed in chronological order when sorted by filename. Inside the folder you've created, create three subfolders: `data`, for anything you read in; `scripts`, for the `.R` files you write; and `outputs`, for anything you save out, a plot or a table. Keeping these three separate means you always know where to look for something later, in this course and beyond it. This folder structure isn't compulsory, just a suggestion. We'd still recommend sticking with it throughout the course, just to build the habit. Once you're more familiar with R and RStudio, feel free to work with whatever folder structure works for you. Some employers - particularly in sectors like data science - have their own naming conventions and folder structures, so it's good professional practice to get used to following coding conventions and structures.

1.4.3 Creating an RStudio Project

An RStudio Project turns your folder into something RStudio understands directly. RStudio will consider all folders and files relative to the specific project folder, rather than all the folders on the computer. This allows you to navigate to files using relative paths rather than having to type a full path every time (we will cover this later) and is also helpful when working across multiple projects, and when working as part of a team of analysts on a project.

1. In RStudio's menu, go to **File → New Project…**
2. Choose **Existing Directory**, since the folder already exists.
3. Browse to the project folder you just built, and click **Create Project**.
4. RStudio reopens with that folder as the **working directory**. You can confirm this by checking the Files pane, bottom-right, which now shows the folder's contents, and the title bar, which now shows the project's name.

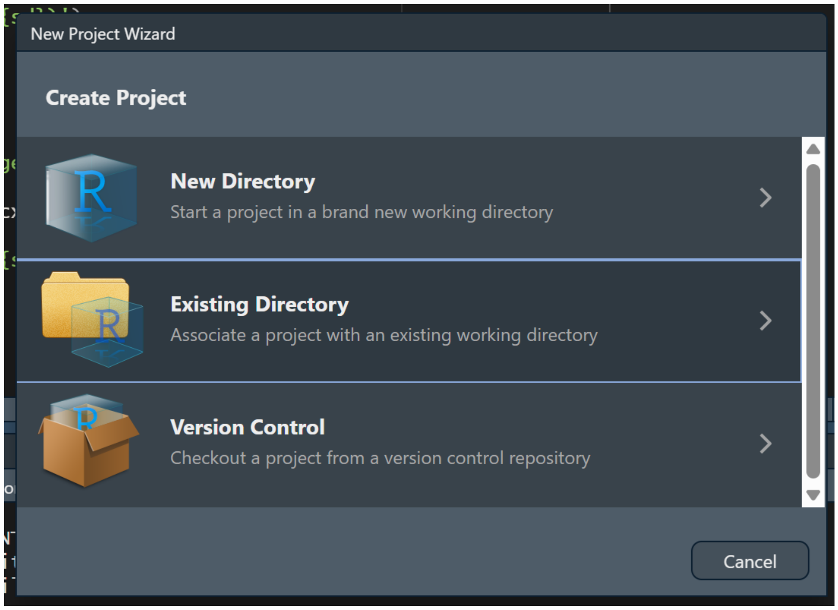


Figure 1.2: The New Project wizard, with Existing Directory highlighted.

A Project's working directory is the folder R treats as 'here' when it looks for a file. Once we start reading data in Chapter 2, this is what lets a path like `data/mph_nhanes_20241107.csv` work on any machine, no matter where the project folder actually sits on that machine's disk.

1.4.4 Console versus script

We can type a line directly into the console and press Enter, and it runs immediately. That's fine for a quick check, but it isn't saved anywhere. Close RStudio, and it's gone.

For anything worth keeping, we write it in a script in the source pane instead, then run it from there:

- **Ctrl+Enter** (Windows) or **Cmd+Return** (Mac) runs the single line the cursor is on, or a selected block of lines, and sends it to the console.
- Highlighting several lines and pressing the same shortcut runs all of them in order.
- The **Source** button, top-right of the source pane, runs the entire script from top to bottom.

The reason this course insists on scripts instead of the console is simple: we will need to do this again, and we won't remember exactly what we typed. A saved script is a record anyone can rerun from scratch and get the same result, which is why every exercise in this book asks for a script instead of console work.

1.5 The log we're going to build

From here on, every section in this chapter adds one real piece to a small dataset: a toy clinic visit log for four patients. By the end of the chapter we'll have this:

patient_id	age	smoker	weight_kg	chol
P01	34	No	72	5.2
P02	61	Yes	88	6.1
P03	45	No	65	NA
P04	29	Yes	59	4.8

Four patients, four pieces of information about each. Using R's language: Assignment gives us one value. A vector strings several values into one column. A data frame lines several columns up side by side, and the table above shows a data frame. Every section that follows explains the terminology used in R, and builds one more piece of the table in R, for real, rather than a table drawn on a page or typed into a spreadsheet.

1.6 Objects and assignment

In R, objects are pieces of information that R uses to perform statistics or data manipulation. You must give each object in your project a unique name, and when you define what information appears in an object, we refer to that as assignment. Storing information in objects means we can use it again later without retyping it, and build on it later in the project workflow rather than starting over. R creates and assigns values to objects with the arrow and line symbol `<-`. A keyboard shortcut for typing is **Alt+-** or **Option+-**. The name of the object appears to the left of the arrow, and the information we want to place in the object appears at the other end of the arrow. Patient P01 is 34 years old:

```
age <- 34
age
```

```
[1] 34
```

`age` now holds the value 34. It appears in the Environment pane, top right, and stays there for the rest of the session, or until we overwrite it by assigning something else to the same name.

1.7 Vectors and a first look at data frames

Most of what we do with data in this course means handling many values of the same variable at once, everyone's age in a sample, say, not just one person's. `c()` combines several values into one object like that, called a **vector**. The single `age` we assigned a moment ago only records one patient; the log needs all four. The code below combines all four ages and assigns the result to `age`, overwriting the single value from before, exactly the way the Objects and assignment section said an object could be overwritten:

```
age <- c(34, 61, 45, 29)
age
```

```
[1] 34 61 45 29
```

A vector holds one type of value, several at once, so it's one variable, like `age` above. A **data frame** holds several vectors side by side, as columns, which is exactly what a real dataset is: one column per variable, lined up so row one is always the same person throughout. This is the structure this course actually works with from the next chapter onward. The code below builds the start of our log from three vectors: `age`, from above, a new `patient_id` vector naming each patient, and a new `smoker` vector.

```
patient_id <- c('P01', 'P02', 'P03', 'P04')
smoker <- c('No', 'Yes', 'No', 'Yes')
visits <- data.frame(patient_id, age, smoker)
visits
```

```
  patient_id age smoker
1        P01  34     No
2        P02  61    Yes
3        P03  45     No
4        P04  29    Yes
```

Note that all three vectors have to have the same length of data (4 values each), otherwise R will complain when we create a data frame. That's not a technicality either: it's the same thing that would go wrong with a real log if we recorded an age for a fifth patient we never gave an ID to.

1.8 Data types

R keeps track of what kind of value an object holds, and it matters: R decides what it will let us do with a value based on its type, for example refusing to calculate a mean for text. Knowing a column's type is knowing what's possible on it. `class()` reports it, and our own log is the fastest way to see two of the four:

```r
class(visits$age)
```

```
[1] "numeric"
```

```r
class(visits$smoker)
```

```
[1] "character"
```

```r
class(TRUE)
```

```
[1] "logical"
```

```r
class(factor(c('Yes', 'No')))
```

```
[1] "factor"
```

Four types matter throughout this course:

- **numeric**: a number, like the ages in `visits`
- **character**: text, like `visits$smoker` right now, always in quotes
- **logical**: `TRUE` or `FALSE`
- **factor**: a set of categories. It looks like text but is stored with a fixed, known list of possible categories behind it. `visits$smoker` is a natural candidate for exactly that, though we leave it as character for now. Chapter 3 turns six columns of `nhanes_mph` into proper factors, exactly this way, against the dictionary file.

1.9 Functions and arguments

Nearly everything we do in R, from here to the last chapter, is calling a function, so we look at the shape of one properly now. A function is a machine that takes some input, processes it, and produces some output. Our log needs one more column: what each patient weighed on the clinic scale, in kilograms, before rounding. P01's reading was 71.83 kg. In the example below, `round()` takes that reading and rounds it to a specific precision, whole kilograms, which is the precision we actually want in the log. Note that a function is always followed by a pair of parentheses `()`.

A function takes different **arguments**, basically the inputs the machine runs on. A function's arguments can be matched two ways: by **position**, in the order the function expects them, or by **name**, in any order at all.

```
round(71.83, 0)
```

```
[1] 72
```

```
round(71.83, digits = 0)
```

```
[1] 72
```

```
round(digits = 0, x = 71.83)
```

```
[1] 72
```

All three lines return the same result. The first matches purely by position: `round()`'s first argument is `x`, its second is `digits`. The second and third name `digits` explicitly, so its position in the call no longer matters, which is why the third line can put `digits` before `x` without changing the answer.

The examples above simply run the function, so the output is shown in the console. If we're going to reuse the output later, we'll have to save it, the same way as the assignment above. All four patients' scale readings, rounded the same way, become our log's weight column:

```
scale_kg <- c(71.83, 88.42, 65.06, 58.57)
weight_kg <- round(scale_kg, digits = 0)
weight_kg
```

```
[1] 72 88 65 59
```

```r
visits <- data.frame(visits, weight_kg)
visits
```

```
  patient_id age smoker weight_kg
1        P01  34     No        72
2        P02  61    Yes        88
3        P03  45     No        65
4        P04  29    Yes        59
```

Some arguments have a **default**: a value the function uses automatically if we don't supply one. P03's bloods were never taken, so her cholesterol reading is missing:

```r
chol <- c(5.2, 6.1, NA, 4.8)
mean(chol)
```

```
[1] NA
```

```r
mean(chol, na.rm = TRUE)
```

```
[1] 5.366667
```

`chol` has one missing value, `NA`, for P03. `mean()`'s `na.rm` argument defaults to `FALSE`, so by default a single missing value makes the whole result `NA`, exactly as the first line shows. Setting `na.rm = TRUE` tells `mean()` to drop that one missing value first, then average the three that remain. We'll meet `na.rm` constantly from here on, and it always means exactly this: drop the missing values before calculating, and be clear about how many were dropped. `chol` joins the log the same way `weight_kg` just did:

```r
visits <- data.frame(visits, chol)
visits
```

```
  patient_id age smoker weight_kg chol
1        P01  34     No        72  5.2
2        P02  61    Yes        88  6.1
3        P03  45     No        65   NA
4        P04  29    Yes        59  4.8
```

1.10 Comparison and logical operators

These build a condition that's either `TRUE` or `FALSE` for each value in a vector, which is what every `filter()` from Chapter 3 onward runs on:

```r
visits$age > 50
```

```
[1] FALSE  TRUE FALSE FALSE
```

```r
visits$age == 34
```

```
[1]  TRUE FALSE FALSE FALSE
```

```r
visits$age != 34
```

```
[1] FALSE  TRUE  TRUE  TRUE
```

`!` negates a condition, `&` combines two conditions and requires both to be true, `|` combines two conditions and requires either one to be true, and `%in%` tests whether each value appears anywhere in a set:

```r
!(visits$age > 50)
```

```
[1]  TRUE FALSE  TRUE  TRUE
```

```r
visits$age > 50 & visits$smoker == 'Yes'
```

```
[1] FALSE  TRUE FALSE FALSE
```

```r
visits$age > 80 | visits$age < 30
```

```
[1] FALSE FALSE FALSE  TRUE
```

```r
visits$age %in% c(34, 45)
```

```
[1]  TRUE FALSE  TRUE FALSE
```

One patient, P02, is both over 50 and a smoker.

1.11 Packages

Everything used so far, `c()`, `class()`, `round()`, `mean()`, comes built into R itself. Most of what this course does lives in add-on packages instead: `dplyr` for manipulating data, `ggplot2` for plotting, and others still to come. Getting a package ready to use is two separate actions. **Installing** downloads it onto this computer, once. **Loading**, with `library()`, makes its functions available in the current R session, and has to be done again every time we start a fresh session, even though we never need to install the same package twice.

Installing is typically a one-off, run in the console and not saved in a script: `install.packages('dplyr')`. Loading is what actually goes in a script, since every script that uses `dplyr` needs it loaded first:

```r
library(dplyr)
```

Loading a package sometimes prints a short message, often about functions it's replacing from another package. It's suppressed here since it doesn't matter yet. Later in the course it will, for a specific reason.

That message points at a real ambiguity: once two loaded packages both define a function with the same name, R has to pick one, and it's not always the one we meant. `package::function()` sidesteps the whole problem. Writing `dplyr::select()` instead of just `select()` calls `select()` from `dplyr` directly, by name, whichever packages happen to be loaded, and without needing `library()` to have loaded that package first at all. We'll use this notation in prose from here on whenever it helps to be explicit about which package a function comes from.

1.12 `$` versus `select()`

`$` pulls a single column out of a data frame, as a plain vector:

```r
visits$smoker
```

```
[1] "No"  "Yes" "No"  "Yes"
```

`select()`, from `dplyr`, keeps a column too, but keeps it inside a data frame:

```r
select(visits, smoker)
```

```
  smoker
1     No
2    Yes
3     No
4    Yes
```

The values are the same either way. The difference is the container. `visits$smoker` is four bare values with nothing else attached. `select(visits, smoker)` is still a data frame, one column wide, four rows long. This matters practically from Chapter 4 onward: some functions, like `t.test()`, want a bare vector and expect `$`. Most of `dplyr`, starting in Chapter 3, works on data frames throughout and expects `select()`.

1.13 Quoting rules and backticks

Text values need quotes. Numbers, `TRUE`/`FALSE`, and the names of objects or columns don't. Our own log makes the point directly: patient IDs are text and need quotes; ages don't.

```r
class('P01')
```

```
[1] "character"
```

```r
class(34)
```

```
[1] "numeric"
```

`'P01'` is text. `34` is a number, and `class()` treats them completely differently. The same logic applies to object and column names: writing `age` refers to the vector we created earlier, but writing `'age'` is just the three-letter word.

A column name containing a space needs a different kind of quoting mark entirely: backticks, not quotation marks.

```r
toy <- data.frame(`patient id` = c(1, 2, 3), check.names = FALSE)
toy$`patient id`
```

```
[1] 1 2 3
```

Backticks aren't easy to work with, and generally it's not recommended to have variable names containing a space. `data.frame()` normally cleans up column names automatically, turning a space into a dot. `check.names = FALSE` switches that off, purely so this example can show a genuine spaced name. When we read a CSV file with `read_csv()` in Chapter 2, column names come in exactly as written, spaces and all, with no automatic cleanup. Its dictionary file has a column literally called `Variable names`, and backticks are exactly how we'll refer to it.

1.14 The pipe

Some R code can be difficult to understand. For instance `round(mean(visits$chol, na.rm = TRUE), digits = 1)` can be quite complex for a new R user. This is mainly because there are two functions in that line, one enclosing another. If you want to decipher code like this, the best way would be to break it down one function at a time, starting from the most central part.

One way to write more understandable code is by using the pipe `|>`. It takes what's on its left and feeds it in as the first input to what's on its right. Read it aloud as 'and then':

```
visits$chol |> mean(na.rm = TRUE) |> round(digits = 1)
```

```
[1] 5.4
```

Take the data's cholesterol column, and then average it, dropping the one missing value, and then round the answer to one decimal place. The pipe exists so a sequence of steps can be read in the order they actually happen, which is how every multi-step `dplyr` sequence in this book, starting in Chapter 3, is written.

There is a keyboard shortcut for the pipe: **Ctrl+Shift+M** (Windows) or **Cmd+Shift+M** (Mac). We'll write a lot of pipes over the rest of this course, so it'd be useful to memorise this shortcut.

If we see other people's code, we may find `%>%`, the older pipe, which is only available once we've loaded the `dplyr` package. Practically, both pipes can be treated the same.

1.15 Reading errors

The errors below are R telling us our *code* is wrong: a typo, a missing bracket, the wrong function name. That's a different job from checking whether the *data* are wrong, which is Chapter 2's problem once we're reading in a real file rather than typing values ourselves. Even a very experienced R user encounters errors constantly. The skill is reading what the message says, not reacting to the fact that something went wrong. R usually formats errors as a short heading followed by a line starting with `!` stating what went wrong. Five kinds of failure account for nearly everything we'll see in this course.

Object not found. R can't find something by that name, usually a typo or a variable that was never created:

```r
mean(agee)
```

```
Error:
! object 'agee' not found
```

```r
mean(age)
```

```
[1] 42.25
```

Could not find function. Same idea, but for a function name. R is case-sensitive throughout, so `Mean()` is not the same thing as `mean()`:

```r
Mean(age)
```

```
Error in `Mean()`:
! could not find function "Mean"
```

```r
mean(age)
```

```
[1] 42.25
```

Unexpected symbol. R's interpreter (the machine that reads our code and executes it) found something it couldn't make sense of, usually a missing operator or a missing comma between two things that were meant to be one call:

```r
c(34, 61) c(45, 29)
```

```
Error in parse(text = input): <text>:1:11: unexpected symbol
1: c(34, 61) c
              ^
```

```r
c(34, 61, 45, 29)
```

```
[1] 34 61 45 29
```

Notice the fix is just `age` written out longhand: a missing comma between two calls that were meant to be one is exactly the mistake that would have split our own age vector in two.

Unexpected end of input. A bracket was opened and never closed, so R keeps waiting for the rest of the expression that never arrives. In the console this shows up as a `+` prompt instead of the usual `>`, waiting for us to finish the line; here, where the code has to run to completion on its own, it shows as an error instead:

```r
sum(weight_kg
```

```
Error in parse(text = input): <text>:2:0: unexpected end of input
1: sum(weight_kg
   ^
```

```r
sum(weight_kg)
```

```
[1] 284
```

File does not exist. R can't find a file at the path given, usually a typo in the file name or a path built from the wrong folder:

```r
read.csv('nonexistent_file.csv')
```

```
Warning in file(file, "rt"): cannot open file 'nonexistent_file.csv': No
such file or directory
```

```
Error in `file()`:
! cannot open the connection
```

The warning line above states the actual reason, a missing file, before the error itself. We'll meet this one properly in Chapter 2, once there's a real file to read, the clinic's own version of the log we've just spent this chapter building by hand. `read.csv()` is base R's own version of the function; from there onward we use `read_csv()` from the `readr` package instead, which behaves a little differently but fails the same way when a path is wrong.

1.16 Getting help

Every function used in this course has a help page. Typing `?mean` in the console and pressing Enter opens `mean()`'s help page in the Help pane, bottom-right, showing exactly what arguments it takes and what it returns. It's usually faster than searching online, and it's always correct for the version of R actually installed. RStudio also keeps a set of cheat sheets under **Help → Cheatsheets**, one per major package. Beyond that, this book itself stays the main reference for the rest of the course.

1.17 The finished log

Four sections ago we had nothing but an idea for a table. Here's what we've actually built, one piece at a time: patient IDs, ages, smoking status, rounded weights, and cholesterol.

```
visits
```

```
  patient_id age smoker weight_kg chol
1        P01  34     No        72  5.2
2        P02  61    Yes        88  6.1
3        P03  45     No        65   NA
4        P04  29    Yes        59  4.8
```

We can check what type each piece is, exactly the way the Data types section did partway through:

```
class(visits)
```

```
[1] "data.frame"
```

```
class(visits$age)
```

```
[1] "numeric"
```

```
class(visits$smoker)
```

```
[1] "character"
```

Pull out one column as a bare vector, and build a logical condition on it, the same as the Comparison section did:

```
visits$age
```

```
[1] 34 61 45 29
```

```
visits$age > 50 & visits$smoker == 'Yes'
```

```
[1] FALSE  TRUE FALSE FALSE
```

One of the four patients, P02, is a smoker over 50. Compare `$` against `select()` on the same data frame:

```
visits$smoker
```

```
[1] "No"  "Yes" "No"  "Yes"
```

```
select(visits, smoker)
```

```
  smoker
1     No
2    Yes
3     No
4    Yes
```

Same values, different container: a bare vector from `$`, a one-column data frame from `select()`. Chain two steps on it with the pipe, the same calculation the pipe section ran a moment ago:

```
visits$chol |> mean(na.rm = TRUE) |> round(digits = 1)
```

```
[1] 5.4
```

1.17.1 Saving the log

Everything in `visits` only exists inside this R session, in memory. Close RStudio without saving it out, and it's gone, the same way a value typed straight into the console disappeared the moment we moved on. `write.csv()`, from base R, does the opposite of `read.csv()` from the Reading errors section above: it takes an object and a file path, and writes the object out as a real CSV file on disk.

```
write.csv(visits, 'visits.csv', row.names = FALSE)
```

Left out, `write.csv()` adds an extra column at the front of the file, numbering the rows 1 to 4. That's not part of our data, just R's own internal row labels leaking into the file, and it would show up as a stray, unlabelled column the moment the file is opened. `FALSE` tells `write.csv()` to leave it out and write only the columns we actually built.

That line just wrote a real file, `visits.csv`, sitting on disk the moment this page rendered, independent of R and of this book. In our own project, following the `data/scripts/outputs` structure from earlier in this chapter, this is what the `outputs` subfolder is for: a file we saved out, not one we're about to read back in. Opening it, in a spreadsheet application or a plain text editor, shows exactly what `visits` held in R: patient IDs, ages, smoking status, weight, and cholesterol, one row per patient, values separated by commas, no formatting or formulas hidden behind them. Chapter 2 explains why that matters, the moment we start reading files back in instead of writing them out.

Every idea used to build this log, a vector, a data frame, `$`, a comparison, loading a package, the pipe, comes back constantly from Chapter 2 onward. Only the data changes: Chapter 2 brings in this same clinic's real log, all eight patients, exactly as messily kept as real logs actually are.

1.18 Exercises

Exercise 1 (ILO 3). Create a vector called `height_cm` holding the values of 177, 155, 168, 181, using `c()` and `<-`. Check its class with `class()`, and confirm `height_cm` appears in the Environment pane after you run your code.

Exercise 2 (ILOs 4, 5). Using `visits$chol`, round the values to the nearest whole number, naming the `digits` argument explicitly rather than relying on position. Then build a logical condition that identifies which patients have a cholesterol reading above 5 and are smokers, combining a comparison operator with `&`.

Then open `round()`'s help page by typing `?round` in the console. Find the **Usage** section at the top, and write down what value `digits` takes when you don't supply one. Check your answer by running `round()` on the same values with the `digits` argument left out entirely.

Exercise 3 (ILOs 4, 5). Update the data frame `visits` to include the new `height_cm` variable. Calculate the BMI (weight in kg divided by height in metres, squared) of all the patients by using the `$` sign.

Exercise 4 (ILO 5). Write a piped sequence, using `|>`, that takes `visits$age`, finds its mean, and rounds the result to zero decimal places. Then use `select()` to keep just `patient_id` and `smoker` from `visits`, and write one sentence stating how the result differs from `visits$smoker`.

1.19 Self-check

A short multiple-choice check, not a writing exercise. There's no statistical result yet to interpret in a sentence. That starts in Chapter 3. Instead, these five questions target the places students most often get tripped up in R's syntax. Answers are in the Solutions callout below.

1. In `mean_x <- mean(x, na.rm = TRUE)`, what's the difference between `<-` and `=`?

a) They mean exactly the same thing
b) `<-` creates or updates an object; `=` here only names an argument inside the function call
c) `<-` only works inside a function call, `=` only works outside one
d) `=` creates an object; `<-` only names an argument

2. `visits$smoker` and `select(visits, smoker)` return the same four values. What actually differs between them?

a) Nothing, they always return identical output
b) `$` only works on vectors, never on data frames
c) `$` returns a bare vector; `select()` returns a data frame with one column
d) `select()` is base R; `$` is from `dplyr`

3. In `round(x = 3.14159, digits = 2)`, what happens if you write `round(digits = 2, x = 3.14159)` instead?

a) Nothing changes: named arguments can be given in any order
b) R throws an error, because the arguments are in the wrong order
c) R silently ignores `digits` and uses its default instead
d) It only works if `x` is written first, always

4. Why does R separate `install.packages()` from `library()`?

a) They do the same thing; `install.packages()` is just the older name for it
b) `install.packages()` downloads a package once; `library()` loads it into the current session, and has to be run again every time R restarts
c) `library()` downloads the package; `install.packages()` only loads it

d) Both have to be run every single time you use a function from a package

5. Running `Mean(age)` produces `Error in Mean(age) : could not find function 'Mean'`. What's actually wrong?

a) The object `age` doesn't exist
b) R is case-sensitive and there's no function called `Mean`; the function is `mean()`
c) A package needs to be installed first
d) There's a missing comma somewhere in the call

Solutions

Exercise 1

```
height_cm <- c(177, 155, 168, 181)
height_cm
```

```
[1] 177 155 168 181
```

```
class(height_cm)
```

```
[1] "numeric"
```

Exercise 2

```
round(visits$chol, digits = 0)
```

```
[1]  5  6 NA  5
```

```
visits$chol > 5 & visits$smoker == 'Yes'
```

```
[1] FALSE  TRUE FALSE FALSE
```

`?round`'s Usage section reads `round(x, digits = 0)`, so `digits` defaults to 0: leave it out and R rounds to the nearest whole number. Running it both ways confirms that:

```
round(visits$chol)
```

```
[1]  5  6 NA  5
```

```
round(visits$chol, digits = 0)
```

```
[1]  5  6 NA  5
```

The two lines give identical results, which is what a default argument means in practice. The help page is the fastest way to find out what a function does when we don't tell it, and it's always right for the version of R we actually have installed.

Exercise 3

```
visits <- data.frame(visits, height_cm)
bmi <- visits$weight_kg / (visits$height_cm / 100)^2
bmi
```

```
[1] 22.98190 36.62851 23.03005 18.00922
```

Exercise 4

```
visits$age |> mean() |> round(digits = 0)
```

```
[1] 42
```

```
select(visits, patient_id, smoker)
```

```
  patient_id smoker
1        P01     No
2        P02    Yes
3        P03     No
4        P04    Yes
```

`select(visits, patient_id, smoker)` returns a data frame with two columns and four rows. `visits$smoker` returns just the smoker column, on its own, as a bare vector with no `patient_id` alongside it and no data frame structure around it.

Self-check answers

1. **b.** `<-` creates or updates an object. `=` inside a function call only names which argument a value belongs to; it never creates an object there.
2. **c.** Same values, different container: `$` strips the data frame away and leaves a bare vector, `select()` keeps the data frame structure.
3. **a.** Named arguments are matched by name, not position, so their order in the call makes no difference to the result.
4. **b.** Installing is a one-off download. Loading has to happen at the start of every R session, since a fresh session starts with no packages loaded, even ones already installed.
5. **b.** R is case-sensitive throughout. `Mean` and `mean` are different names as far as R is concerned, and only one of them is a real function.

2 Data and data handling (Week 5)

A clinic spreadsheet kept by hand for years has a title sitting in the first row, a blank row under the headers, dates written three different ways, and a column where someone typed 'n/a' one month and '-' the next. Nobody planned it that way. It happened one edit at a time, and files like it are what real analysis actually starts from. The dataset we typed out in Chapter 1 was clean because we invented it. This chapter deals with the other kind.

Making a file like that usable is most of what data analysis actually is, and none of it counts unless it's written down: a cleaned dataset nobody can reproduce is one person's private result. So the chapter runs from the messy spreadsheet all the way through to a saved script that rebuilds the lot from nothing. In between sits the step that catches almost everyone out the first time, which is telling R where a file lives on our own computer, getting it in, and checking what actually arrived instead of assuming. Week 6 is reading week; Chapter 3 picks up from here.

2.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Identify features of a spreadsheet that will prevent it being analysed, and restructure it
2. Import a CSV into R using `readr::read_csv()` and a correctly specified path built with `here::here()`, and diagnose and resolve the most common import failures
3. Examine an imported dataset's dimensions, variable names, and types with `dim()`, `names()`, and `dplyr::glimpse()`, and its missing values with `is.na()`
4. Write, save, and re-run an R script that imports data, summarises it, and produces a plot
5. Locate a downloaded file on your own operating system, construct or copy a valid path to it, and explain what a CSV file physically contains

2.2 Setup

There's nothing to read in yet, so there's nothing to set up in this chapter.

2.3 Tidy data, in a spreadsheet

2.3.1 What makes data usable

A dataset R can work with follows three simple rules: one variable per column, one observation per row, one value per cell. Data laid out this way is called **tidy**. It sounds obvious written down, but very few spreadsheets we get are tidy. People merge cells to make a heading look nice. They use colour instead of writing a word down. They leave blank rows for visual spacing. None of that survives import, and some of it actively breaks it.

2.3.2 What breaks an import

Picture reading the sheet in the screenshot below the way a person would, working down it row by row, and watching where it stops behaving like one clean table.

Row 1 is a title, 'Clinic Visit Log - Week 12', stretched across six columns as one merged cell. A heading merged like that looks fine to a person. But when R reads the underlying columns back out, most of them come back empty, because the merge only stored the value in one of the cells it covers. Row 2 does the same thing on a smaller scale, merging 'Measurements' across just the Weight and Height columns, so a proper header for either one only turns up in row 3.

Read on down the Weight and Height columns themselves and one row breaks the pattern. Most patients are plain numbers, kilograms and metres. One row is text instead: `195 lbs` and `6 ft`. R reads a whole column as text the moment a single cell in it isn't a bare number. So it only takes one row like that to turn an otherwise clean numeric column into text throughout, no matter how many other rows are perfectly formed. This is the same problem Chapter 1's `round()` example was quietly solving for us by hand, turning a raw scale reading into a clean whole-kilogram number before it ever went in the log. Nobody has done that here, and R has no way to do it on our behalf.

Off to the side, in a cell with no column of its own, sits a note about one specific patient's missed follow-up, disconnected from the table it's actually about. Further down, after a blank row inserted purely for visual spacing, which becomes a row of missing values, `NA`, in every column the moment it's imported, sits a second, smaller table: a follow-up schedule, unrelated to the patient log above it. R expects one table per sheet, so this second table gets read as more rows of the first one, and the result makes no sense.

Spreadsheet exercise (ILO 1). `data/05-prep2-messy-clinic.xlsx` is a small, made-up clinic visit log, built specifically to have every one of these problems at once. Open it in Excel, or whatever spreadsheet software you have.

	A	B	C	D	E	F	G
1	Clinic Visit Log - Week 12						
2					Measurements		
3	Patient ID	Name	Visit date	Age	Weight	Height	
4	P01	A Ferreira	03/03/2025	34	72	1.70	
5	P02	B Okafor	03/03/2025	61	88	178	
6	P03	C Nilsson	04/03/2025	45	65	1.62	
7	P04	D Petrova	04/03/2025	29	59	1.65	
8	P05	E Marsh	05/03/2025	52	94	1.74	call P05 back - missed last follow-up
9	P06	F Haddad	05/03/2025	38	195 lbs	6 ft	
10	P07	G Lindqvist	06/03/2025	67	68	1.58	
11	P08	H Osei	06/03/2025	41	81	1.72	
12							
13	Highlighted are smokers						
14							
15	Follow-up schedule						
16	Patient ID	Next visit					
17	P01	17/03/2025					
18	P03	18/03/2025					
19	P05	18/03/2025					
20	P07	20/03/2025					

Figure 2.1: `05-prep2-messy-clinic.xlsx` open in a spreadsheet application: a merged title row, a merged 'Measurements' sub-header spanning the Weight and Height columns, a block of rows highlighted yellow with a caption below the table explaining what the highlighting means, a stray note in one row about a patient's missed follow-up, a blank spacer row, and a second, separate table further down the sheet.

Go through the sheet and list every feature above that you can find in it. There is also one problem in this sheet that wasn't mentioned above. Try to build a clean version based on the principles above. Save your clean version as a `.csv` file. The Solutions section describes the fixes.

2.4 Finding a file, and telling R where it is

We just saved a clean CSV in the exercise above. Before we can read it, or anything else, into R, we need to know exactly where the file we want actually lives. This is the file system skill everything from here on assumes.

Where downloads go. When you download a file from a browser, most browsers save it into a folder called `Downloads` by default, without asking. On Windows that's usually `C:\Users\yourname\Downloads`; on a Mac it's usually `/Users/yourname/Downloads`. If you can't find a file you just downloaded, that folder is the first place to check. Confirm where your own browser is set to save to, since some let you change it.

Reading a path. A file path is a set of directions from somewhere fixed down to one specific file, one folder at a time, separated by slashes. `data/mph_nhanes_20241107.csv` reads as 'the folder called `data`, then inside it the file called `mph_nhanes_20241107.csv`.' A longer path just has more folders to walk through before reaching the file.

Copying a path. We'll rarely type a full path out by hand; instead, we copy it straight from the file browser.

- **Windows:** hold Shift and right-click the file, then choose **Copy as path**.
- **macOS:** right-click the file, then hold the **Option** key, which changes the menu, and choose **Copy 'filename' as Pathname**.

Backslashes versus forward slashes. A path copied this way on Windows uses backslashes, `\`, for example `C:\Users\yourname\Documents\ies_mph\data\file.csv`. R uses forward slashes, `/`, instead, and treats a lone backslash as the start of a special character, not a folder separator. So a pasted Windows path usually needs every `\` replaced with `/` before R will accept it.

What a CSV actually is. Open a `.csv` file in a plain text editor, Notepad on Windows or TextEdit on a Mac, rather than in spreadsheet software, and you'll see plain text: one line per row, values separated by commas. There's no hidden formatting behind it, which is exactly why it's such a safe format to hand between different pieces of software, R included.

2.5 Getting data in

2.5.1 `read_csv()` and `here()`

`readr`'s `read_csv()` reads a `.csv` file into R as a data frame. It needs a path telling it exactly where that file is, using the path skills from earlier in this chapter. Building that path with `here()`, from the `here` package, means the same code works on any machine, regardless of where the project folder actually sits on that machine's disk: `here()` starts from the project folder and adds the pieces you give it, one folder at a time.

This is a different function from `read.csv()`, base R's own version, which Chapter 1 used deliberately, and from `write.csv()`, which Chapter 1 used to save `visits` out. Base R's versions weren't wrong there, just the simpler tool for four hand-typed patients. From here on we use `readr`'s `read_csv()`, and later `write_csv()`, instead. They read faster on a file this size. They also don't silently guess at things the way base R's older functions do: `read.csv()` used to turn every text column into a factor by default unless told not to, a behaviour that caught out R users for years. `read_csv()` and `write_csv()` are the pair this book, and the wider tidyverse, are built around. The data is about to get a lot bigger than four patients, and that's exactly when a guess R gets wrong starts to matter.

This chapter needs three packages: `readr` for reading files, `here` for building paths, and `ggplot2` for the plot at the end. As Chapter 1 explained, installing a package and loading it are two different actions. Unless these are installed already, run this once in the console, not inside this document:

```
install.packages(c('readr', 'here', 'ggplot2'))
```

After that, `library()` is all we need, in this session and every session after it:

```
library(readr)
library(here)
```

`library(here)` prints one line when it loads, which this book hides because it's too wide to fit the page. In your own console it says `here() starts at ...` followed by the full path to your project folder. Read it the first time you see it, because it's `here()` telling you exactly which folder it will build every path from. If that path isn't the project folder you built in Chapter 1, nothing below will find its files, and this is where you'd find that out.

2.5.2 Four things that go wrong

With the path skills just covered, we're ready to actually read a file in, and the fastest way to get comfortable with that is watching it fail a few ways on purpose first. Almost every import problem we'll see is one of four things.

A wrong filename. Close, but not exact:

```
read_csv(here('data', 'mph_nhanes_20241107_data.csv'))
```

```
Error:
! '/projects/ies_stats_ebook/data/mph_nhanes_20241107_data.csv' does
  not exist.
```

The real file is called `mph_nhanes_20241107.csv`, no `_data` in the name. R's message states plainly that the file doesn't exist at the path it tried, which is exactly what Chapter 1's 'file does not exist' error type looks like with a real file behind it.

```
read_csv(here('data', 'mph_nhanes_20241107.csv'))
```

A path pasted straight from Windows Explorer. Shift+right-click → *Copy as path*, from earlier in this chapter, gives you a full path using backslashes, for example `C:\Users\yourname\Documents\new_folder\mph_nhanes_20241107.csv`. Paste that directly into R as a string, without changing anything:

```
read_csv('C:\Users\yourname\Documents\new_folder\mph_nhanes_20241107.csv')
```

```
Error: '\U' used without hex digits in character string (<input>:1:14)
```

This isn't even a 'file not found' error. It never gets that far. `\U` is one of a handful of backslash sequences R's parser treats specially inside a string, and `\Users` isn't a valid one, so R refuses to parse the string at all. Every Windows path copied this way starts with `C:\Users\...`, so this isn't a rare edge case, it's the default shape of a pasted Windows path. The fix from earlier in this chapter still applies: replace every backslash with a forward slash before using a path in R. Better still, once the path is inside the project folder, `here()` avoids the problem entirely, since we never type the backslash-filled part of the path at all.

The right file, wrong location. Forgetting which subfolder the file lives in:

```
read_csv(here('mph_nhanes_20241107.csv'))
```

```
Error:
! '/projects/ies_stats_ebook/mph_nhanes_20241107.csv' does not
  exist.
```

The filename is exactly right this time. What's missing is `'data'`, the subfolder it actually lives in. R's error message looks identical to the wrong-filename case above, which is worth noticing: the message tells us *that* nothing was found at that path, not *why* the path was wrong. We have to check that ourselves.

```
read_csv(here('data', 'mph_nhanes_20241107.csv'))
```

The wrong file type. `data/05-prep2-messy-clinic.xlsx`, our own clinic file from the exercise above, is an Excel file, not a CSV. Reading it with `read_csv()` doesn't throw an error at all.

To see what actually came in, we need three functions. `dim()` reports how many rows and columns a dataset has, and `names()` lists its column names. Both get a proper introduction in the next section, where we use them on real data; here they're just the quickest way to look. `substr()`, from base R, takes a piece of text and keeps only part of it, given a starting and an ending position: `substr(x, 1, 50)` keeps the first 50 characters and throws the rest away. We need it because the column name that comes back is far too long to print in full.

There's one extra argument in the call below, `show_col_types = FALSE`. `read_csv()` normally prints a short report of what it found and what type it guessed for each column. We meet that report properly in a moment, on the real data file, where it's useful. Here it would just be several lines of nonsense about a file that hasn't imported, so we turn it off to keep the broken result below the only thing on screen.

```
messy_attempt <- read_csv(here('data', '05-prep2-messy-clinic.xlsx'),
  show_col_types = FALSE)
```

```
Multiple files in zip: reading '[Content_Types].xml'
```

```
dim(messy_attempt)
```

```
[1] 1 1
```

```
substr(names(messy_attempt), 1, 50)
```

```
[1] "<?xml version=\"1.0\" encoding=\"UTF-8\" standalone=\"y"
```

No error, but nothing usable either: one column, zero rows, and a column name that's a fragment of raw XML. An `.xlsx` file is actually a small zip archive of several XML files bundled together. `read_csv()` has gone looking inside that archive and started reading one of those XML files as if it were plain text. This is the quietest kind of import failure there is: nothing crashes, nothing complains, and the result still looks like a data frame at a glance. Always check what actually came in, which is exactly what the next section is for.

Put right, all four attempts above collapse into the one call we actually want, and this is the real import we use for the rest of this book. But first, notice what we're about to leave behind: four patients, hand typed in Chapter 1, were only ever enough to practise R's syntax on. No real public health question gets answered from four people. The file we're about to read holds `nhanes_mph` instead, an extract of a large American health survey in which adults were interviewed and then given a physical examination, tens of thousands of them rather than four. Chapter 3 introduces it properly. For today we only need it to be a real file with real data in it, at the scale our toy log never had:

```
nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
```

We don't necessarily need to use the `here` package. We can use an absolute or a relative path instead, as below. However, if the RStudio project is set up as recommended, using `here` or a relative path is easier to work with.

```
nhanes <- read_csv('./data/mph_nhanes_20241107.csv')
```

```
nhanes <- read_csv('C:/Users/yourname/Documents/ies_mph/data/mph_nhanes_20241107.csv')
```

An absolute path like that one only works on the machine it was written on, since it names a specific person's folder by name. `here()` avoids exactly that problem. It finds the project folder wherever it happens to sit. That's why this book uses it throughout, and why the restart test later in this chapter only works if our script uses `here()` instead of a path like the one above.

2.5.3 The block of output we'll see

Run that line and `read_csv()` prints eight lines the moment it finishes. This book hides them, because one of them is too wide to fit the page, so here they are as they'll appear in your console:

```
Rows: 41765 Columns: 27
  Column specification
Delimiter: ','
chr  (3): smoking, phq.c, lcod
dbl (24): SEQN, age, gender, eth, edu, alc_wk, vpa_w, ...

  Use `spec()` to retrieve the full column specification
   for this data.
  Specify the column types or set `show_col_types = FALSE`
   to quiet this message.
```

It looks like something went wrong. Nothing did. This is a **message**, not a warning and not an error, and it appears every single time `read_csv()` reads a file successfully. Read it once, properly, because it's the first thing we know about a dataset we've never opened.

`Rows: 41765 Columns: 27` is the size of what came in. That's the same fact `dim()` gives us below, arriving before we ask for it. If a file we expected to have a few hundred rows reports three, we've learned that here instead of ten minutes later.

`Delimiter: ','` is `read_csv()` confirming it split each line at commas, which is what the 'comma' in 'comma separated values' means, and what we saw for ourselves earlier in this chapter when we opened a CSV in a text editor.

Then the column specification: `chr (3)` lists three columns read as character, that is text, and `dbl (24)` lists twenty-four read as numbers. `dbl` is short for 'double', which is R's ordinary way of storing a number that can have decimal places. This is `read_csv()` telling us what it *guessed*, one column at a time, and that guess is the thing worth checking. Notice `smoking` and `phq.c` sitting in the character list, and `gender` and `eth` sitting in the number list, and hold onto that: it comes back in the next section but one.

The two lines beginning with an   are `read_csv()` offering a way to see the full specification, and a way to make the whole message go away. That second one is `show_col_types = FALSE`, the argument we used on the broken `.xlsx` import above. Now we know what it silences. Use it when we already know what's in a file and don't want the report again, not to make untidy output go away. 'Untidy output' and 'the only warning we were going to get' often look identical.

2.5.4 A first look

The message already told us the size and the column types. Four functions let us check that against what we actually have.

```r
dim(nhanes)
```

```
[1] 41765    27
```

```r
names(nhanes)
```

```
 [1] "SEQN"     "age"      "gender"   "eth"      "edu"      "smoking"
 [7] "alc_wk"   "vpa_w"    "mpa_w"    "wal_r"    "vpa_r"    "mpa_r"
[13] "mpa_t"    "vpa_t"    "met_w"    "met_r"    "met_t"    "sed"
[19] "chol"     "phq"      "phq.c"    "blm.cvd"  "dth"      "lcod"
[25] "ucod_dm"  "ucod_ht"  "fu_month"
```

`dim()` gives rows, then columns. `names()` lists every column name, in order, useful for checking a name before we type it.

```r
head(nhanes)
```

```
# A tibble: 6 x 27
   SEQN   age gender   eth   edu smoking   alc_wk vpa_w mpa_w wal_r vpa_r
  <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
1    12    37      1     3     3 Never      0.460    NA    NA    NA    NA
```

```
2    20    23       2     1     1 Previous  3         NA    NA    NA    NA
3    34    38       2     4     2 Current   1         NA    NA    NA    NA
4    66    37       1     1     3 Never    NA         NA    NA    NA    NA
5    69    27       1     4     3 Never     2         NA    NA    NA    NA
6    81    30       1     3     3 Never     1         NA    NA    NA    NA
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <chr>, blm.cvd <dbl>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

head() prints the first six rows, so we can eyeball what real values actually look like.

```r
summary(nhanes$age)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  18.00   31.00   46.00   47.52   63.00   85.00 
```

summary(), run on one column, gives its minimum, maximum, quartiles, and mean in one line, plus a count of missing values if there are any. Run on the whole data frame instead of one column, it repeats this for every column at once, which is a fast way to eyeball the whole dataset in one go.

Look at gender and smoking in the output above. gender is a plain number, 1 or 2, standing in for a category. smoking is already text, 'Never', 'Previous', or 'Current'.

2.5.5 When R guesses the wrong type

read_csv() decides each column's type by looking at the values in it. Most of the time that guess is right. It goes wrong in exactly the situation the messy spreadsheet exercise built on purpose: a column that's plain numbers for every patient except one, who was recorded as '195 lbs' instead of a number in kilograms. R has no way to know that 'lbs' should be split off, converted, and the rest treated as a number. It just sees text and stores the whole column as text, one stray row is all it takes. This is why the clean version of that spreadsheet needed every patient converted to the same unit, with that unit moved into the column *name* (weight_kg) and out of every individual *value*: it's the only way read_csv() ends up with a column it can actually calculate on.

2.5.6 Spotting missing data

`is.na()` checks each value in a vector and returns `TRUE` where it's missing, `FALSE` where it isn't. `sum()` on that result counts the `TRUE`s, since R treats `TRUE` as 1 and `FALSE` as 0 when we add them up.

```
sum(is.na(nhanes$vpa_w))
```

```
[1] 7354
```

That's how many participants have no value recorded for `vpa_w`, vigorous physical activity at work, out of 41,765 participants in total. `nrow()`, from base R, returns just the row count on its own, the same number `dim()` gives as its first value above, useful whenever we want the total on its own rather than alongside the column count. It's a real gap, not a rounding artefact, and it matters. Whatever we go on to calculate from `vpa_w` only ever describes the participants who actually have a value. We should say so out loud whenever we report it, instead of letting it pass silently.

2.5.7 Using the dictionary file

`nhanes_mph` comes with a dictionary which names every column and states what it means. Reading it prints the same column specification message we just unpacked, saying two columns of text this time instead of twenty-seven columns of mixed types. This book hides it from here on, since it says the same kind of thing every time and we've now read one properly. You'll still see it in your own console, and it's still worth a glance:

```
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
dictionary
```

```
# A tibble: 27 x 2
   `Variable names` Description
   <chr>            <chr>
 1 SEQN             Participant identifier
 2 age              Age in years
 3 gender           Gender (1=Male; 2=Female)
 4 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Blac~
 5 edu              Educaton (1=Less than high school; 2=high school gradua~
 6 smoking          Smoking status (Never/Previous/Current)
 7 alc_wk           Alcohol drinking per week
 8 vpa_w            Vigourous physical activity (hour/day) at work
```

```
 9 mpa_w            Moderate physical activity (hour/day) at work
10 wal_r            Walking (hour/day) outside work
# i 17 more rows
```

The dictionary's first column is called `Variable names`, with a space in it, so referring to it needs backticks, exactly as Chapter 1 covered. `%in%`, from the same chapter, tests whether each value is a member of a set. Put inside square brackets after a data frame's column, with the row's index implied, this keeps only the rows where the condition is true:

```
dictionary[dictionary$`Variable names` %in% c('age', 'chol', 'smoking'), ]
```

```
# A tibble: 3 x 2
  `Variable names` Description
  <chr>            <chr>
1 age              Age in years
2 smoking          Smoking status (Never/Previous/Current)
3 chol             Total cholesterol (mmol/L)
```

That's the habit to build now: when a column's meaning isn't obvious from its name, the answer lives in this file, not in a guess. Chapter 3 uses this same file to build proper factors out of six of these columns; for today, just knowing how to look one up is enough.

2.6 A first analysis, and your first script

2.6.1 One summary statistic

```
mean(nhanes$chol, na.rm = TRUE)
```

```
[1] 4.973316
```

`chol` is total cholesterol, in mmol/L, and 2,589 participants have no value recorded for it. `na.rm = TRUE` drops those missing values before averaging the rest, exactly as it did back in Chapter 1's `mean()` example on our own clinic log. Leave it out and the whole calculation returns `NA`, since R won't silently guess what a missing value should have been.

2.6.2 A first taste of a grouped summary

`nhanes$chol` pulls the whole cholesterol column out as a bare vector, as `$` did in Chapter 1. Adding square brackets straight after it, with a condition inside, keeps only the values where that condition is `TRUE`:

```
summary(nhanes$chol[nhanes$smoking == 'Never'])
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NAs
  1.780   4.220   4.890   4.968   5.610  15.830   1839
```

```
summary(nhanes$chol[nhanes$smoking == 'Current'])
```

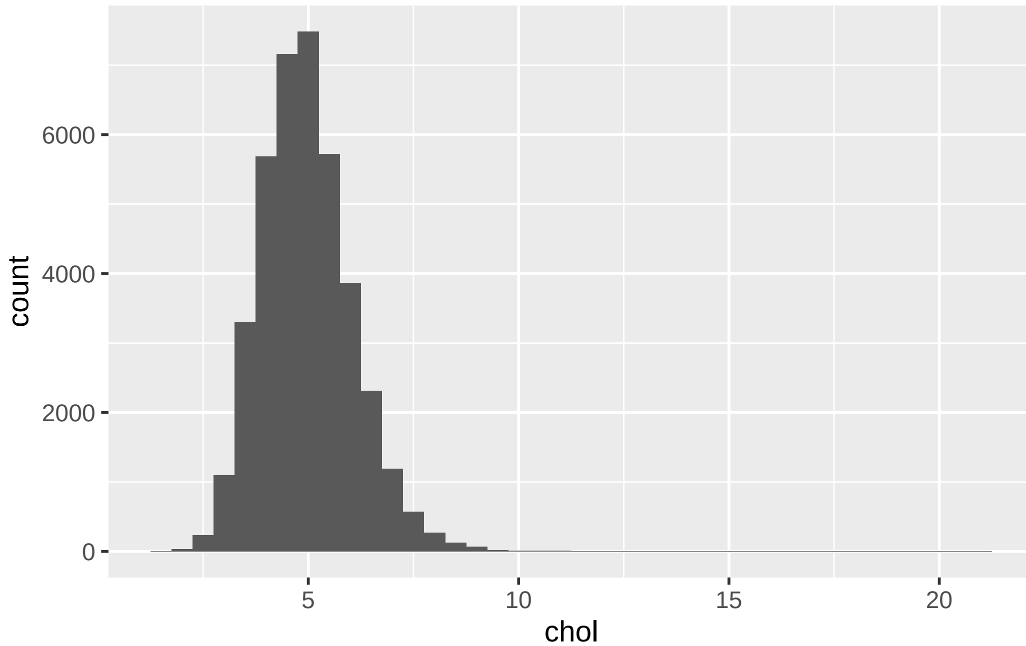
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NAs
  1.530   4.220   4.890   4.996   5.660  15.900    882
```

That's a genuine grouped summary, built entirely from tools already familiar: `$`, `==`, and now `[` and `]` with a condition inside it. It works, but it's clunky already with just two groups, and `smoking` has three. Chapter 3 introduces `group_by()` and `summarise()`, a far nicer way to do exactly this for as many groups as we like, in one line instead of several. This is only a preview of what's coming, not the tool we'll actually use going forward.

2.6.3 A first taste of a plot

```
library(ggplot2)

ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5)
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```



`ggplot2` code is joined with `+`, not the pipe. That's a real inconsistency in R, not a mistake in this book, and Chapter 3 explains why and unpacks the rest of the grammar properly. For today, just notice the shape: cholesterol piles up around 4 to 6 mmol/L, with a long tail of higher values trailing off to the right.

2.6.4 That warning above the plot

The plot came with a line starting `Warning:`, saying it removed 2,589 rows. Read it, because we'll see this one a lot: here's what it actually says, and when it matters.

Firstly, a warning is not an error. An error means R gave up and produced nothing. A warning means R did the job and is telling us something about how. The plot is there and it's fine.

Secondly, look at the number. It's 2,589, exactly the count of participants with no cholesterol value that we found a moment ago. A histogram puts each participant somewhere along the x-axis, and a missing value has nowhere to go, so `ggplot2` left those participants out and told us how many it dropped.

Why we leave these warnings switched on. R can be told to hide them, and plenty of people do, because they look untidy. This book doesn't, and neither should we for now. That warning is a free count of how much of our data a plot is actually based on. Hide it, and a plot built on half our dataset looks exactly like a plot built on all of it. It's the same reason we write `na.rm = TRUE` deliberately and say what it dropped, rather than typing it out of habit until it stops registering.

When it does need attention. The number is the thing to check, not the warning itself. Compare it against what we expect, the way we just did with `is.na()`. If it matches, carry on. If it's different than expected, stop and find out why before trusting anything the plot shows: usually it means the wrong column, an import that didn't go as planned, or an earlier step that threw away more rows than intended. And whenever we report a result, we should say how many observations it's based on and how many were missing. A warning R printed for us is not the same as a reader being told.

2.6.5 From console to script

Everything we've done so far in this chapter has been one line at a time. That's the right way to *try* something: type it in the console, look at what comes back, fix it if it's wrong. But Chapter 1 already made the argument against leaving it there. We will need to do this again, and we will not remember what we typed.

So the working habit for the rest of this course is two steps, not one. Try the line in the console until it does what we want. Then move the line that worked into a script, and save it. The console is where we experiment; the script is what we keep.

To build one now:

1. In RStudio, go to **File → New File → R Script**, or key in **Ctrl+Shift+N** or **Cmd+Shift+N**. A blank file opens in the source pane, top-left.
2. Type the lines you want to keep. For a first script, that's the `library()` calls, the `read_csv()` line, one summary, and one plot.
3. Save it into the `scripts/` folder you built in Chapter 1, with a name that follows its conventions: no spaces, no `&` or `#`, something that says what it does. `first_analysis.R` is fine. `Untitled1.R` is not.

Here's what a complete first script looks like, start to finish. Nothing in it is new; it's the pieces from earlier in this chapter, collected in the order they need to run:

```r
library(readr)
library(here)
library(ggplot2)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))

mean(nhanes$chol, na.rm = TRUE)
sum(is.na(nhanes$chol))

ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5)
```

Notice the shape of it. The `library()` calls come first, because nothing below them works until the packages are loaded. The `read_csv()` line comes next, because every line after it needs `nhanes` to exist. A script runs top to bottom, so the order isn't decoration, it's the thing that makes it work.

2.6.6 Proving it actually works

Here's the part people skip, and it's the part that matters. A script that runs on your machine right now, in a session where you've been typing for an hour, is not yet proof of anything. Half of what it needs might already be sitting in memory from something you typed and forgot.

So test it properly. In RStudio, go to **Session → Restart R**. That throws away every object and every loaded package, leaving you with an empty session, exactly like the one a colleague, a marker, or you-in-six-months would start from. Now run your script from the top.

If it runs clean, the script is genuinely self-contained. If it fails, the error tells us precisely what we'd forgotten to write down, and the fix belongs *in the script*, not typed into the console again. That restart is the reproducibility argument from Chapter 1, made concrete: not 'scripts are good practice' but 'here is how we find out whether ours actually is one.'

That's the script this whole chapter has been building toward, starting all the way back with a merged title row that wouldn't import cleanly: not just an import that runs once, but one a stranger, or you in six months, could pick up cold and trust. Chapter 3 starts from exactly this point, with `nhanes` already read in and ready.

2.7 Exercises

Exercise 1 (ILO 2). This `read_csv()` call is supposed to read the dictionary file but doesn't work:

```r
read_csv(here('data', 'mph_nhanes_dictionary.csv'))
```

Work out what's wrong with the path and write the corrected call.

Exercise 2 (ILO 3). Check `mpa_w` (moderate physical activity at work): find its type with `class()`, and count how many values are missing with `is.na()` and `sum()`.

Exercise 3 (ILO 4). Produce one summary statistic and one plot for `wal_r` (walking, hours/day, outside work). Remember `na.rm = TRUE` if it turns out to have missing values.

Exercise 4 (ILO 4). Collect your answer to Exercise 3 into a proper script. Open a new R script, and write into it everything needed to go from a completely empty session to that summary and that plot: the `library()` calls, the `read_csv()` line, then the summary and the plot themselves. Save it into your `scripts/` folder with a sensible name.

Then test it the way this chapter described. Go to **Session → Restart R**, and run your script from the top. If it errors, read the message, work out what the fresh session was missing, and add that line to the script rather than typing it into the console. Repeat until it runs clean from a restart. That is the point of the exercise, not the summary itself.

2.8 Self-check

Five questions on the concepts this chapter leans on hardest: paths, file types, and reading a real import problem. Answers are in the Solutions section below.

1. `read_csv(here('data', 'mhp_nhanes_20241107.csv'))` fails, even though `mph_nhanes_20241107.csv` really is in the `data` folder. Which of the four import failures is this?

a) The file is the wrong type
b) A path pasted with backslashes
c) A wrong filename, here a typo (`mhp` instead of `mph`)
d) The file is in the wrong folder

2. What's the actual difference between `dim(nhanes)` and `names(nhanes)`?

a) They return exactly the same information in two formats
b) `dim()` gives the number of rows and columns; `names()` lists every column name
c) `names()` gives row and column counts; `dim()` lists column names
d) `dim()` only works after `names()` has been run first

3. `sum(is.na(nhanes$vpa_w))` returns `7354`. What does that number actually mean?

a) The sum of all values of `vpa_w`
b) 7,354 participants have `vpa_w` recorded as exactly zero
c) 7,354 participants have no value recorded for `vpa_w`
d) There are 7,354 participants in the dataset in total

4. `read_csv()` is used to read both `mph_nhanes_20241107.csv` and `mph_nhanes_20241107_dictionary.csv`, in two separate calls. Why two calls, and not one?

a) `read_csv()` can only read one row at a time
b) They're two separate files, each holding a different table, so each needs its own call
c) The dictionary file is not really needed, but the call is included anyway
d) `read_csv()` automatically finds and reads both once you name one of them

5. Reading `05-prep2-messy-clinic.xlsx` with `read_csv()` produces a data frame with 1 column and 0 rows, and no error message. What does that tell you?

a) The file is empty
b) `read_csv()` refuses to read `.xlsx` files and stops immediately
c) `.xlsx` is the wrong file type for `read_csv()`, and the lack of an error is exactly why you always check what actually came in
d) The import worked correctly and the file genuinely has one column

2.9 Writing exercise

Using the mean and the histogram from Exercise 3, write a short paragraph, about 60 words, describing `wal_r` as you would in the results section of a report. State the mean and its units, say how many participants have no value recorded, and describe the shape of the distribution the histogram shows. This is the smallest version of a habit the rest of this book builds on: a number on its own, with no sentence around it, is not yet a result.

Solutions

Spreadsheet exercise
The one problem not named in the list above is colour used to carry information: a block of rows is filled in yellow to mean 'smoker', with no `Smoker` column recording it at all, and the only place that says what the colour means is a caption sitting several rows away, disconnected from the table itself. R reads cell values, not cell fills, so the colour disappears completely on import, and the caption comes in as an isolated row of text with nothing left to connect it back to the patients it was describing.
The fixes needed, matching every problem in the sheet:

- Split the merged title row and the merged 'Measurements' sub-header back into one header row naming every column individually.
- Convert every patient's weight and height to the same units, kilograms and metres. Then move those units out of the values and into the column names instead (`weight_kg`, `height_m`), leaving plain numbers behind.
- Add a proper `Smoker` column holding an actual word ('Yes'/'No'), decoded from which rows were highlighted and what the caption said the highlighting meant.
- Delete both blank spacer rows.
- Delete or relocate the stray note about Patient 5's missed follow-up. It belongs in its own column, or nowhere in this table at all.
- Either delete the second table (the follow-up schedule) or save it as its own, separate CSV file. One sheet should hold one table.

Exercise 1

```
read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

```
# A tibble: 27 x 2
   `Variable names` Description
   <chr>            <chr>
 1 SEQN             Participant identifier
 2 age              Age in years
 3 gender           Gender (1=Male; 2=Female)
 4 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Blac~
 5 edu              Educaton (1=Less than high school; 2=high school gradua~
 6 smoking          Smoking status (Never/Previous/Current)
 7 alc_wk           Alcohol drinking per week
 8 vpa_w            Vigourous physical activity (hour/day) at work
 9 mpa_w            Moderate physical activity (hour/day) at work
10 wal_r            Walking (hour/day) outside work
# i 17 more rows
```

The folder and filename were both fine. The problem was the letters `mph` typed as `mhp`.

Exercise 2

```r
class(nhanes$mpa_w)
```

```
[1] "numeric"
```

```r
sum(is.na(nhanes$mpa_w))
```

```
[1] 7391
```

Exercise 3

```r
mean(nhanes$wal_r, na.rm = TRUE)
```

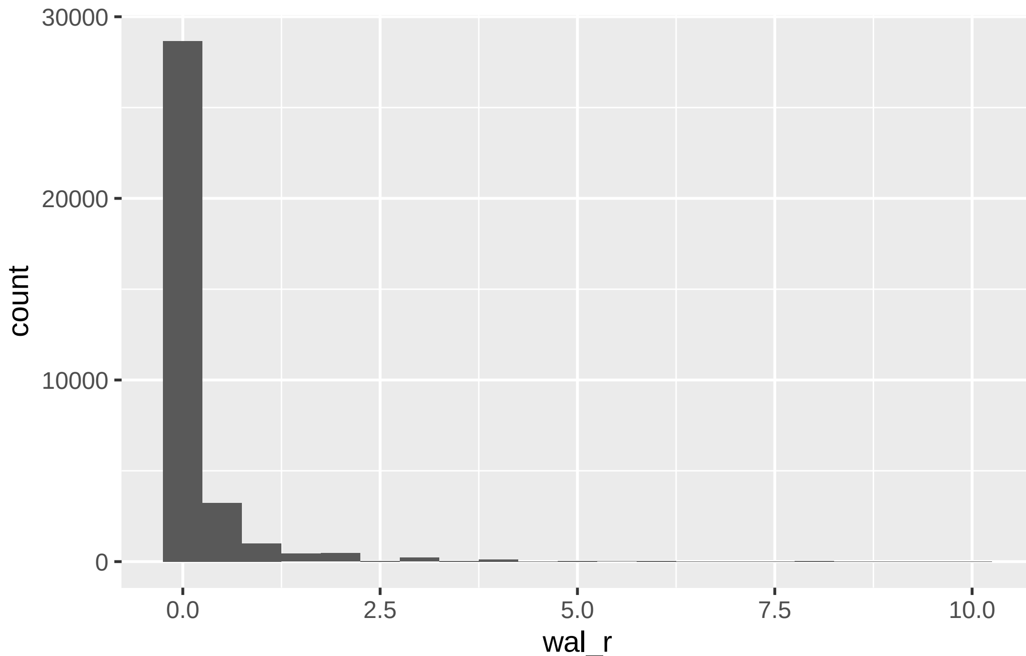
```
[1] 0.2060187
```

```r
sum(is.na(nhanes$wal_r))
```

```
[1] 7347
```

```r
library(ggplot2)
ggplot(nhanes, aes(x = wal_r)) +
  geom_histogram(binwidth = 0.5)
```

```
Warning: Removed 7347 rows containing non-finite outside the scale range
(`stat_bin()`).
```



Exercise 4
The finished script, in full. Saved as `scripts/wal_r_analysis.R`:

```
library(readr)
library(here)
library(ggplot2)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))

mean(nhanes$wal_r, na.rm = TRUE)
sum(is.na(nhanes$wal_r))

ggplot(nhanes, aes(x = wal_r)) +
  geom_histogram(binwidth = 0.5)
```

The two lines people most often leave out are the ones that were already true before they started: `library(readr)` and the `read_csv()` call. Both had been run earlier in the session, so `nhanes` existed and `read_csv()` worked. Neither felt like part of 'the analysis.' After a restart, neither is true any more, and the script fails on its first real line with `could not find function 'read_csv'` or `object 'nhanes' not found`, both of which Chapter 1 covered. That's exactly what the restart is for: it finds the assumptions we didn't know we were making.

Self-check answers

1. **c.** `mhp` versus `mph` is a plain typo in the filename, not a folder problem, a file-type problem, or a backslash problem.
2. **b.** `dim()` returns row and column counts. `names()` returns the column names themselves. Neither substitutes for the other.
3. **c.** `is.na()` marks each missing value `TRUE`, and `sum()` adds those up. The number is a count of participants with no recorded value, nothing about the values that are present.
4. **b.** `mph_nhanes_20241107.csv` and `mph_nhanes_20241107_dictionary.csv` are two separate files on disk, each holding a different table, so each needs its own `read_csv()` call.
5. **c.** `.xlsx` is a zip archive of XML files, not a CSV, so `read_csv()` produces nonsense rather than a real dataset. The absence of an error is exactly the trap: always check what actually came in after reading a file, not just whether it 'worked.' `dim()`, used earlier in this chapter, is one way to do that. Chapter 3 introduces a second, `glimpse()` from `dplyr`, which shows every column with its type and first few values in one compact layout, better suited to a wide dataset like `nhanes_mph` than `dim()` alone.

Writing exercise (sample answer)

Participants reported a mean of 0.21 hours/day of walking outside work (`wal_r`), based on the 34,418 participants with a value recorded, with 7,347 participants who had a missing value. The distribution is heavily right-skewed: most participants reported no walking outside work at all, and a small number reported several hours a day, pulling the mean above what a typical participant's value actually looks like.

3 Describing and visualising data (Week 7)

Scrolling through a dataset of tens of thousands of participants tells us almost nothing. Twenty rows we can read; twenty thousand we can't hold in our head at all. Chapter 2 got a dataset of exactly that size into R and confirmed every column had arrived, which isn't the same as knowing what any of it looks like. Seeing that takes summary statistics and graphs, and building them is the work of this chapter.

Which statistic is the right one depends entirely on the kind of variable in front of us. A mean and an SD describe a continuous variable and say nothing sensible about a categorical one, where a count and a percentage do the job instead. So the types come first here, and the summaries, the plots, and the choice of what to show all follow from getting them right. The chapter's destination is a 'Table 1', the participant-characteristics table that opens most published health research, along with the paragraph of prose that has to go with it.

3.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Classify a variable as nominal, ordinal, binary, count, or continuous, and distinguish its statistical type from the class R has stored it as
2. Use `dplyr`'s `select()`, `filter()`, `mutate()`, and `arrange()` to manipulate a dataset, and produce grouped summaries with `group_by()` and `summarise()`
3. Construct frequency and cross-tabulations using `table()` and `prop.table()`
4. Build histograms, boxplots, bar charts, and scatterplots using `ggplot2`
5. Modify a plot's bin width, scale, and labels, and evaluate the effect on interpretation
6. Convert a character variable to a factor using `factor()` and set its reference level using `relevel()`
7. Produce a basic 'Table 1' of participant characteristics using `gtsummary::tbl_summary()`, and describe it in writing

3.2 The data: `nhanes_mph`

NHANES is the National Health and Nutrition Examination Survey, run by the National Center for Health Statistics, part of the US Centers for Disease Control and Prevention. It's designed to describe the health of the US population, and it's unusual in that it doesn't just ask people questions. Participants are interviewed at home and then examined in a mobile clinic, where measurements are taken and blood samples drawn. That's why our data can carry a real measured cholesterol value rather than just a self-report question.

`nhanes_mph` is a curated extract of it, prepared specifically for this course: 41,765 adults aged 18 to 85, described by 27 variables. Those cover demographics, education, smoking and alcohol, physical activity split by domain, sedentary time, total cholesterol, depression symptoms, and whether the participant died during follow-up. The full list is in the dictionary file, which we read in alongside the data below and use throughout.

3.2.1 One simplification

NHANES is not a simple random sample of Americans. It's a **complex survey**. Some groups are deliberately sampled more heavily than others in certain years, and participants are recruited in geographic clusters rather than one at a time. Every participant also carries a survey weight, saying how many people in the population they stand for. Ideally, all of that would go into the calculation, to get an estimate representative of the US population. This course treats `nhanes_mph` as though it were a simple cohort instead: every participant counts once, and counts the same. That's a real simplification, and it means the numbers we produce here would not be the right numbers to publish as population estimates. Survey weights will not be covered anywhere in the MPH curriculum, not deferred to Advanced Statistics.

3.3 Variable types

Every variable falls into one of two broad kinds, categorical and numeric.

A **categorical** variable puts each participant into one of a fixed set of groups. `gender` and `eth` are **nominal**: the categories have no natural order. `phq.c`, depression classification, is **ordinal**: normal, mild symptoms, and possible depression run in a genuine order, even though the gap between them isn't a measured distance. `blm.cvd`, baseline cardiovascular disease, is **binary**: a two-category special case of categorical, present or absent, even though in the data it takes the value 0 or 1.

A **numeric** variable is a number, and numbers come in two shapes that matter differently for analysis. `phq`, the raw depression symptom score behind `phq.c`, is a **count** (sometimes called discrete): whole numbers only, 0 through 6. `chol`, `age`, and `sed` are **continuous**: any value in a range is possible, 4.7 mmol/L as easily as 4.71.

The variable type decides the summary and the plot. A mean and an SD, or a median and an IQR, describe a **continuous** variable. A **categorical** variable, however few its categories, is summarised as a proportion or a frequency table instead: for a binary variable like `blm.cvd`, the proportion of 1s and the mean of its 0/1 values come out as exactly the same number, but we still call it a proportion, since what it summarises is a category, not a measurement. This book says *continuous* from here on when that's specifically what's meant, not the vaguer *numeric*, since numeric alone doesn't say whether a variable is a count or continuous.

3.4 Setup

As in Chapter 2, we read the data with `read_csv()` from the `readr` package. We build the file path with `here()` from the `here` package. This builds a path from the project folder down to the file, so the code runs on any machine no matter where the project sits. `nhanes_mph` arrives as two files: the data, and a separate dictionary that describes every variable. Both are plain CSV files, so we read both with `read_csv()`.

This chapter loads one package new to the book: `gtsummary`, for building tables. `dplyr`, for manipulating data, was loaded back in Chapter 1, and `ggplot2`, for plotting, in Chapter 2, so both are already familiar. Installing and loading it are still the two separate actions Chapter 1 distinguished, so if `gtsummary` isn't installed already, run this once in the console. You don't need to put it in the script.

```
install.packages('gtsummary')
```

After that, the `library()` calls below are all we need, this session and every session after it.

```
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

Running that in your own console prints more than it does on this page. The `library()` calls announce themselves, and each `read_csv()` prints the column specification message Chapter 2 unpacked. This book hides both from here on, as both have been explained.

`glimpse()`, from `dplyr`, shows every column, its type, and its first few values. It's useful for checking what actually came in, rather than what we expected to come in.

```
glimpse(nhanes)
```

```
Rows: 41,765
Columns: 27
$ SEQN     <dbl> 12, 20, 34, 66, 69, 81, 97, 107, 120, 143, 158, 167, 170,~
$ age      <dbl> 37, 23, 38, 37, 27, 30, 32, 30, 30, 22, 29, 32, 28, 31, 3~
$ gender   <dbl> 1, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 2, 1, 2, 2, 1, 1, 2, 2, ~
$ eth      <dbl> 3, 1, 4, 1, 4, 3, 4, 1, 3, 1, 1, 2, 1, 4, 3, 3, 2, 3, 1, ~
$ edu      <dbl> 3, 1, 2, 3, 3, 3, 3, 2, 3, 1, 1, 3, 1, 1, 2, 3, 1, 2, 1, ~
$ smoking  <chr> "Never", "Previous", "Current", "Never", "Never", "Never"~
$ alc_wk   <dbl> 0.46029919, 3.00000000, 1.00000000, NA, 2.00000000, 1.000~
$ vpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ wal_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sed      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ chol     <dbl> 4.03, 3.75, 5.04, 5.56, 5.66, 8.90, NA, 3.57, 6.57, 6.67,~
$ phq      <dbl> 0, 0, NA, 0, 0, 0, 0, 0, 0, 0, 0, NA, 0, NA, 2, 0, 0, 0, ~
$ phq.c    <chr> "0 Normal", "0 Normal", NA, "0 Normal", "0 Normal", "0 No~
$ blm.cvd  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ dth      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ lcod     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "hrt", NA, NA, NA, NA~
$ ucod_dm  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ ucod_ht  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ fu_month <dbl> 236, 245, 244, 238, 245, 243, 242, 243, 247, 231, 235, 24~
```

Six columns describe a category but aren't usable as one yet: `gender`, `eth`, `edu`, `smoking`, `phq.c`, and `blm.cvd`. `glimpse()` reports how R has **stored** each column, which is a different question from what the variable statistically **is**. `blm.cvd` is binary, so categorical, but R read its 0s and 1s as numbers and lists it as `<dbl>`; `gender` and `edu` are the same, plain integers standing in for a label. `smoking` and `phq.c` are categorical too, just stored as text: `glimpse()` lists both as `<chr>`, not numbers at all. Whichever way R happened to store a column, the variable's statistical type doesn't change, and R can't infer one from the other just by looking at it. Whatever the raw form, we should consult the data documentation, e.g. the data dictionary, whenever possible.

```r
dictionary |>
  filter(`Variable names` %in% c('gender', 'eth', 'edu', 'smoking', 'phq.c', 'blm.cvd'))
```

```
# A tibble: 6 x 2
  `Variable names` Description
  <chr>            <chr>
1 gender           Gender (1=Male; 2=Female)
2 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Black~
3 edu              Educaton (1=Less than high school; 2=high school graduat~
4 smoking          Smoking status (Never/Previous/Current)
5 phq.c            Depression classification - PHQ-2 (0 Normal/1 Mild sympt~
6 blm.cvd          Baseline CVD (0=No; 1=Yes)
```

The dictionary gives a numeric code against each category for `gender`, `eth`, `edu`, and `blm.cvd`: `1=Male; 2=Female`, and so on. Those are the four we have to translate. For `smoking` and `phq.c` it just lists the categories, and `glimpse()` already told us why no translation is needed: they don't arrive as bare numbers at all. `smoking` is already the text 'Never', 'Previous', or 'Current'. `phq.c` is already a label too, with the original code number kept at the front of the string, for example `'0 Normal'`. The dictionary isn't exhaustive, either. `dth`, the death indicator we use in Chapter 7, has not had a coding given to it. But by programming convention, `0` is no (or the logical `FALSE`), and `1` is yes (or the logical `TRUE`).

We turn every one of the six into a factor with `mutate()` and `factor()`, whether the raw values are numbers or text. `factor()` takes the raw values (`levels`) and the labels we want to give them (`labels`), in matching order. The labels are ours to choose, and we take one small liberty below: the dictionary abbreviates `eth`'s first category to 'Mexican', and we write it out as 'Mexican American', the full NHANES category name. One of the six, `phq.c`, is ordinal: its categories run from normal to possible depression in a natural order, so we build it with `ordered = TRUE`. `gender`, `eth`, `smoking`, and `blm.cvd` are nominal, with no natural order, so they're plain factors for the more familiar reason. By default, the first level listed becomes the *reference* level, the category everything else gets compared against. The next section says more about what that means and why it's set deliberately.

```r
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c('Male', 'Female')),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c('Mexican American', 'Other Hispanic', 'White', 'Black', 'Others')),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c('Less than high school', 'High school graduate', 'Above high school')),
    smoking = factor(smoking,
      levels = c('Never', 'Previous', 'Current')),
```

```r
    phq.c = factor(phq.c,
      levels = c('0 Normal', '1 Mild symptoms', '2 Possible depression'),
      labels = c('Normal', 'Mild symptoms', 'Possible depression'),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c('No', 'Yes'))
  )

glimpse(nhanes)
```

```
Rows: 41,765
Columns: 27
$ SEQN     <dbl> 12, 20, 34, 66, 69, 81, 97, 107, 120, 143, 158, 167, 170,~
$ age      <dbl> 37, 23, 38, 37, 27, 30, 32, 30, 30, 22, 29, 32, 28, 31, 3~
$ gender   <fct> Male, Female, Female, Male, Male, Male, Male, Female, Fem~
$ eth      <fct> White, Mexican American, Black, Mexican American, Black, ~
$ edu      <fct> Above high school, Less than high school, High school gra~
$ smoking  <fct> Never, Previous, Current, Never, Never, Never, Previous, ~
$ alc_wk   <dbl> 0.46029919, 3.00000000, 1.00000000, NA, 2.00000000, 1.000~
$ vpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ wal_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sed      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ chol     <dbl> 4.03, 3.75, 5.04, 5.56, 5.66, 8.90, NA, 3.57, 6.57, 6.67,~
$ phq      <dbl> 0, 0, NA, 0, 0, 0, 0, 0, 0, 0, 0, NA, 0, NA, 2, 0, 0, 0, ~
$ phq.c    <ord> Normal, Normal, NA, Normal, Normal, Normal, Normal, Norma~
$ blm.cvd  <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, No, N~
$ dth      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ lcod     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "hrt", NA, NA, NA, NA~
$ ucod_dm  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ ucod_ht  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ fu_month <dbl> 236, 245, 244, 238, 245, 243, 242, 243, 247, 231, 235, 24~
```

The blank and missing values in `smoking`, `edu`, `phq.c`, and `blm.cvd` have become `NA` in the process, just as they were before recoding. `factor()` relabels the values that are present and carries missingness through unchanged. From here on, every chapter opens with a short setup chunk that repeats these same steps, so each chapter stands on its own.

3.5 Checking and setting a reference level

The reference level decides which category a table or a chart reads everything else against. It's a reporting choice before it's a modelling one, so we set it deliberately from here, not only once a categorical predictor enters a regression model in Chapter 6. Let's examine the levels in a factor. `levels()`, from base R, lists a factor's categories in their current order:

```
levels(nhanes$smoking)
```

```
[1] "Never"    "Previous" "Current"
```

`'Never'` is first, so `'Never'` is the reference. Everything else gets read against it: 'Previous' means previous compared with never, and 'Current' means current compared with never. That's the sensible choice here, and it isn't an accident. It's what we asked for when we wrote `levels = c('Never', 'Previous', 'Current')` in the recode above. Change the order in that line and we change the reference.

Sometimes we want a different reference without rebuilding the factor. `relevel()`, also from base R, takes a factor and moves the level named in `ref` to the front:

```
eth_white_ref <- relevel(nhanes$eth, ref = 'White')
levels(eth_white_ref)
```

```
[1] "White"            "Mexican American" "Other Hispanic"
[4] "Black"            "Others"
```

`'White'` has moved to the front, so it would now be the reference, with the other four groups read against it. The rest keep their order, just shifted along.

Notice we assigned the result to a new object, `eth_white_ref`, rather than writing it back into `nhanes`. That's deliberate. Every chapter in this book rebuilds `nhanes` with the same recode. If we changed this factor here, this chapter's `nhanes` would stop matching the one later chapters build, and a table or model would come out different for no visible reason. When we're demonstrating something, we work on a copy.

Right now the reference level only decides the order categories appear in a table, or along a bar chart's axis. Once a categorical predictor enters a regression model in Chapter 6, it decides what every coefficient means.

3.6 Manipulating a dataset: the dplyr verbs

A public health question is almost never about everyone in a dataset at once. Four verbs from `dplyr` cover most of what we need to get from the raw dataset down to the subgroup, the columns, and the order a question actually asks about.

`select()` keeps only the named columns:

```
nhanes |> select(age, gender, sed, chol)
```

```
# A tibble: 41,765 x 4
     age gender   sed  chol
   <dbl> <fct>  <dbl> <dbl>
 1    37 Male      NA  4.03
 2    23 Female    NA  3.75
 3    38 Female    NA  5.04
 4    37 Male      NA  5.56
 5    27 Male      NA  5.66
 6    30 Male      NA  8.9 
 7    32 Male      NA NA   
 8    30 Female    NA  3.57
 9    30 Female    NA  6.57
10    22 Female    NA  6.67
# i 41,755 more rows
```

`filter()` keeps only the rows that satisfy a condition. Here, that's participants under 40:

```
nhanes |> filter(age < 40)
```

```
# A tibble: 16,566 x 27
    SEQN   age gender eth      edu   smoking  alc_wk vpa_w mpa_w wal_r vpa_r
   <dbl> <dbl> <fct>  <fct>    <fct> <fct>     <dbl> <dbl> <dbl> <dbl> <dbl>
 1    12    37 Male   White    Abov~ Never    0.460     NA    NA    NA    NA
 2    20    23 Female Mexican~ Less~ Previo~  3         NA    NA    NA    NA
 3    34    38 Female Black    High~ Current  1         NA    NA    NA    NA
 4    66    37 Male   Mexican~ Abov~ Never   NA         NA    NA    NA    NA
 5    69    27 Male   Black    Abov~ Never    2         NA    NA    NA    NA
 6    81    30 Male   White    Abov~ Never    1         NA    NA    NA    NA
 7    97    32 Male   Black    Abov~ Previo~ NA         NA    NA    NA    NA
 8   107    30 Female Mexican~ High~ Never    0.0385    NA    NA    NA    NA
 9   120    30 Female White    Abov~ Previo~ NA         NA    NA    NA    NA
```

```
10   143    22 Female Mexican~ Less~ Never    0         NA    NA    NA    NA
# i 16,556 more rows
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <ord>, blm.cvd <fct>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

mutate() adds a new column, or changes an existing one, as we already saw above in the recoding step. Here it adds a new column that classifies participants as under or over 65:

```r
nhanes |> mutate(over_65 = age >= 65) |> select(age, over_65)
```

```
# A tibble: 41,765 x 2
     age over_65
   <dbl> <lgl>
 1    37 FALSE
 2    23 FALSE
 3    38 FALSE
 4    37 FALSE
 5    27 FALSE
 6    30 FALSE
 7    32 FALSE
 8    30 FALSE
 9    30 FALSE
10    22 FALSE
# i 41,755 more rows
```

arrange() sorts the rows by a column, lowest to highest by default:

```r
nhanes |> arrange(age)
```

```
# A tibble: 41,765 x 27
    SEQN   age gender eth       edu   smoking alc_wk vpa_w mpa_w wal_r vpa_r
   <dbl> <dbl> <fct>  <fct>     <fct> <fct>    <dbl> <dbl> <dbl> <dbl> <dbl>
 1 62253    18 Male   White     <NA>  <NA>     0      0    0         0 0.857
 2 62311    18 Male   White     <NA>  <NA>     0      0    0.107     0 0.214
 3 62455    18 Male   Black     <NA>  <NA>     0      0    0         0 0
 4 62598    18 Male   Black     <NA>  <NA>     0.230  0    0         0 0.429
 5 62659    18 Male   Others    <NA>  <NA>     0.192  2.86 2.86      0 4.29
 6 62833    18 Female White     <NA>  <NA>    NA      0    4         0 0
 7 62837    18 Female White     <NA>  <NA>     0.460  0    0         0 0
```

```
 8 62925    18 Female White     <NA>  <NA>     0       0     0          0 0
 9 62976    18 Female White     <NA>  <NA>     0.25    0     0.107      0 0
10 62978    18 Female Mexican ~ <NA>  <NA>     0.690   0     0          0 1.29
# i 41,755 more rows
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <ord>, blm.cvd <fct>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

These four combine in a short pipe to answer a real question. For example, who are the ten oldest current smokers in the sample?

```r
nhanes |>
  filter(smoking == 'Current') |>
  arrange(desc(age)) |>
  select(age, gender, smoking) |>
  slice(1:10)
```

```
# A tibble: 10 x 3
     age gender smoking
   <dbl> <fct>  <fct>
 1    85 Female Current
 2    85 Female Current
 3    84 Female Current
 4    84 Male   Current
 5    84 Female Current
 6    83 Male   Current
 7    82 Male   Current
 8    82 Male   Current
 9    81 Male   Current
10    81 Female Current
```

`desc()` reverses the sort order. `slice()`, also from `dplyr`, keeps only the rows at the given positions. Here, that's the first ten after sorting.

3.7 Grouped summaries

`group_by()` splits a dataset into groups without changing how it looks. `summarise()` then collapses each group down to one row of statistics. Used together, they answer questions like this: what's average cholesterol, separately for men and women?

```r
nhanes |>
  filter(!is.na(gender), !is.na(chol)) |>
  group_by(gender) |>
  summarise(
    mean_chol = mean(chol, na.rm = TRUE),
    sd_chol = sd(chol, na.rm = TRUE),
    median_chol = median(chol, na.rm = TRUE),
    q1_chol = quantile(chol, probs = 0.25, na.rm = TRUE),
    q3_chol = quantile(chol, probs = 0.75, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 2 x 7
  gender mean_chol sd_chol median_chol q1_chol q3_chol     n
  <fct>      <dbl>   <dbl>       <dbl>   <dbl>   <dbl> <int>
1 Male        4.89    1.09        4.81    4.14    5.53 18867
2 Female      5.05    1.08        4.97    4.29    5.69 20309
```

`filter()` here takes two conditions separated by a comma, which means the same as `&`: a row only stays if both are true. That's a shorter way to write `filter(!is.na(gender) & !is.na(chol))`, and it's the form used throughout this book whenever several conditions apply at once.

Four of those functions are new. `mean()` we already met in Chapter 1. `sd()`, from base R, returns the standard deviation: a measure of how spread out the values are, in the same units as the variable itself. `median()`, also base R, returns the middle value once everything is sorted, so half the participants sit below it and half above. `quantile()`, also base R, calculates any quantile we ask for: `probs = 0.25` and `probs = 0.75` ask for the first and third quartile, the two values the interquartile range sits between. There's a separate function, `IQR()`, that returns just the gap between them as a single number, in the same units as the variable itself, instead of the two quartiles. We'll use it in the worked example below.

`na.rm = TRUE` tells the functions to drop missing values before calculating. Without it, any `NA` in a column makes the whole result `NA`. The `filter()` step above drops rows where `gender` or `chol` itself is missing, so the group sizes reported in `n` reflect exactly the rows the means are calculated from. `n()`, used inside `summarise()`, counts the rows in each group.

The `gender` half of that filter turns out to change nothing: every participant in this extract has a gender recorded. The `chol` half of the same filter is doing real work: 2,589 participants have no cholesterol value, and without it they'd be counted in `n` while contributing nothing to the mean.

Whether we report the mean and SD or the median and IQR depends on shape. A skewed distribution gets misrepresented by a mean, because the mean gets pulled toward the tail. A rule of thumb is to look at the shape before choosing. We'll build the plot we need for that in a moment, but categorical variables need a different kind of summary first: a count, not a mean.

3.8 Frequency and cross-tabulations

`table()`, from base R, counts how many rows fall into each category of a variable. Given one variable, it produces a **frequency table**:

```r
table(nhanes$smoking)
```

```

   Never Previous  Current 
   23394     9468     8580 
```

`table()` drops `NA` values by default, not counting them as their own category, so this count is out of everyone with a recorded smoking status, not out of everyone in the dataset. Raw counts are hard to compare across groups of different sizes, so we usually want percentages instead. `prop.table()`, also from base R, takes a table and turns its counts into proportions of the total:

```r
table(nhanes$smoking) |> prop.table()
```

```

    Never  Previous   Current 
0.5644998 0.2284639 0.2070363 
```

Multiplying by 100 and rounding turns those proportions into percentages, in whatever way is easiest to read.

Giving `table()` two variables instead of one produces a **cross-tabulation**: a two-way table with one variable's categories as rows and the other's as columns, one cell per combination. Here's smoking status by gender:

```r
smoking_gender <- table(nhanes$smoking, nhanes$gender)
smoking_gender
```

```
         Male Female
Never    9302  14092
Previous 5704   3764
Current  4913   3667
```

The rows are `smoking`, the columns are `gender`, because that's the order the two variables were given to `table()` in. As with the frequency table above, raw counts are easier to compare as percentages, but now there's a choice to make: percentage of what? `prop.table()` takes a `margin` argument for this. `margin = 1` divides each row by its own row total, giving **row percentages**, the breakdown of gender within each smoking category:

```r
prop.table(smoking_gender, margin = 1)
```

```
              Male    Female
Never    0.3976233 0.6023767
Previous 0.6024504 0.3975496
Current  0.5726107 0.4273893
```

`margin = 2` divides each column by its own column total instead, giving **column percentages**, the breakdown of smoking status within each gender:

```r
prop.table(smoking_gender, margin = 2)
```

```
              Male    Female
Never    0.4669913 0.6547414
Previous 0.2863598 0.1748827
Current  0.2466489 0.1703759
```

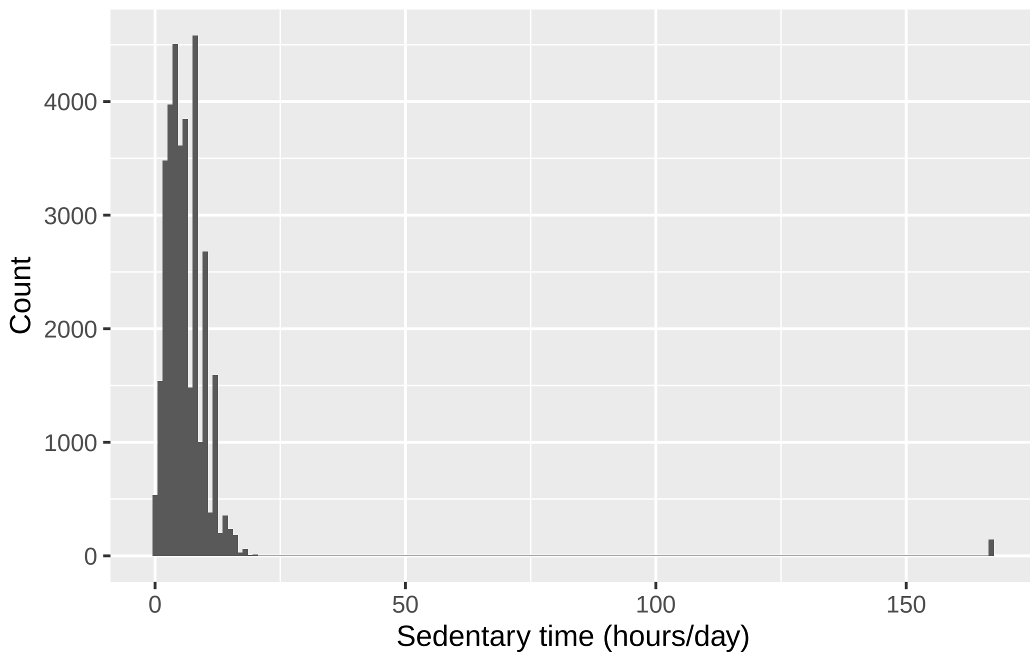
Both tables are built from the same counts, but they answer different questions, and the research question decides which one belongs in a report. This is the same two-by-two logic the epidemiology half already uses for exposure-by-outcome tables, just with more than two categories on one side.

3.9 Plotting with `ggplot2`

Every `ggplot2` plot is built from the same three pieces: the **data**, the **aesthetic mapping** (`aes()`, which decides which variables go on which axis or colour), and the **geometry** (`geom_*()`, which decides what kind of shape represents the data). These pieces are joined with `+`, not the pipe `|>` we've been using so far, because `+` does a different job. `x() |> y()` passes the output of `x()` as an input to `y()`. `+` in `ggplot2` means **layering** instead, an idea introduced by Leland Wilkinson's book *The Grammar of Graphics*: a plot is built as layers stacked on top of one another, data first (`ggplot(nhanes, ...)` below), then the aesthetic mapping (`aes(x = sed)` below), then the geometry (`geom_histogram()` below).

```
ggplot(nhanes, aes(x = sed)) +
  geom_histogram(binwidth = 1) +
  labs(x = 'Sedentary time (hours/day)', y = 'Count')
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_bin()`).
```



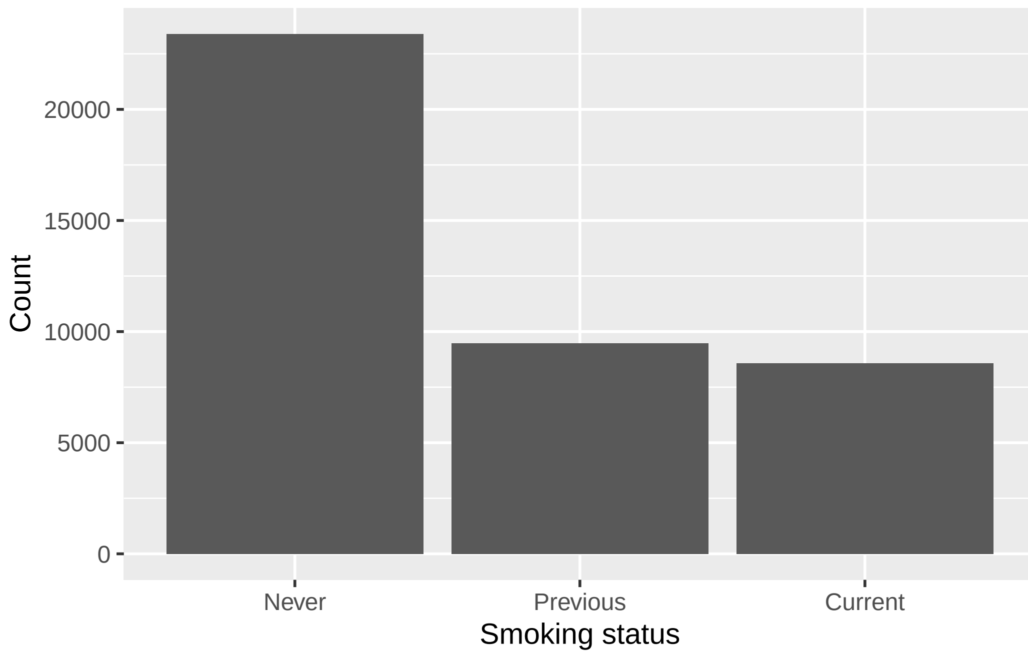
`ggplot()` names the data and the mapping, `geom_histogram()` says to draw it as a histogram, and `labs()` sets the axis labels. Use `labs()` every time: left out, the axes are labelled with the raw column name, so this one would have read `sed` instead of 'Sedentary time (hours/day)', and anyone outside the course would have to guess what that means.

That warning about removed rows is the one Chapter 2 explained: `sed` has 7,331 missing values, so `ggplot2` left them out and said so. It will appear above most plots in this chapter, with a different number each time depending on the variable. Glance at the number, check it's the size we expect, and carry on.

`ggplot2` is highly versatile and can produce nearly any plot we need. Some examples below:

A bar chart of smoking status:

```
ggplot(nhanes |> filter(!is.na(smoking)), aes(x = smoking)) +
  geom_bar() +
  labs(x = 'Smoking status', y = 'Count')
```

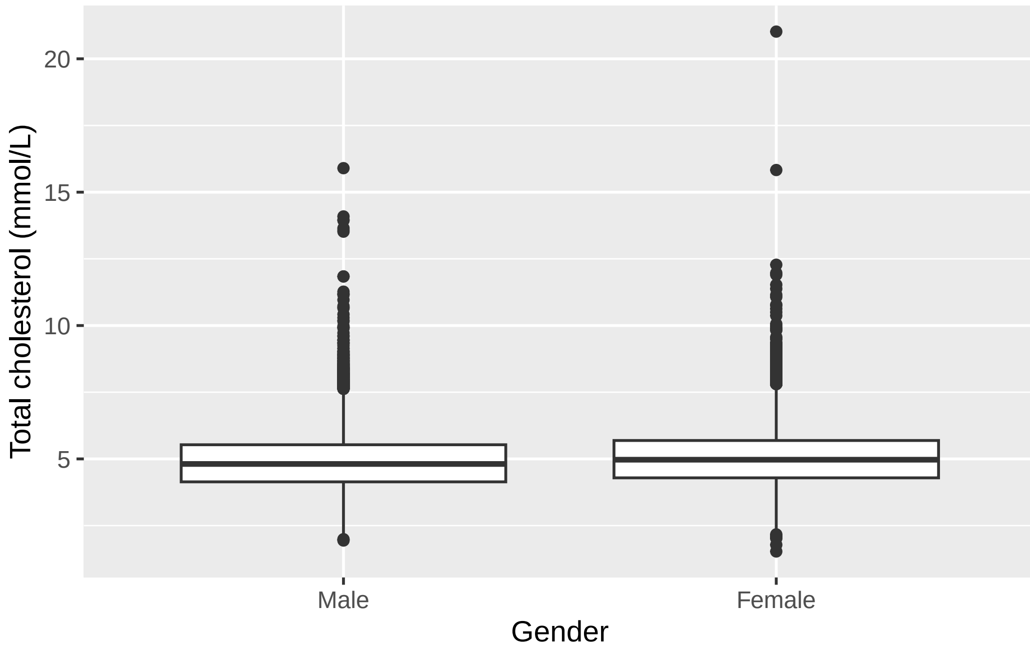


`geom_bar()` is like a histogram for a categorical variable, another way to represent the frequency table we built above. The code is very similar.

A boxplot of cholesterol by gender:

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Total cholesterol (mmol/L)')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```

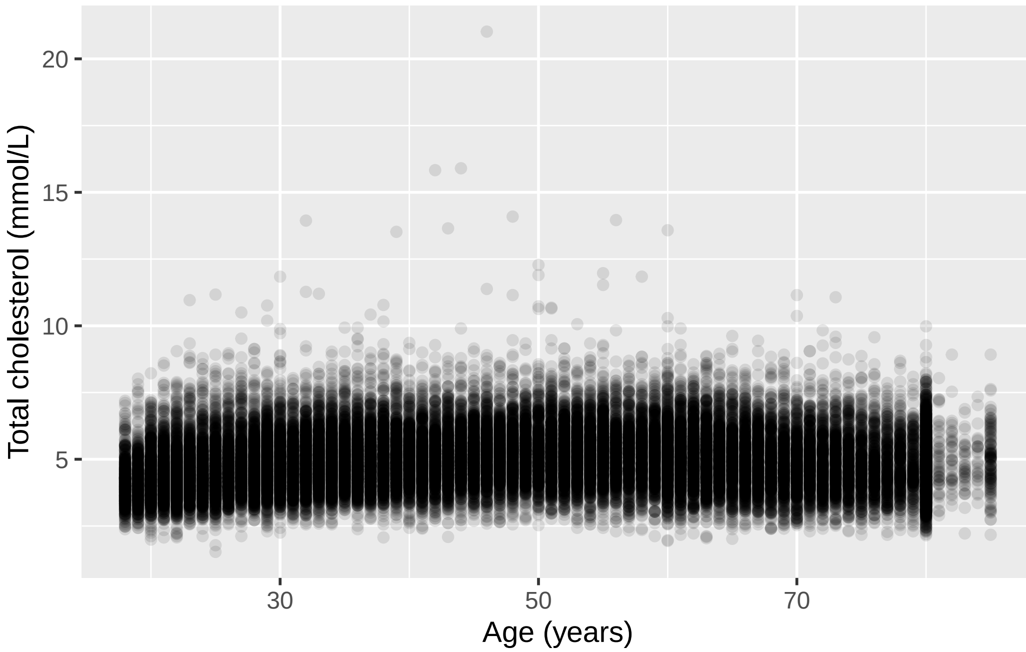


This is useful for examining the relationship between a continuous variable and a categorical one. The code is similar, but with two differences: it explicitly filters out participants with missing gender, and it names both the x and y axis variables. `geom_boxplot()` replaces the `geom_histogram()` from above.

A scatterplot of cholesterol against age:

```
ggplot(nhanes, aes(x = age, y = chol)) +
  geom_point(alpha = 0.1) +
  labs(x = 'Age (years)', y = 'Total cholesterol (mmol/L)')
```

```
Warning: Removed 2589 rows containing missing values or values outside the scale
range (`geom_point()`).
```



`geom_point()` is useful to examine the relationship between two continuous variables. `alpha = 0.1` makes each point mostly transparent. Where thousands of points overlap, the darker regions show where the data are dense. A plain scatterplot of this many points would otherwise just be a solid block of ink.

> **Try it yourself: bin width and scale**
>
> Take the histogram of `sed` above. Change `binwidth` to `0.1`, then to `5`, and re-run it each time. Then add `+ ylim(0, 500)` to the original plot. `ylim()`, from `ggplot2`, fixes the lower and upper limits of the y-axis by hand instead of letting the plot choose them to fit the data. Be careful with it: anything falling outside the limits you set is dropped from the plot altogether rather than being drawn cut off at the edge, so a bar taller than the limit doesn't get shortened, it disappears. For each change, write one sentence on what altered in the plot, and one sentence on what the data themselves did not change. This is the same distinction the seminar drew between a plot design choice and the underlying distribution.
>
> This is the first of several 'Try it yourself' boxes in the book, and none of them has a written solution. Running the code is the answer.

3.10 A first Table 1

A 'Table 1' is a common first table in a health research report. It shows participant characteristics, often split by a grouping variable. We could do it manually with the descriptive statistics commands above (e.g. `mean()`, `table()`) and it is a good exercise. However, the more convenient way is to use `tbl_summary()`, from `gtsummary`.

The code below creates the summary table overall for the five selected variables.

```
nhanes |>
  select(age, gender, eth, smoking, chol) |>
  tbl_summary()
```

Characteristic	**N = 41,765**
age	46 (31, 63)
gender	
Male	20,085 (48%)
Female	21,680 (52%)
eth	
Mexican American	6,849 (16%)
Other Hispanic	3,893 (9.3%)
White	17,297 (41%)
Black	9,086 (22%)
Others	4,640 (11%)
smoking	
Never	23,394 (56%)
Previous	9,468 (23%)
Current	8,580 (21%)
Unknown	323
chol	4.89 (4.22, 5.61)
Unknown	2,589

[1] Median (Q1, Q3); n (%)

Adding `by =` splits the table by a grouping variable, giving one column per group:

```
nhanes |>
  select(age, gender, eth, smoking, chol) |>
  tbl_summary(by = gender)
```

Characteristic	**Male** N = 20,085	**Female** N = 21,680
age	47 (32, 63)	45 (31, 62)
eth		
Mexican American	3,284 (16%)	3,565 (16%)
Other Hispanic	1,717 (8.5%)	2,176 (10%)
White	8,482 (42%)	8,815 (41%)
Black	4,339 (22%)	4,747 (22%)
Others	2,263 (11%)	2,377 (11%)
smoking		
Never	9,302 (47%)	14,092 (65%)
Previous	5,704 (29%)	3,764 (17%)
Current	4,913 (25%)	3,667 (17%)
Unknown	166	157
chol	4.81 (4.14, 5.53)	4.97 (4.29, 5.69)
Unknown	1,218	1,371

[1] Median (Q1, Q3); n (%)

`tbl_summary()`'s default display for a continuous variable is the median and IQR. To change it to mean and SD, we tell it what statistic to use with the `statistic` argument:

```
nhanes |>
  select(age, gender, eth, smoking, chol) |>
  tbl_summary(
    by = gender,
    statistic = list(all_continuous() ~ '{mean} ({sd})')
  )
```

Characteristic	**Male** N = 20,085	**Female** N = 21,680
age	48 (19)	47 (18)
eth		
Mexican American	3,284 (16%)	3,565 (16%)
Other Hispanic	1,717 (8.5%)	2,176 (10%)
White	8,482 (42%)	8,815 (41%)
Black	4,339 (22%)	4,747 (22%)
Others	2,263 (11%)	2,377 (11%)
smoking		
Never	9,302 (47%)	14,092 (65%)
Previous	5,704 (29%)	3,764 (17%)
Current	4,913 (25%)	3,667 (17%)
Unknown	166	157
chol	4.89 (1.09)	5.05 (1.08)
Unknown	1,218	1,371

[1] Mean (SD); n (%)

Three things in that line are new. `list()`, from base R, bundles several settings together, one per variable or group of variables, which is what `statistic` needs when there's more than one display rule to give. `all_continuous()`, from `gtsummary`, means 'every continuous variable in this table', so one line covers both `age` and `chol` at once rather than naming each separately. The `~` between it and the text that follows just pairs the two: which variables, and what to show for them. This book properly introduces `~` as formula syntax in Chapter 4; here it's doing a related but different job, pairing a selection with a display rule rather than an outcome with a predictor. And `'{mean} ({sd})'` is a piece of text with slots in it. `gtsummary` works out the mean and the standard deviation for each variable, then drops each number into the spot where its name appears in curly braces. It comes out as something like `47.3 (18.1)`. Everything outside the braces, here a space and a pair of round brackets, is printed exactly as written. `tbl_summary()` is highly customisable. We can change the statistic for any specific variable, or format different variables differently. That's outside the scope of this course, and left as an exploration exercise for anyone interested.

3.11 Worked example: describing age

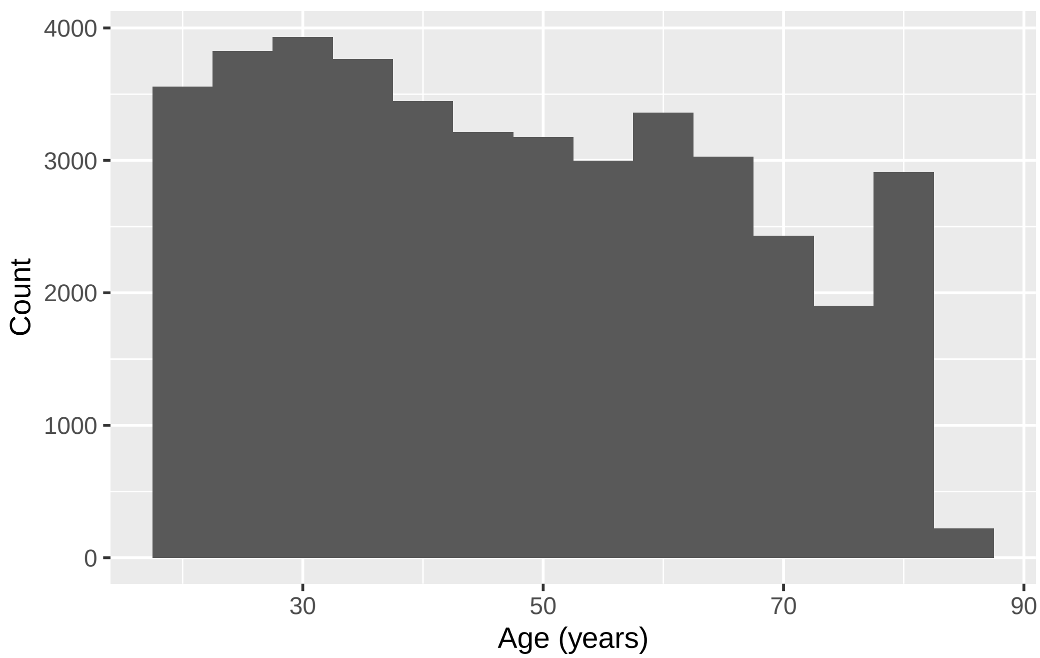
How old are the participants in this sample, and how much does that vary? `age` hasn't come up elsewhere in this chapter, so it's a clean place to put everything above to work at once.

```
nhanes |>
  summarise(
    mean_age = mean(age, na.rm = TRUE),
```

```
    sd_age = sd(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    iqr_age = IQR(age, na.rm = TRUE)
  )
```

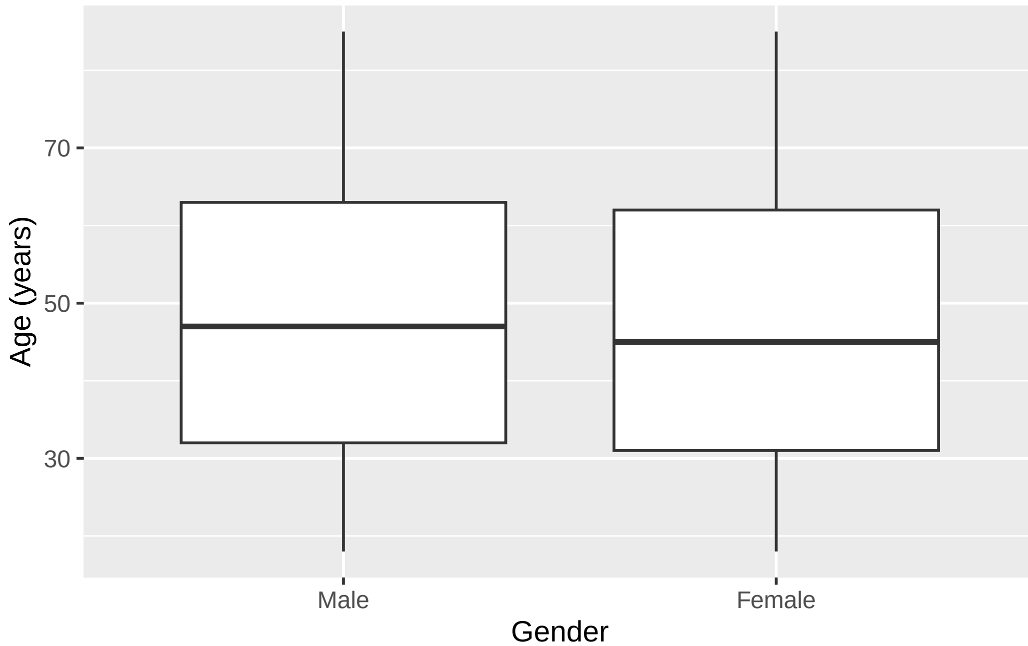
```
# A tibble: 1 x 4
  mean_age sd_age median_age iqr_age
     <dbl>  <dbl>      <dbl>   <dbl>
1     47.5   18.5         46      32
```

```
ggplot(nhanes, aes(x = age)) +
  geom_histogram(binwidth = 5) +
  labs(x = 'Age (years)', y = 'Count')
```



The histogram is close to symmetric, with no strong tail in either direction. So the mean and SD are a fair summary here. Median and IQR would tell much the same story. Now let's split by gender:

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = age)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Age (years)')
```




```
nhanes |>
  select(age, gender) |>
  tbl_summary(by = gender)
```

Characteristic	Male N = 20,085	Female N = 21,680
age	47 (32, 63)	45 (31, 62)

[1] Median (Q1, Q3)

Age looks similar across the two gender groups, both in the boxplot and in the Table 1. There's no obvious difference to report here, and it would be wrong to manufacture one.

3.12 Exercises

Exercise 1 (ILO 2). Using the four `dplyr` verbs, and a pipe, answer this: among participants aged 65 or over who have never smoked, what are the ten highest cholesterol values? Keep only `age`, `gender`, and `chol` in your answer. Then add a column called `chol_high` that is `TRUE` when `chol` is above 6 mmol/L and `FALSE` otherwise.

Exercise 2 (ILOs 2, 4). Describe the distribution of sedentary time (`sed`). Report the mean and SD or the median and IQR, whichever the shape justifies, and support your choice with a histogram and a boxplot. Then produce a scatterplot of `chol` against `sed`, label both axes with their units, and write one sentence on whether the plot suggests any relationship between the two.

Exercise 3 (ILOs 4, 5). Build a histogram of total cholesterol (`chol`) with a `binwidth` of 0.5, labelled properly with `labs()`. Then produce two more versions of it: one with a `binwidth` of 0.05, and one with the original bin width but with `+ ylim(0, 1000)` added. Write one sentence for each of the two changes saying what it does to the impression the plot gives, and one sentence saying which of the three you would put in a report and why.

Exercise 4 (ILOs 2, 4, 6). Compare total cholesterol (`chol`) by gender. Produce a grouped summary and a boxplot, and write one sentence comparing the two groups. Then use `levels()` to state which gender is currently the reference level, and `relevel()` to make the other one the reference instead, assigning the result to a new object rather than overwriting `nhanes`.

Exercise 5 (ILOs 3, 6, 7). Produce a Table 1 of `age`, `gender`, `eth`, `smoking`, and `chol`, stratified by depression classification (`phq.c`).

Exercise 6 (ILOs 3, 4). Build a cross-tabulation of smoking status (`smoking`) against education (`edu`) with `table()`. Produce both the row percentages and the column percentages with `prop.table()`.

Then answer this question using whichever of the two is appropriate: *among people who never smoked, how is education distributed?* State which of your two tables answers it, and say in one sentence why the other one doesn't. Finally, produce a bar chart of `edu`, labelled properly.

3.13 Writing exercise

Every chapter in this book ends with one of these. The course's assessment is a structured paper on a dataset you haven't seen, where every question asks for an analysis *and* an interpretation, so these exercises can be treated as formative assessments.

Using the Table 1 from Exercise 5, write a short paragraph of a maximum of 150 words. Describe how participant characteristics differ, or don't, across the three depression classification groups. State which characteristics look similar across groups and which look different, and avoid overstating a difference the table doesn't support.

Start by stating how many participants the table is actually based on, and how many were excluded because they had no depression classification recorded. The number is in the message printed above the table.

> **i** Solutions
>
> **Exercise 1**

```r
nhanes |>
  filter(age >= 65, smoking == 'Never') |>
  arrange(desc(chol)) |>
  select(age, gender, chol) |>
  slice(1:10)
```

```
# A tibble: 10 x 3
     age gender  chol
   <dbl> <fct>  <dbl>
 1    73 Female 11.1
 2    70 Female 10.4
 3    80 Female  9.98
 4    76 Female  9.57
 5    73 Female  9.36
 6    65 Female  9.13
 7    71 Female  9.05
 8    71 Female  9.05
 9    82 Female  8.92
10    85 Female  8.92
```

The comma inside `filter()` still means `&`, both conditions have to be true. Then `arrange(desc(chol))` sorts from highest cholesterol down, `select()` keeps the three columns asked for, and `slice(1:10)` takes the top ten rows.
The `mutate()` step is separate, because adding a column to ten hand-picked rows isn't much use. Applied to the whole dataset it is:

```r
nhanes |>
  mutate(chol_high = chol > 6) |>
  select(age, chol, chol_high)
```

```
# A tibble: 41,765 x 3
     age  chol chol_high
   <dbl> <dbl> <lgl>
 1    37  4.03 FALSE
 2    23  3.75 FALSE
 3    38  5.04 FALSE
 4    37  5.56 FALSE
 5    27  5.66 FALSE
 6    30  8.9  TRUE
 7    32 NA    NA
 8    30  3.57 FALSE
```

```
 9    30  6.57 TRUE
10    22  6.67 TRUE
# i 41,755 more rows
```

`chol > 6` is a comparison, exactly like the ones in Chapter 1, and it returns `TRUE` or `FALSE` for every row. Participants with no cholesterol value recorded get `NA` rather than `FALSE`, which is correct: we don't know whether they're above 6, and guessing 'no' would be inventing data.

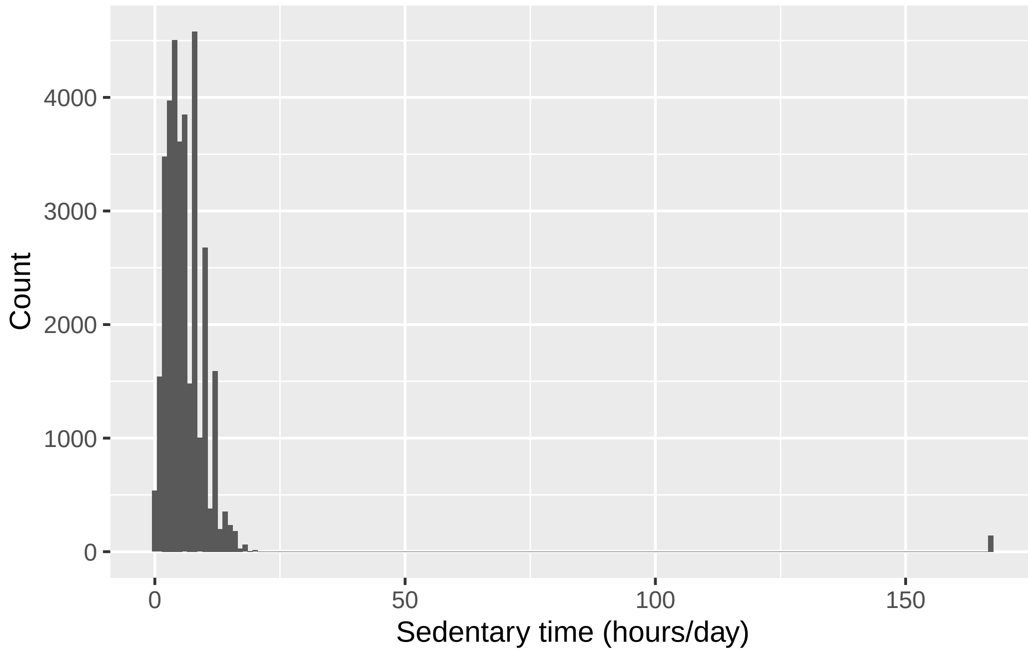
Exercise 2

```r
nhanes |>
  summarise(
    median_sed = median(sed, na.rm = TRUE),
    iqr_sed = IQR(sed, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 2
  median_sed iqr_sed
       <dbl>   <dbl>
1          5       5
```

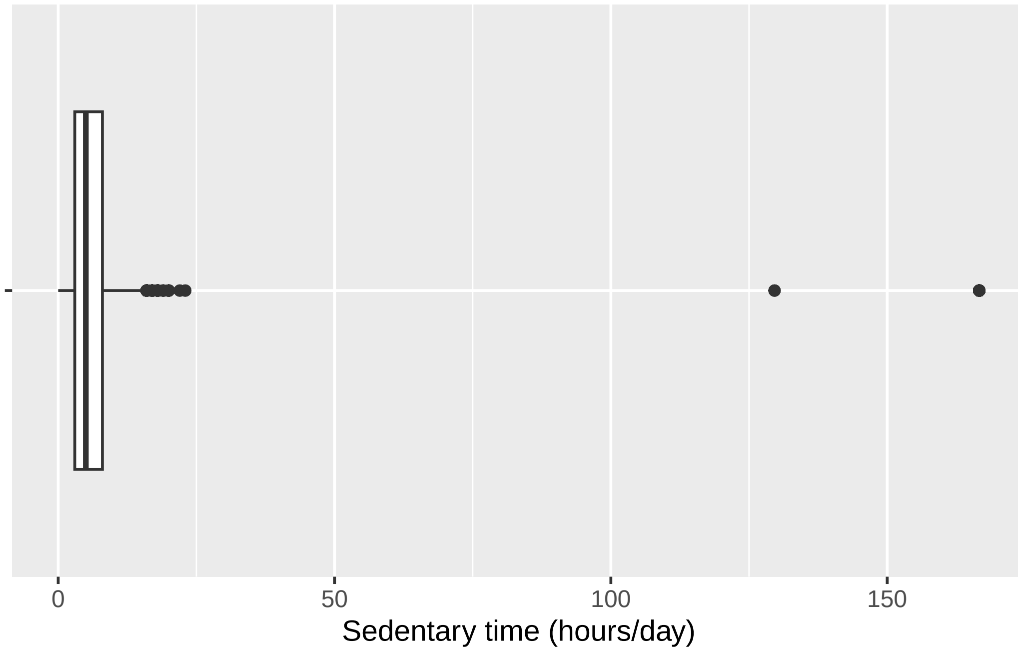
```r
ggplot(nhanes, aes(x = sed)) +
  geom_histogram(binwidth = 1) +
  labs(x = 'Sedentary time (hours/day)', y = 'Count')
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_bin()`).
```




```
ggplot(nhanes, aes(x = sed, y = '')) +
  geom_boxplot() +
  labs(x = 'Sedentary time (hours/day)', y = NULL)
```

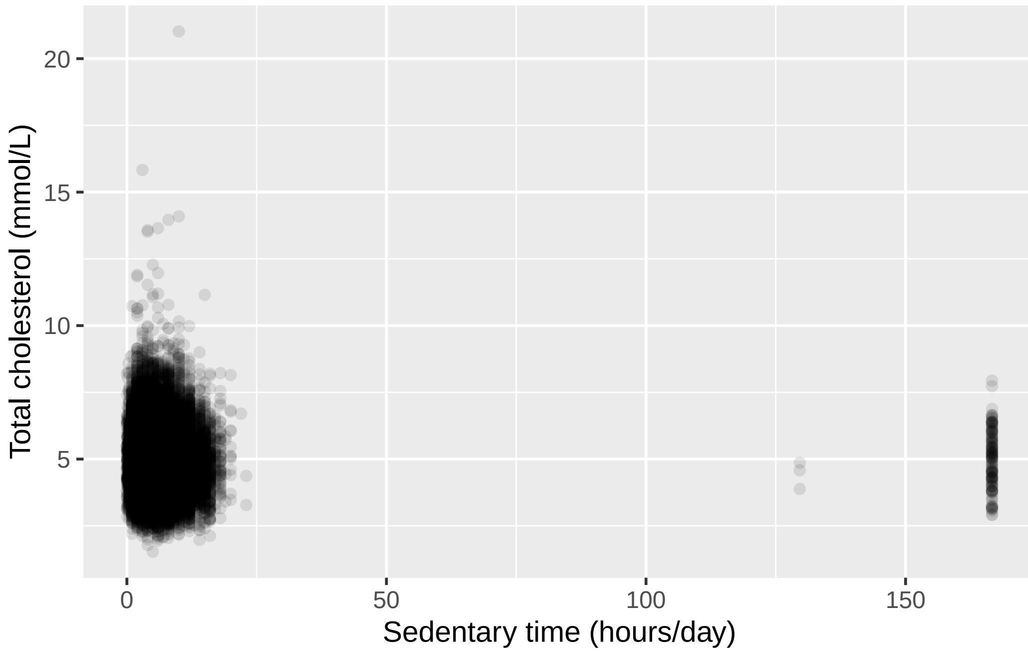
```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



The histogram is right-skewed, with a long tail of participants reporting many hours of sedentary time. So the median and IQR describe it more fairly than the mean and SD would.

```
ggplot(nhanes, aes(x = sed, y = chol)) +
  geom_point(alpha = 0.1) +
  labs(x = 'Sedentary time (hours/day)',
       y = 'Total cholesterol (mmol/L)')
```

```
Warning: Removed 9456 rows containing missing values or values outside the scale
range (`geom_point()`).
```

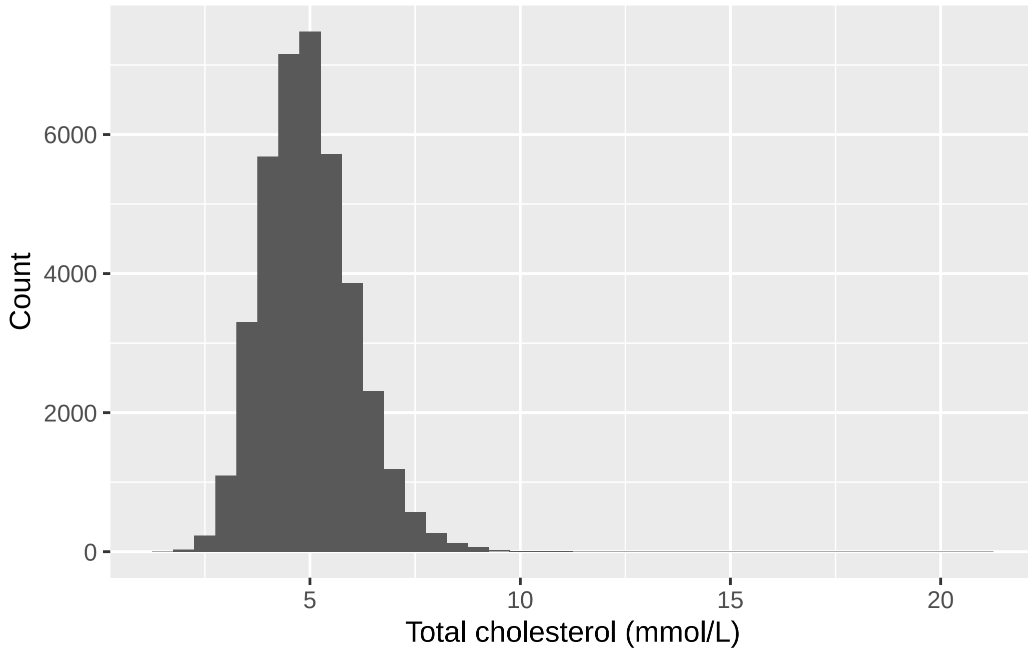


The scatterplot shows no clear relationship. The cloud of points is roughly the same height right across the range of sedentary time, rather than drifting up or down as we move left to right. Whatever else predicts cholesterol here, sedentary time on its own doesn't look like much of it. Chapter 6 puts a number and a confidence interval on that impression instead of leaving it to the eye.

Exercise 3

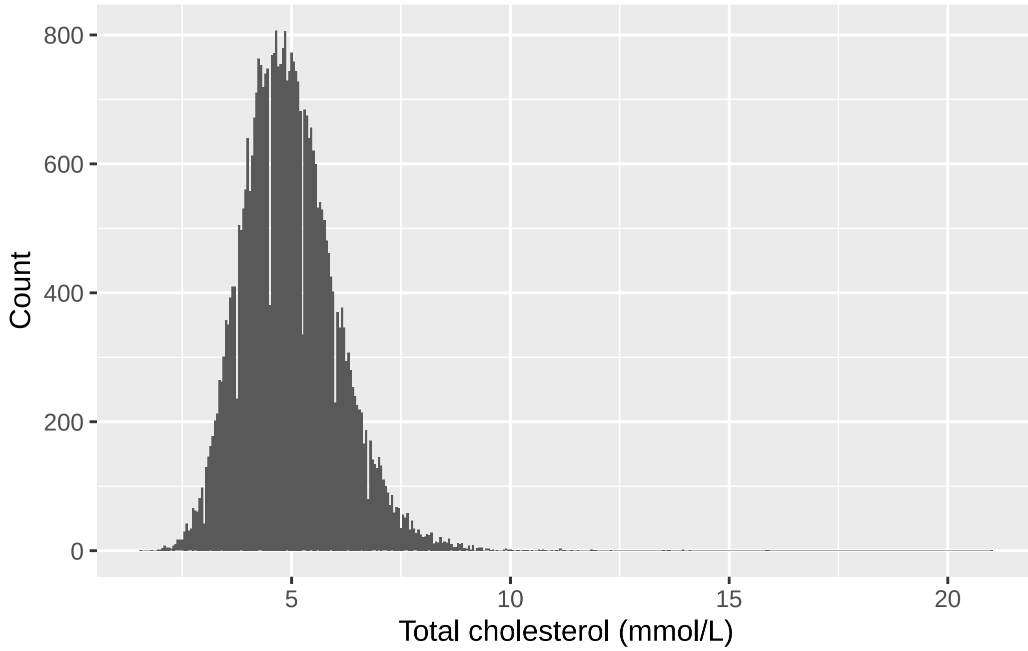
```
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5) +
  labs(x = 'Total cholesterol (mmol/L)', y = 'Count')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```



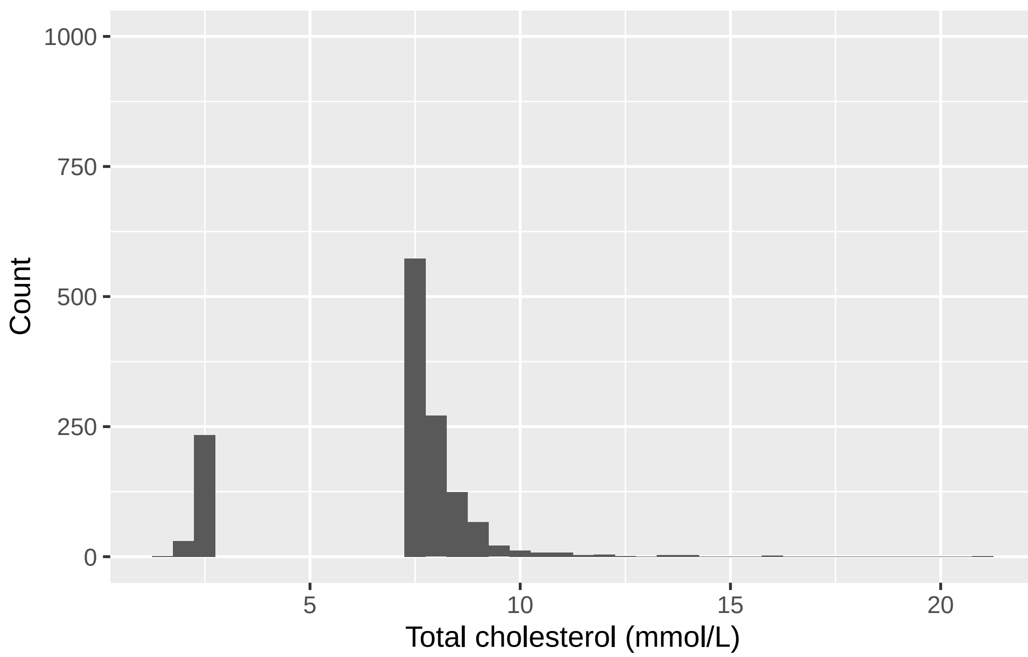

```r
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.05) +
  labs(x = 'Total cholesterol (mmol/L)', y = 'Count')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```




```
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5) +
  ylim(0, 1000) +
  labs(x = 'Total cholesterol (mmol/L)', y = 'Count')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).

Warning: Removed 9 rows containing missing values or values outside the scale range
(`geom_bar()`).
```



Narrowing the bins to 0.05 breaks the smooth shape into a spiky, gappy outline. Cholesterol is recorded to a limited number of decimal places, so at this bin width many bins fall in the gaps between recorded values and come out empty. Those spikes are an artefact of how the values were written down, not a real feature of cholesterol.

Capping the y-axis at 1,000 does something worse. Every bar taller than 1,000 vanishes from the plot completely. The tallest bar here counts over 7,000 participants, and the whole middle of the distribution is taller than the cap, so what's left is the two tails with a hole where most of the data should be.

The first version is the one to put in a report. It shows the whole distribution at a bin width where the shape is readable, and it doesn't drop anything. Notice that all three plots are drawn from exactly the same numbers: nothing about cholesterol changed between them, only the choices we made about how to draw it. That's the seminar's point about misleading figures.

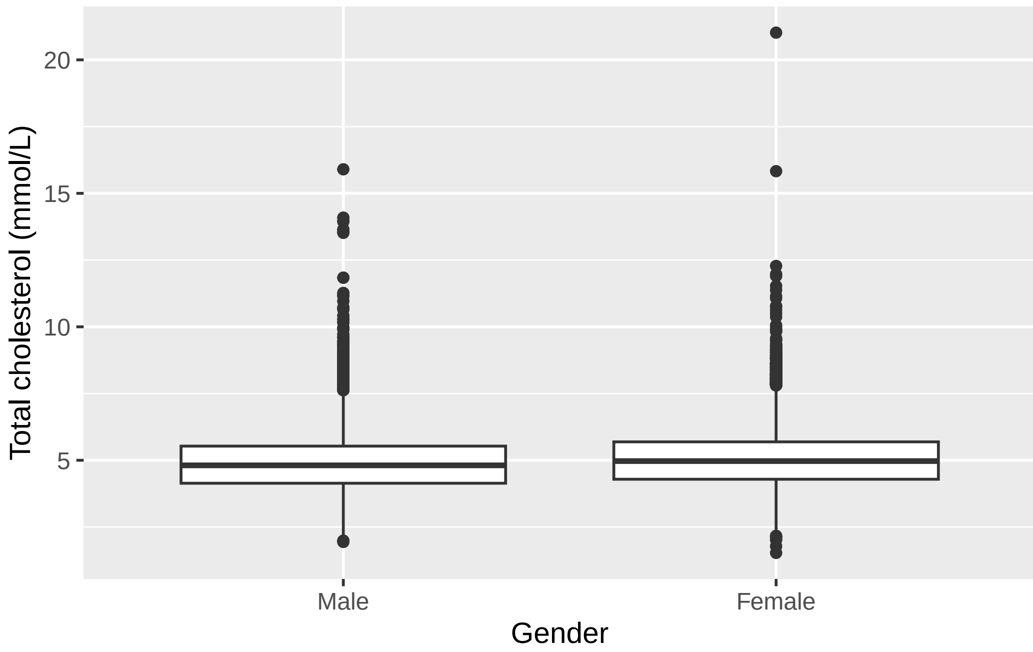
Exercise 4

```r
nhanes |>
  filter(!is.na(gender)) |>
  group_by(gender) |>
  summarise(
    mean_chol = mean(chol, na.rm = TRUE),
    sd_chol = sd(chol, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 2 x 4
  gender mean_chol sd_chol     n
  <fct>      <dbl>   <dbl> <int>
1 Male        4.89    1.09 20085
2 Female      5.05    1.08 21680
```

```r
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Total cholesterol (mmol/L)')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



Mean cholesterol is slightly higher in women than in men, and the boxplots overlap substantially. That's a small difference in the centre, not a separation of the two distributions.

```
levels(nhanes$gender)
```

```
[1] "Male"   "Female"
```

```
gender_female_ref <- relevel(nhanes$gender, ref = 'Female')
levels(gender_female_ref)
```

```
[1] "Female" "Male"
```

`'Male'` is listed first, so `'Male'` is currently the reference level, which follows from the recode listing the numeric code `1` first and labelling it `'Male'`. After `relevel()`, `'Female'` is at the front of the new object and would be the reference instead. `nhanes` itself is untouched.

Exercise 5

```
nhanes |>
  select(age, gender, eth, smoking, chol, phq.c) |>
  tbl_summary(by = phq.c)
```

```
4499 missing rows in the "phq.c" column have been removed.
```

Characteristic	**Normal** N = 25,178	**Mild symptoms** N = 8,628	**Possible depression** N = 3,460
age	46 (31, 63)	46 (30, 61)	51 (36, 63)
gender			
Male	13,041 (52%)	3,755 (44%)	1,406 (41%)
Female	12,137 (48%)	4,873 (56%)	2,054 (59%)
eth			
Mexican American	4,102 (16%)	1,448 (17%)	564 (16%)
Other Hispanic	2,101 (8.3%)	917 (11%)	437 (13%)
White	10,955 (44%)	3,509 (41%)	1,348 (39%)
Black	5,304 (21%)	1,870 (22%)	828 (24%)
Others	2,716 (11%)	884 (10%)	283 (8.2%)
smoking			
Never	14,617 (58%)	4,546 (53%)	1,491 (43%)
Previous	5,978 (24%)	1,894 (22%)	786 (23%)
Current	4,396 (18%)	2,090 (25%)	1,164 (34%)
Unknown	187	98	19
chol	4.89 (4.22, 5.61)	4.89 (4.22, 5.64)	4.91 (4.22, 5.69)
Unknown	1,251	394	185

[1] Median (Q1, Q3); n (%)

Notice the message above the table: `4499 missing rows in the 'phq.c' column have been removed.` That's `gtsummary` telling us something important, not complaining. It isn't an error, and the table is fine.

What it means is this. We asked for the table split three ways, by `phq.c`. A participant with no depression classification recorded can't go in any of the three columns, and `tbl_summary()` left them out. The table describes the 37,266 who remain.

It matters here more than usual, because this is the table the writing exercise asks you to describe. Describing it as though it covered everybody would overstate what it shows by 4,499 people.

Exercise 6

```r
smoking_edu <- table(nhanes$smoking, nhanes$edu)
smoking_edu
```

```
          Less than high school High school graduate Above high school
  Never                    5164                 4656             12778
  Previous                 2405                 2221              4811
  Current                  2707                 2504              3284
```

```r
prop.table(smoking_edu, margin = 1)
```

```
         Less than high school High school graduate Above high school
Never                0.2285158            0.2060359         0.5654483
Previous             0.2548479            0.2353502         0.5098018
Current              0.3186580            0.2947616         0.3865803
```

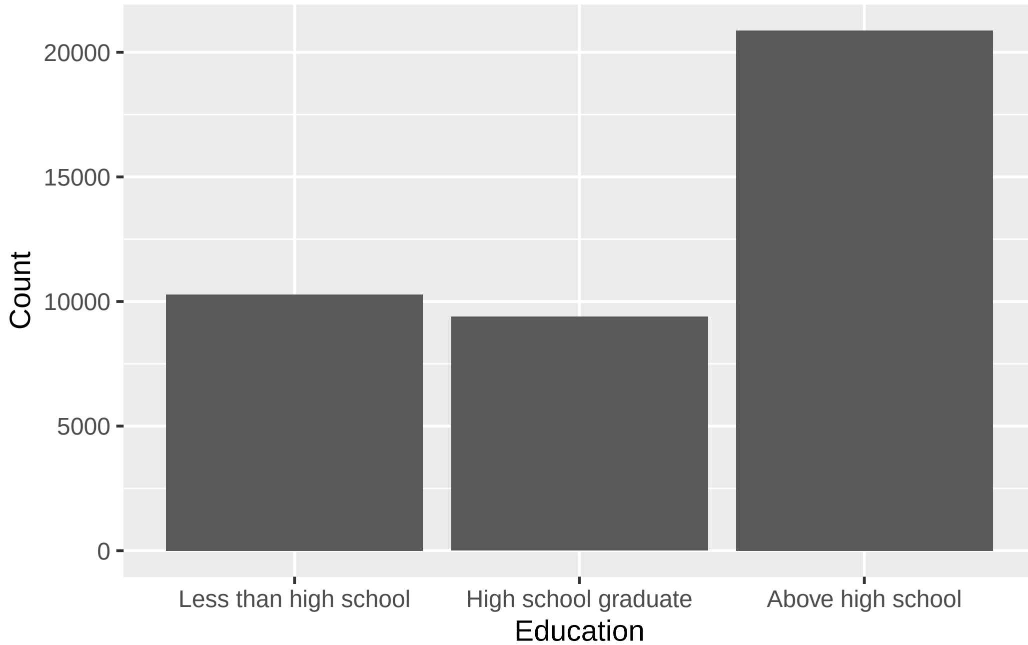
```r
prop.table(smoking_edu, margin = 2)
```

```
         Less than high school High school graduate Above high school
Never                0.5025302            0.4963224         0.6121784
Previous             0.2340405            0.2367551         0.2304891
Current              0.2634293            0.2669225         0.1573324
```

The row percentages are the ones that answer the question. `smoking` was given to `table()` first, so smoking is the rows, and `margin = 1` divides each row by its own total. The 'Never' row then reads as the education breakdown *within* people who never smoked, which is exactly what was asked, and those three numbers add to 100%.

The column percentages answer a different question: within each education group, what share smoke, used to smoke, and never smoked. That's a perfectly good question, just not this one. Both tables come from the same counts, so neither is more correct than the other in the abstract. What decides it is which total we want the percentages to be a share of, and that comes from the question, not from the data.

```r
ggplot(nhanes |> filter(!is.na(edu)), aes(x = edu)) +
  geom_bar() +
  labs(x = 'Education', y = 'Count')
```



Writing exercise (sample answer)
Depression classification was recorded for 37,266 of the 41,765 participants, with the rest being excluded. Among those classified, age, gender, and ethnicity distributions were broadly similar across the three groups. Current smoking was more common among participants classified with possible depression than among those classified as normal. Mean total cholesterol was similar across all three groups. These are descriptive comparisons only, based on group summaries and counts, not any formal test of difference. Hypothesis tests come in Chapter 5.

4 Estimation, precision, and sample size (Week 8)

Chapter 3 taught us to describe a variable with a mean and a standard deviation. Take a different sample from the same population and both numbers come out different. That wobble isn't a flaw in the method and it isn't something to apologise for in a write-up. It's measurable, and measuring it is what turns a number into an estimate we can defend. Sampling variability and the central limit theorem, from this week's seminar, are what make it predictable enough to measure.

Two terms that sound alike carry the whole chapter, and they aren't interchangeable. The standard deviation describes the spread of the *data*. The standard error describes the precision of an *estimate*. Out of the second comes the confidence interval, which we calculate by hand before checking it against R's own functions, since a formula met once from the inside is harder to misread later. The same intervals cover the difference between two groups as well as a single mean, and each one has to be stated in the units of the problem: an interval nobody can put into a sentence hasn't really been interpreted. The machinery also runs backwards. An interval narrows as the sample grows, so fixing the width we want tells us the sample we need, which is how a study gets sized before anyone has collected anything.

4.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Calculate a standard error and construct a confidence interval by formula for a mean and for a proportion from first principle, and reproduce each using `t.test()` and `prop.test()`
2. Construct confidence intervals for a difference in means and a difference in proportions
3. Interpret each interval in the units and context of the problem, in writing
4. Compare how an interval's width responds to sample size
5. Calculate a sample size for a two-group comparison using `power.prop.test()` and `power.t.test()`, and explain why the requirement changes with the assumed effect size

4.2 Setup

As in Chapter 3, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary. This chapter also brings in a second dataset for the first time: `strep_tb`. Unlike `nhanes_mph`, `strep_tb` is built into the `medicaldata` package.

`medicaldata` is the first package this book uses that doesn't come with the tidyverse, so it almost certainly isn't on your machine yet. It has to be installed before it can be loaded. Run `install.packages('medicaldata')` once in the console. Once it's installed, `library(medicaldata)` below works for this session, exactly like every other package we load.

```
library(readr)
library(dplyr)
library(here)
library(medicaldata)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

Two small things about that chunk. There's no `library(ggplot2)`, because this chapter draws no plots and there's no reason to load a package we aren't going to use. And `dictionary` is read in even though no code below refers to it, because it's the thing we reach for the moment we're unsure what a column means. Every chapter reads it for that reason, not because the chapter's own code needs it.

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c('Male', 'Female')),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c('Mexican American', 'Other Hispanic', 'White', 'Black', 'Others')),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c('Less than high school', 'High school graduate', 'Above high school')),
    smoking = factor(smoking,
      levels = c('Never', 'Previous', 'Current')),
    phq.c = factor(phq.c,
      levels = c('0 Normal', '1 Mild symptoms', '2 Possible depression'),
      labels = c('Normal', 'Mild symptoms', 'Possible depression'),
```

```
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c('No', 'Yes'))
  )
```

This is the same recode we introduced in Chapter 3. See Chapter 3 for the reasoning behind each variable. We repeat it here, unchanged, so this chapter stands on its own too.

4.3 Point estimate and standard error, by hand

A mean handed to a health board with no sense of how precise it is invites a decision the data may not actually support. A point estimate is our 'best guess' from the data, most commonly the mean, and on its own it says nothing about how precise it is. The standard error attaches that precision, calculated from two quantities already familiar from Chapter 3: the standard deviation, `sd()`, and the sample size, `n()`. Let's apply this to sedentary time (`sed`):

```
sed_stats <- nhanes |>
  filter(!is.na(sed)) |>
  summarise(
    mean_sed = mean(sed),
    sd_sed = sd(sed),
    n_sed = n()
  ) |>
  mutate(se_sed = sd_sed / sqrt(n_sed))

sed_stats
```

```
# A tibble: 1 x 4
  mean_sed sd_sed n_sed se_sed
     <dbl>  <dbl> <int>  <dbl>
1     6.56   10.9 34434 0.0588
```

`sqrt()` is base R's square root function. The standard error of a mean is the standard deviation divided by the square root of the sample size, the algebra behind the seminar's precision argument: as `n` grows, the denominator grows and the standard error shrinks.

`sed` was shown in Chapter 3 to be right-skewed, with a long tail of participants reporting many sedentary hours. That skew doesn't disqualify it from a confidence interval on its *mean*, because of the Central Limit Theorem. As the sample size grows, the *sampling distribution* gets closer to a bell-shaped normal distribution.

4.4 From standard error to a confidence interval

A 95% confidence interval for a mean is the point estimate plus or minus 1.96 standard errors. 1.96 is the value that cuts off the middle 95% of a normal distribution:

```
sed_stats <- sed_stats |>
  mutate(
    ci_lower = mean_sed - 1.96 * se_sed,
    ci_upper = mean_sed + 1.96 * se_sed
  )

sed_stats
```

```
# A tibble: 1 x 6
  mean_sed sd_sed n_sed se_sed ci_lower ci_upper
     <dbl>  <dbl> <int>  <dbl>    <dbl>    <dbl>
1     6.56   10.9 34434 0.0588     6.45     6.68
```

R has a function that does this directly. `t.test()` expects a single numeric vector, not a data frame, so we pull `sed` out of `nhanes` with `$`. `nhanes$sed` reads as 'the `sed` column, inside `nhanes`', and it returns that column on its own, as a plain vector of numbers, rather than a data frame containing it. That's different from `select(sed)`, which keeps a one-column data frame instead of pulling the column out. `t.test()` then returns the mean and its 95% confidence interval without any of the arithmetic above:

```
t.test(nhanes$sed)
```

```

	One Sample t-test

data:  nhanes$sed
t = 111.73, df = 34433, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 6.449124 6.679429
sample estimates:
mean of x
 6.564277
```

`t.test()` also reports a test statistic, degrees of freedom, and a p-value. These belong to a hypothesis test, which is Chapter 5's subject. For now, look only at the confidence interval, and check that it closely matches the interval we calculated by hand above.

One thing `t.test()` does quietly is it ignores all missing values. We passed it `nhanes$sed` whole, missing values and all, with no `filter()` and no `na.rm = TRUE`, and it ran anyway. It drops missing values itself, before calculating. The evidence is in the output: the degrees of freedom are 34,433, one less than the 34,434 participants we filtered down to by hand, which is why the two intervals agree at all. So whenever we run a `t.test()` in R, we should count the missing values ourselves.

The two intervals are close but not identical, because `t.test()` doesn't use a fixed multiplier of 1.96. It uses a value from the t-distribution, which is usually a better match to the actual sampling distribution, since it depends on the sample size through its degrees of freedom (`n - 1`). For a sample size this large, the normal and t-distributions are very similar and shouldn't make a material difference. In practice, we'd use `t.test()` rather than working this out by hand every time, but the formula above is exactly what it's doing underneath.

4.4.1 Saying what the interval means, and what it doesn't

We now have an interval. Writing it down correctly is a separate skill from calculating it, and it's the one the seminar spent time on, contrasting the right sentence with the wrong one. Here is the right one, for the interval we just built:

> Mean sedentary time was 6.56 hours/day (95% CI: 6.45 to 6.68 hours/day).

And here is the wrong one, which is the single most common error in this part of the course:

> There is a 95% probability that the true mean sedentary time lies between 6.45 and 6.68 hours/day.

The true mean sedentary time in the population is a fixed number. We don't know it, but it isn't moving around, and it doesn't have a 95% chance of being anywhere. What moves is the *interval*. Draw a different sample and we get a different mean, a different standard error, and so a different pair of numbers. The 95% belongs to that procedure: of all the intervals we could build this way, 95% of them would contain the true value. This particular interval either contains it or it doesn't, and we have no way of knowing which.

In practice the fix is easy, because the correct sentence is the shorter one. State the estimate, put the interval after it, and stop. The moment we write 'probability', 'chance', or 'there's a 95% likelihood the true value is', we have said something about the parameter that the interval doesn't support. Every interval in the rest of this book gets reported the first way, and so should yours.

4.5 A confidence interval for a proportion

`strep_tb`, from the `medicaldata` package, records the 1948 MRC streptomycin trial: a randomised controlled trial of streptomycin against tuberculosis in 107 patients, generally considered the first modern randomised controlled trial. Each patient was assigned to the `Streptomycin` or `Control` arm and followed up. Patients in the control arm received bed rest alone, not a placebo. `improved` records whether their chest X-ray had improved at six months. As with any new dataset, we look at what actually came in before writing code against it:

```
glimpse(strep_tb)
```

```
Rows: 107
Columns: 13
$ patient_id          <chr> "0001", "0002", "0003", "0004", "0005", "0006"~
$ arm                 <fct> Control, Control, Control, Control, Control, C~
$ dose_strep_g        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ dose_PAS_g          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ gender              <fct> M, F, F, M, F, M, F, M, F, M, F, M, F, M, F, M~
$ baseline_condition  <fct> 1_Good, 1_Good, 1_Good, 1_Good, 1_Good, 1_Good~
$ baseline_temp       <fct> 1_98-98.9F, 3_100-100.9F, 1_98-98.9F, 1_98-98.~
$ baseline_esr        <fct> 2_11-20, 2_11-20, 3_21-50, 3_21-50, 3_21-50, 3~
$ baseline_cavitation <fct> yes, no, no, no, no, no, yes, yes, yes, yes, n~
$ strep_resistance    <fct> 1_sens_0-8, 1_sens_0-8, 1_sens_0-8, 1_sens_0-8~
$ radiologic_6m       <fct> 6_Considerable_improvement, 5_Moderate_improve~
$ rad_num             <dbl> 6, 5, 5, 5, 5, 6, 5, 5, 5, 5, 6, 5, 5, 5, 5, 6~
$ improved            <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE~
```

`arm` and `gender` are already factors. `improved`, the outcome we care about here, is **logical**: `TRUE` or `FALSE`, not a factor. R stores `TRUE` and `FALSE` as 1 and 0, which will matter again shortly.

Below we will look at the proportion of the outcome. `table()`, already met in Chapter 3, counts how many patients fall into each category of `improved`, and `prop.table()` converts those counts to proportions:

```
table(strep_tb$improved)
```

```
FALSE  TRUE
   52    55
```

```
prop.table(table(strep_tb$improved))
```

```
    FALSE      TRUE 
0.4859813 0.5140187 
```

4.5.1 The standard error of a proportion, by hand

A proportion is an estimate like any other, so it has a standard error too. The formula is different from the one for a mean, because a proportion carries its own variability: once we know p, we know how spread out a yes/no variable is, so there's no separate SD to measure. The standard error of a proportion is the square root of $p \times (1 - p) \div n$.

`sum()` of a logical vector counts how many values are `TRUE`, since R stores `TRUE` as 1 and `FALSE` as 0. So `sum(strep_tb$improved)` is the number of patients who improved, out of `nrow(strep_tb)` in total.

```
tb_stats <- strep_tb |>
  summarise(
    n_improved = sum(improved),
    n_total = n(),
    p_improved = n_improved / n_total,
    se_improved = sqrt(p_improved * (1 - p_improved) / n_total)
  )

tb_stats
```

```
# A tibble: 1 x 4
  n_improved n_total p_improved se_improved
       <int>   <int>      <dbl>       <dbl>
1         55     107      0.514      0.0483
```

The confidence interval is then built exactly the way it was for a mean: the estimate, plus or minus 1.96 standard errors.

```
tb_stats$p_improved - 1.96 * tb_stats$se_improved
```

```
[1] 0.4193158
```

```r
tb_stats$p_improved + 1.96 * tb_stats$se_improved
```

```
[1] 0.6087216
```

4.5.2 The same interval from `prop.test()`

`prop.test()` returns a confidence interval for a proportion directly, given a count of successes (`x`) and a total (`n`), with none of the arithmetic above:

```r
prop.test(x = tb_stats$n_improved, n = tb_stats$n_total)
```

```

	1-sample proportions test with continuity correction

data:  tb_stats$n_improved out of tb_stats$n_total, null probability 0.5
X-squared = 0.037383, df = 1, p-value = 0.8467
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.4159553 0.6110633
sample estimates:
        p 
0.5140187 
```

The two intervals are close but not identical, and for the same kind of reason the hand-built and `t.test()` intervals differed earlier. The ± 1.96 version above is the textbook approximation, and it works well when n is large and p isn't near 0 or 1. `prop.test()` uses a better method that behaves more sensibly in small samples and never runs past 0 or 1, which the approximation happily will if pushed too far. With 107 patients and a proportion near a half, both are perfectly reasonable and they agree to within a small margin of error.

Giving `prop.test()` the count and total directly, rather than a cross-tabulated table, avoids any doubt about which of the two categories we're counting. A table of `improved` alone would still show both proportions clearly, but a table fed straight into `prop.test()` reports the proportion for only whichever category comes first, which is not necessarily the one we want.

By default, `prop.test()` also tests whether this proportion differs from 50%. That default comparison isn't of interest here, and it isn't the subject of this chapter. The 95% confidence interval it reports alongside is what we want.

4.6 Confidence interval for a difference in means

`t.test()` also compares two groups directly, using **formula syntax**. `chol ~ gender` reads as '`chol`, modelled by `gender`'. R splits `chol` into the groups defined by `gender` and compares them:

```
t.test(chol ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```

	Welch Two Sample t-test

data:  chol by gender
t = -15.318, df = 38908, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
 -0.1901172 -0.1469821
sample estimates:
  mean in group Male mean in group Female
            4.885939             5.054489
```

Notice there's no `$` anywhere in that call, even though `chol` and `gender` both are only found inside `nhanes`. The `data =` argument tells `t.test()` which data frame to search for names like `chol` and `gender`, so it looks them up there itself instead of us pulling each one out by hand first with `$`. `$` is still the right tool for getting a single column on its own, the way `nhanes$sed` was above; a formula's `data =` argument does that lookup for every variable named in the formula at once.

This is the same pairing we showed descriptively in Chapter 3's boxplot and Table 1: cholesterol by gender. Now it's attached to a confidence interval for the *difference* in means between the two groups. By default, `t.test()` doesn't assume the two groups have equal variance, i.e. *Welch's t-test*. That's a sensible default and not something we need to adjust here.

4.6.1 Which way round is the difference?

Both ends of that interval are negative, and a negative difference in cholesterol isn't obviously meaningful until we know what has been subtracted from what. R tells us, on the line beginning 'alternative hypothesis': *true difference in means between group Male and group Female*. So it has taken the men's mean and subtracted the women's.

That order isn't R's choice, and it isn't alphabetical. It's the order the factor's levels are in, which we set ourselves in the setup chunk:

```r
levels(nhanes$gender)
```

```
[1] "Male"   "Female"
```

`'Male'` is first, so men are group 1 and women are group 2, and the difference runs first minus second. A negative interval therefore says the first group is *lower*: mean cholesterol is lower in men than in women, by somewhere between 0.15 and 0.19 mmol/L.

It is useful to check the alternative-hypothesis line before reading any two-group interval. If we'd rather write the comparison the other way round, which is often more natural to read, we flip the whole thing together. 'Cholesterol was 0.17 mmol/L higher in women than in men (95% CI: 0.15 to 0.19)' says exactly the same as R's output, with every sign reversed. What we must never do is flip the words and leave the numbers, or flip one end of the interval and not the other. The direction of the estimate and the direction of the interval have to agree.

4.7 Confidence interval for a difference in proportions

The same idea extends to two proportions: how many improved in each trial arm. `group_by()` and `summarise()`, already familiar from Chapter 3, give us the count and the number improved in each arm:

```r
strep_arm_summary <- strep_tb |>
  group_by(arm) |>
  summarise(
    n_arm = n(),
    n_improved = sum(improved),
    prop_improved = n_improved / n_arm
  )

strep_arm_summary
```

```
# A tibble: 2 x 4
  arm          n_arm n_improved prop_improved
  <fct>        <int>      <int>         <dbl>
1 Streptomycin    55         38         0.691
2 Control         52         17         0.327
```

`prop_improved` here is exactly what `mean(improved)` would give directly. `mean()` of a logical vector is a shortcut worth knowing: R treats `TRUE` as 1 and `FALSE` as 0, so the mean of a column of `TRUE`/`FALSE` values is the proportion that are `TRUE`.

If we pass the two counts and the two totals to `prop.test()`, rather than a cross-tabulated table, it compares the two proportions unambiguously. `prop 1` and `prop 2` below correspond, in order, to the two rows of `strep_arm_summary` above:

```
prop.test(x = strep_arm_summary$n_improved, n = strep_arm_summary$n_arm)
```

```

	2-sample test for equality of proportions with continuity
	correction

data:  strep_arm_summary$n_improved out of strep_arm_summary$n_arm
X-squared = 12.756, df = 1, p-value = 0.0003548
alternative hypothesis: two.sided
95 percent confidence interval:
 0.168726 0.559246
sample estimates:
   prop 1    prop 2 
0.6909091 0.3269231 
```

The confidence interval is for the difference in the proportion improved between the two arms. Before reading it, check the direction, exactly as we did for the difference in means above. The same rule applies: R takes the first group and subtracts the second, and the order comes from the factor's levels.

```
levels(strep_tb$arm)
```

```
[1] "Streptomycin" "Control"
```

`'Streptomycin'` is first, so `prop 1` is the streptomycin arm and `prop 2` is the control arm, and the difference runs streptomycin minus control. Both ends of the interval are positive, which therefore says the streptomycin arm improved *more*. Had the two levels been the other way round, every number in that interval would have carried a minus sign and meant exactly the same thing. Chapter 5 puts this same factor the other way round on purpose, which is exactly why checking the levels here first matters.

The heading says 'with continuity correction': `prop.test()` applies a small adjustment by default that nudges the p-value up and the interval slightly wider. Chapter 5 explains what it does, so leave it for now, but notice that the p-value printed here, 0.0004, is the one that adjustment produces. That p-value itself belongs to a hypothesis test, which is still Chapter 5's subject.

Because strep_tb is a randomised trial rather than an observational study, a difference this large and this precisely estimated supports a causal reading in a way that an association in nhanes_mph would not. We can quite safely conclude, from the findings, that streptomycin improved the proportion of patients with X-ray improvement by about 36 percentage points, compared with bed rest alone. We say percentage points rather than percent here: percent would describe how much bigger one proportion is relative to the other, not the size of the gap between them, which is what this interval actually estimates.

4.8 Width and sample size, without programming

The seminar's rule was that a confidence interval narrows as sample size grows. We can show this directly, by computing the same interval on the full sample and on two smaller subsets of it. slice_sample(), from dplyr, keeps a random fraction of the rows. set.seed(), from base R, fixes the sequence of 'random' numbers R uses, so the same subset gets drawn every time this code is run, rather than a different one each time.

The full sample. This time we save the result into an object with <- as well as printing it, because we want to reuse its numbers in a moment:

```
tt_full <- t.test(nhanes$chol)
tt_full
```

```

	One Sample t-test

data:  nhanes$chol
t = 902.16, df = 39175, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.962511 4.984121
sample estimates:
mean of x
 4.973316
```

A 50% subset:

```
set.seed(1)
chol_50 <- nhanes |>
  filter(!is.na(chol)) |>
  slice_sample(prop = 0.5)
```

```r
tt_50 <- t.test(chol_50$chol)
tt_50
```

```

    One Sample t-test

data:  chol_50$chol
t = 637.89, df = 19587, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.955667 4.986217
sample estimates:
mean of x
 4.970942
```

A 10% subset:

```r
set.seed(1)
chol_10 <- nhanes |>
  filter(!is.na(chol)) |>
  slice_sample(prop = 0.1)

tt_10 <- t.test(chol_10$chol)
tt_10
```

```

    One Sample t-test

data:  chol_10$chol
t = 289.21, df = 3916, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.933782 5.001132
sample estimates:
mean of x
 4.967457
```

The point estimates are similar across all three, because all three are samples from the same population. But the confidence interval widens as the sample size drops.

That's hard to see across three blocks of output, so let's put the three intervals in one small table. A `t.test()` result isn't a data frame, but `$` still works on it, pulling out one named piece at a time exactly as it pulls one column out of a data frame. `tt_full$conf.int` is the pair of numbers at the ends of the interval, and `[1]` and `[2]` take the lower and the upper one, the same square-bracket indexing we used in Chapter 2. `data.frame()`, from Chapter 1, builds the table out of them, and `mutate()` adds the column we actually care about:

```
widths <- data.frame(
  sample = c('Full', '50% subset', '10% subset'),
  n = c(sum(!is.na(nhanes$chol)), nrow(chol_50), nrow(chol_10)),
  lower = c(tt_full$conf.int[1], tt_50$conf.int[1], tt_10$conf.int[1]),
  upper = c(tt_full$conf.int[2], tt_50$conf.int[2], tt_10$conf.int[2])
) |>
  mutate(width = upper - lower)

widths
```

```
      sample     n    lower    upper      width
1       Full 39176 4.962511 4.984121 0.02161000
2 50% subset 19588 4.955667 4.986217 0.03054926
3 10% subset  3917 4.933782 5.001132 0.06734956
```

Now the pattern is one column instead of six numbers on three screens. Cut the sample in half and the interval gets about 40% wider; cut it to a tenth and it's roughly three times wider than we started. That last number is the useful one to hold onto. Ten times less data did not make the interval ten times wider, it made it about three times wider, and three is roughly the square root of ten. That's the $SE = s/\sqrt{n}$ formula showing up in an actual result. Precision improves with the square root of the sample size, not with the sample size itself. That's exactly why doubling a study's size buys much less than it feels like it should.

4.9 Sample size and power

The same logic runs in reverse. Instead of asking how precise an estimate from a given sample size will be, we can size a study in advance to detect a given effect. `strep_tb` supplies the natural example, since it's a trial with a binary outcome, and `strep_arm_summary` above already has the proportion improved in each arm: roughly 33% in the control arm and 69% in the streptomycin arm.

`power.prop.test()`, from base R, answers a practical question: how large would a two-arm trial need to be to reliably detect a gap this size? We give it the two proportions, a significance level, and a target power:

```r
power.prop.test(p1 = 0.33, p2 = 0.69, sig.level = 0.05, power = 0.8)
```

```

     Two-sample comparison of proportions power calculation

              n = 29.0615
             p1 = 0.33
             p2 = 0.69
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

`sig.level = 0.05` is the Type I error rate the seminar introduced. `power = 0.8` is the conventional target: an 80% chance of detecting the effect if it's really there. `n` is the number **per group** required. The gap observed in the original trial was unusually large. A smaller assumed gap needs a correspondingly larger trial:

```r
power.prop.test(p1 = 0.33, p2 = 0.45, sig.level = 0.05, power = 0.8)
```

```

     Two-sample comparison of proportions power calculation

              n = 258.158
             p1 = 0.33
             p2 = 0.45
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

The same function, `power.t.test()`, sizes a study around a continuous outcome instead, using the variability of `chol`:

```r
chol_sd <- nhanes |>
  filter(!is.na(chol)) |>
  summarise(sd_chol = sd(chol)) |>
  pull(sd_chol)
```

```r
power.t.test(delta = 0.5, sd = chol_sd, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

              n = 75.72811
          delta = 0.5
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

`pull()`, from `dplyr`, extracts a single column as a plain vector instead of a one-column tibble. We need this because `power.t.test()` expects a single number for `sd`, not a table containing one. `delta` is the assumed difference in means the study is being sized to detect. As with the proportions above, a smaller assumed difference requires a larger sample:

```r
power.t.test(delta = 0.2, sd = chol_sd, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

              n = 468.1853
          delta = 0.2
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Both functions size a study *before* it is run, from an assumed effect. Power calculated after the fact, from a result already in hand, only re-expresses the p-value already obtained and adds nothing. The seminar's point about post-hoc power applies to both `power.prop.test()` and `power.t.test()` equally.

> **Try it yourself: assumed effect size**
>
> Re-run `power.prop.test()` with `p2` set closer and closer to `p1` (for example 0.40, then 0.35), keeping `power = 0.8`. Watch what happens to `n`, and say in one sentence what it means for a researcher planning a trial to detect a genuinely small effect.

4.10 Worked example: a confidence interval for age

A mean on its own says nothing about how much confidence we should place in it. So how precisely can we pin down the average age of participants in this sample? `age` hasn't appeared elsewhere in this chapter, which makes it a fair test of the machinery just built.

```
age_stats <- nhanes |>
  filter(!is.na(age)) |>
  summarise(
    mean_age = mean(age),
    sd_age = sd(age),
    n_age = n()
  ) |>
  mutate(
    se_age = sd_age / sqrt(n_age),
    ci_lower = mean_age - 1.96 * se_age,
    ci_upper = mean_age + 1.96 * se_age
  )

age_stats
```

```
# A tibble: 1 x 6
  mean_age sd_age n_age se_age ci_lower ci_upper
     <dbl>  <dbl> <int>  <dbl>    <dbl>    <dbl>
1     47.5   18.5 41765 0.0905     47.3     47.7
```

```
t.test(nhanes$age)
```

```


    One Sample t-test

data:  nhanes$age
t = 524.82, df = 41764, p-value < 2.2e-16
```

```
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 47.33953 47.69445
sample estimates:
mean of x
 47.51699
```

The hand-calculated interval and the one from `t.test()` agree closely. Written as it would appear in a results section: the mean age of participants was 47.5 years (95% CI: 47.3 to 47.7 years). This is a statement about the precision of the *estimate* of the mean, not a description of how spread out individual ages are. That's what the SD is for.

4.11 Exercises

Exercise 1 (ILOs 1, 2, 3). Calculate the mean and standard error of total cholesterol (`chol`), construct a 95% confidence interval by hand, and confirm it with `t.test()`. Report the result the way a results section would state it, in one sentence.

Then use `t.test()` with formula syntax to obtain a 95% confidence interval for the *difference* in mean cholesterol between smoking groups, comparing current smokers with those who have never smoked. You'll need `filter()` to restrict `smoking` to those two categories first, since `t.test()` compares exactly two groups. Write one sentence interpreting that interval, and one sentence saying how it differs in meaning from the interval you built in the first half of this exercise.

Exercise 2 (ILOs 1, 2, 3). In `strep_tb`, find the proportion of patients with baseline cavitation present (`baseline_cavitation`) and calculate its standard error by hand, then build a 95% confidence interval from it and check the result against `prop.test()`. Then construct a 95% confidence interval for the difference in the proportion improved between men and women (`gender`). Write one interpretive sentence for each interval.

Exercise 3 (ILOs 3, 4). Repeat the width exploration above (full sample, 50% subset, 10% subset) using `sed` instead of `chol`, and collect the three intervals into one table the way the chapter did, so their widths sit side by side. Describe what changes and what stays about the same as the sample size drops.

Exercise 4 (ILOs 3, 5). Using `power.t.test()`, find the sample size needed to detect a difference in `chol` of 0.3 mmol/L, and again for a difference of 0.8 mmol/L, both at `power = 0.8`. Write one sentence explaining why the two required sample sizes differ so much.

4.12 Writing exercise

Using the cholesterol confidence interval from Exercise 1 and the `strep_tb` arm-improvement confidence interval from earlier in this chapter, write one sentence for each, in the same reporting style. Use the correct units, and don't describe the interval as a probability statement about where the true value falls.

Solutions

Exercise 1

```r
chol_stats <- nhanes |>
  filter(!is.na(chol)) |>
  summarise(
    mean_chol = mean(chol),
    sd_chol = sd(chol),
    n_chol = n()
  ) |>
  mutate(
    se_chol = sd_chol / sqrt(n_chol),
    ci_lower = mean_chol - 1.96 * se_chol,
    ci_upper = mean_chol + 1.96 * se_chol
  )

chol_stats
```

```
# A tibble: 1 x 6
  mean_chol sd_chol n_chol se_chol ci_lower ci_upper
      <dbl>   <dbl>  <int>   <dbl>    <dbl>    <dbl>
1      4.97    1.09  39176 0.00551     4.96     4.98
```

```r
t.test(nhanes$chol)
```

```


	One Sample t-test

data:  nhanes$chol
t = 902.16, df = 39175, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.962511 4.984121
sample estimates:
```

```
mean of x
 4.973316
```

Mean total cholesterol in this sample was 4.97 mmol/L (95% CI: 4.96 to 4.98 mmol/L). For the difference between the two smoking groups:

```r
chol_smoking <- nhanes |>
  filter(smoking %in% c('Never', 'Current'))

t.test(chol ~ smoking, data = chol_smoking)
```

```

	Welch Two Sample t-test

data:  chol by smoking
t = -1.9231, df = 13580, p-value = 0.05449
alternative hypothesis: true difference in means between group Never and group Current is n
95 percent confidence interval:
 -0.0561225720  0.0005350405
sample estimates:
  mean in group Never mean in group Current
             4.967995              4.995789
```

`smoking` still has three levels after the `filter()`, but only two of them have any rows left, and `t.test()` rebuilds the factor from the data it's actually given, so the empty third group drops out on its own.

Check the direction before reading the interval, as above. `'Never'` comes first in `smoking`'s levels, so R has subtracted current smokers from never smokers, and the difference is never minus current.

Mean total cholesterol was -0.028 mmol/L in never smokers compared with current smokers (95% CI: -0.056 to 0.001 mmol/L), so a little lower on average in those who had never smoked. Notice that the estimate sits inside its own interval. Always check that: if it doesn't, something is wrong, likely a sign has been flipped somewhere. The interval includes zero, so these data are consistent with no difference between the two groups, though they don't rule out a small one either.

The two intervals mean different things. The first is about a single quantity, mean cholesterol, and says how precisely this sample pins it down. The second is about a *comparison* between two groups, and the value that matters in it is zero: an interval containing zero is one where the data can't establish which group is higher. From Chapter 5 onward, almost every interval in this course is of the second kind.

Exercise 2

```r
cav_stats <- strep_tb |>
  summarise(
    n_cav = sum(baseline_cavitation == 'yes'),
    n_total = n(),
    p_cav = n_cav / n_total,
    se_cav = sqrt(p_cav * (1 - p_cav) / n_total)
  )

cav_stats
```

```
# A tibble: 1 x 4
  n_cav n_total p_cav se_cav
  <int>   <int> <dbl>  <dbl>
1    62     107 0.579 0.0477
```

```r
cav_stats$p_cav - 1.96 * cav_stats$se_cav
```

```
[1] 0.4859025
```

```r
cav_stats$p_cav + 1.96 * cav_stats$se_cav
```

```
[1] 0.6729761
```

```r
prop.test(x = cav_stats$n_cav, n = cav_stats$n_total)
```

```
	1-sample proportions test with continuity correction

data:  cav_stats$n_cav out of cav_stats$n_total, null probability 0.5
X-squared = 2.3925, df = 1, p-value = 0.1219
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.4801030 0.6729991
sample estimates:
        p
0.5794393
```

```r
gender_summary <- strep_tb |>
  group_by(gender) |>
  summarise(n_gender = n(), n_improved = sum(improved))
gender_summary
```

```
# A tibble: 2 x 3
  gender n_gender n_improved
  <fct>     <int>      <int>
1 F            59         28
2 M            48         27
```

```
prop.test(x = gender_summary$n_improved, n = gender_summary$n_gender)
```

```

    2-sample test for equality of proportions with continuity
    correction

data:  gender_summary$n_improved out of gender_summary$n_gender
X-squared = 0.50491, df = 1, p-value = 0.4773
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.2963678  0.1205203
sample estimates:
   prop 1    prop 2
0.4745763 0.5625000
```

The hand-built interval and the one from `prop.test()` agree closely, as they did for the proportion improved earlier in the chapter.
Just over half of patients (58%) had cavitation present on their baseline chest X-ray, but the 95% confidence interval includes 50%, so these data don't rule out the true proportion being exactly one half. The confidence interval for the difference in improvement between women (`prop 1`) and men (`prop 2`) includes zero, consistent with little or no difference in outcome by gender in this trial.
Exercise 3

```r
tt_sed_full <- t.test(nhanes$sed)

set.seed(1)
sed_50 <- nhanes |>
  filter(!is.na(sed)) |>
  slice_sample(prop = 0.5)
tt_sed_50 <- t.test(sed_50$sed)

set.seed(1)
sed_10 <- nhanes |>
  filter(!is.na(sed)) |>
  slice_sample(prop = 0.1)
tt_sed_10 <- t.test(sed_10$sed)

data.frame(
  sample = c('Full', '50% subset', '10% subset'),
  n = c(sum(!is.na(nhanes$sed)), nrow(sed_50), nrow(sed_10)),
  lower = c(tt_sed_full$conf.int[1], tt_sed_50$conf.int[1],
            tt_sed_10$conf.int[1]),
  upper = c(tt_sed_full$conf.int[2], tt_sed_50$conf.int[2],
            tt_sed_10$conf.int[2])
) |>
  mutate(width = upper - lower)
```

```
      sample     n    lower    upper     width
1       Full 34434 6.449124 6.679429 0.2303057
2 50% subset 17217 6.403063 6.728992 0.3259286
3 10% subset  3443 6.219105 6.983209 0.7641046
```

As with **chol**, the point estimate is broadly similar across all three samples, but the confidence interval widens substantially in the 10% subset, and by almost exactly the same factor of about three. The same pattern holds for a skewed variable as for a symmetric one, because it's the *sample size*, not the shape of the raw data, that governs the width of the interval on the mean.

Exercise 4

```r
power.t.test(delta = 0.3, sd = chol_stats$sd_chol, sig.level = 0.05, power = 0.8)
```

```
     Two-sample t test power calculation
```

```
             n = 208.6194
         delta = 0.3
            sd = 1.091122
     sig.level = 0.05
         power = 0.8
   alternative = two.sided

NOTE: n is number in *each* group
```

```r
power.t.test(delta = 0.8, sd = chol_stats$sd_chol, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

             n = 30.19253
         delta = 0.8
            sd = 1.091122
     sig.level = 0.05
         power = 0.8
   alternative = two.sided

NOTE: n is number in *each* group
```

Detecting the smaller difference (0.3 mmol/L) needs a much larger sample than detecting the larger difference (0.8 mmol/L). Smaller effects are harder to distinguish from chance variation, so we need more participants to have the same 80% chance of detecting them.

Writing exercise (sample answer)

Mean total cholesterol was 4.97 mmol/L (95% CI: 4.96 to 4.98 mmol/L). In the streptomycin trial, the proportion of patients improved was about 36 percentage points higher in the streptomycin arm than in the control arm, with a 95% confidence interval for the difference that excludes zero. This is consistent with a real difference in improvement between the two arms, rather than one attributable to chance alone.

5 Hypothesis tests and measures of association (Week 9)

A confidence interval says how precise an estimate is. It doesn't say whether the effect behind it is distinguishable from none at all: two groups' cholesterol means could sit half a millimole apart with intervals that overlap, and whether that gap is a real difference or ordinary sampling noise is exactly the question Chapter 4 leaves open. A hypothesis test closes it, and gets reported alongside the interval rather than in place of it. Binary outcomes raise a version of the same question the descriptive statistics from Chapter 3 can't answer on their own: is a health outcome more common in one group than another, and by how much? The risk ratio and the odds ratio from epidemiology measure that, and here we attach a confidence interval and a p-value to each.

Every test in this chapter runs on the same logic underneath: a null hypothesis, an alternative, a test statistic, and a p-value stating how surprising the observed data would be if the null were actually true. Which test applies depends on the shape of the data in front of us, one measurement per person or two, one group or several, continuous or count, and telling those apart correctly matters as much as running the test itself. Reported together, an effect size, its 95% confidence interval, and a p-value make up the standard this book writes results in from here on.

5.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Perform and interpret a one-sample, paired, and independent-samples t-test using `t.test()`
2. Construct a contingency table and perform a chi-squared test using `table()` and `chisq.test()`
3. Determine when the above tests are valid, and when a non-parametric or exact alternative is needed (`wilcox.test()`, `fisher.test()`)
4. Calculate risk ratios and odds ratios with confidence intervals using `epitools::riskratio()` and `epitools::oddsratio()`
5. Write up a test result in an appropriate academic style

5.2 Setup

As in Chapters 3 and 4, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary. We also bring back `strep_tb`, from the `medicaldata` package, for the categorical half of this chapter.

This chapter adds one new package, `epitools`. Install it with `install.packages('epitools')` in the console, the same way as any other package, if it isn't on your machine already. Once it's installed, `library(epitools)` below makes its functions available for this session.

```r
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)
library(medicaldata)
library(epitools)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

```r
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c('Male', 'Female')),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c('Mexican American', 'Other Hispanic', 'White', 'Black', 'Others')),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c('Less than high school', 'High school graduate', 'Above high school')),
    smoking = factor(smoking,
      levels = c('Never', 'Previous', 'Current')),
    phq.c = factor(phq.c,
      levels = c('0 Normal', '1 Mild symptoms', '2 Possible depression'),
      labels = c('Normal', 'Mild symptoms', 'Possible depression'),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c('No', 'Yes'))
  )
```

This is the same recode we introduced in Chapters 3 and 4. See Chapter 3 for the reasoning behind each variable.

5.3 The logic of a test

A classical hypothesis test starts from two competing statements about the **population**. The **null hypothesis**, written H0, says there's nothing going on: no difference between groups, or no association between variables. The exact wording depends on what is being tested, but it always describes the population, never the sample we happen to have measured. The **alternative hypothesis**, H1, says there is a difference. A one-sided alternative hypothesis, specifying the direction of the difference in advance, exists too, but is rarely used in practice; every test in this chapter uses the two-sided form. A test never proves the null hypothesis true. It only asks whether the pattern shown from the sample is surprising enough to reject the null hypothesis.

A test turns the sample data into a single number, a **test statistic**, that measures how far the data sit from what the null hypothesis predicts under the sampling distribution. R then converts that statistic into a **p-value**: the probability of seeing that test statistic at least this extreme, if the null hypothesis were actually true. A p-value is not the probability that the null hypothesis is true, and it's not the probability the result happened by chance. A small p-value, conventionally below 0.05, means the data would be unusual under the null hypothesis, so we reject it in favour of the alternative. A p-value alone says nothing about how large an effect is, only about how surprising it is.

A confidence interval and a hypothesis test carry the same information, read two different ways. If a 95% confidence interval for a difference excludes the null value, 0 for a difference, 1 for a risk ratio or odds ratio, the matching test rejects the null hypothesis at the 5% level, and vice versa. That's why Chapter 4 could already show us a real effect using intervals alone, before this chapter ever named a null hypothesis. From here on, every test in this book states its null and alternative hypothesis in words before running the test, always as a statement about the population, never about the sample or the particular study, and every result is reported the way the seminar set out: an effect size, a confidence interval, and a p-value, together.

5.4 One-sample t-test: total cholesterol against a clinical benchmark

Guideline bodies often set a single reference value for a measurement: a 'desirable' total cholesterol of under 5.2 mmol/L is a standard clinical benchmark. A one-sample t-test asks whether the population's mean differs from a single fixed value like that one, instead of from another group's mean.

Null hypothesis. Mean total cholesterol in the population is 5.2 mmol/L.

Alternative hypothesis. Mean total cholesterol in the population is not 5.2 mmol/L.

`t.test()` runs a one-sample test when we give it one variable and a value to test it against, `mu`:

```r
t.test(nhanes$chol, mu = 5.2)
```

```

	One Sample t-test

data:  nhanes$chol
t = -41.12, df = 39175, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 5.2
95 percent confidence interval:
 4.962511 4.984121
sample estimates:
mean of x 
 4.973316 
```

R prints a 95% confidence interval for the mean itself, 4.96 to 4.98 mmol/L, not for the quantity the test is actually about, which is how far that mean sits from the 5.2 mmol/L benchmark. Subtracting 5.2 from both ends turns it into the effect size we want: mean total cholesterol in this sample is about 4.97 mmol/L, 0.23 mmol/L below the 5.2 mmol/L benchmark (95% CI: 0.22 to 0.24 mmol/L below, $p < 0.001$). This is the same move the independent-samples section further on makes with its own confidence interval: read what R printed, then work out what it means for the comparison actually being asked.

A one-sample t-test assumes the **sampling distribution** of the mean is roughly normally distributed, not that `chol` itself is. We never see a sampling distribution directly, since we only have the one sample, so the shape of the data is the evidence we have about it. The central limit theorem says the sampling distribution of the mean gets closer to normal as the sample grows, whatever shape the data themselves have, and the more skewed the data, the larger a sample that takes. Chapter 3's histogram of `chol` showed a distribution with a mild right skew, not a perfectly symmetric bell shape, but with a sample this large, tens of thousands of participants, that skew is easily large enough for the theorem to apply. So the question a histogram answers here is not 'is `chol` normal?' but 'is this sample large enough, given how skewed `chol` is?', and for `chol` the answer is a comfortable yes. `wilcox.test()` is the non-parametric alternative, for a smaller sample (typically <30) or a more pronounced skew where that reassurance is less safe.

Null hypothesis (`wilcox.test()`). The population median of `chol` is 5.2 mmol/L.

Alternative hypothesis. The population median is not 5.2 mmol/L.

wilcox.test(), from base R, takes the same mu argument for a one-sample test. Rather than a mean, it tests a **median**, using the ranks of how far each value sits from mu, so one extreme value can't dominate the result the way it can dominate a mean:

```r
wilcox.test(nhanes$chol, mu = 5.2)
```

```
	Wilcoxon signed rank test with continuity correction

data:  nhanes$chol
V = 268972135, p-value < 2.2e-16
alternative hypothesis: true location is not equal to 5.2
```

wilcox.test() returns a p-value and nothing else, no effect size, so the size of the effect has to come from a descriptive statistic that matches what the test is about: a median, not a mean. The sample median of chol is 4.89 mmol/L (IQR: 4.22 to 5.61 mmol/L), below the 5.2 mmol/L benchmark, consistent with the p-value far below 0.001. Typical cholesterol in this sample sits below the clinical benchmark whichever way we measure the middle of the distribution. With a sample this large, a t-test and its non-parametric alternative usually agree, but we should stay alert for when the central limit theorem's reassurance runs thin, in a smaller sample or a more skewed one.

5.5 Paired t-test: two domains of activity in one person

nhanes_mph records physical activity as two separate totals for each participant, both in MET-hours/day: met_w, activity at work, and met_r, activity outside work, exactly as the dictionary describes it. These are two different domains of activity, measured at the same time, in the same person.

That distinction decides which test applies. The one-sample test above compared one variable against a fixed value. Here there's no fixed value: two measurements to compare, both taken on the same people. That's not the same as comparing two separate groups, an independent-samples test, introduced later in this chapter. Every met_w value here has exactly one matching met_r value, from the same participant. The right tool is a paired t-test. Instead of comparing the two columns directly, it collapses them into a single column of within-person differences (met_w minus met_r) and tests whether that difference is, on average, zero.

Null hypothesis. In the population, mean work activity and mean outside-work activity are the same: the mean within-person difference is zero.

Alternative hypothesis. The mean within-person difference is not zero.

t.test() runs a paired test when we set paired = TRUE:

```
t.test(nhanes$met_w, nhanes$met_r, paired = TRUE)
```

```

	Paired t-test

data:  nhanes$met_w and nhanes$met_r
t = 43.111, df = 34284, p-value < 2.2e-16
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 2.927984 3.206905
sample estimates:
mean difference
       3.067444
```

On average, work activity ran about 3.07 MET-hours/day higher than activity outside work (95% CI: 2.93 to 3.21 MET-hours/day, $p < 0.001$).

A paired t-test also arises in a before-and-after design, where the same measurement is taken at two time points on the same person. That is a different study design from the one here, even though the R code looks identical. There is no repeated measurement over time in this data, and describing the result as though there were would plant a habit that causes real trouble in a longitudinal design later on.

We can also get the paired t-test result by running an ordinary one-sample t-test on the differences directly, testing whether `met_w - met_r` has a mean of zero, exactly the calculation the one-sample section above just introduced:

```
t.test(nhanes$met_w - nhanes$met_r)
```

```

	One Sample t-test

data:  nhanes$met_w - nhanes$met_r
t = 43.111, df = 34284, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 2.927984 3.206905
sample estimates:
mean of x
 3.067444
```

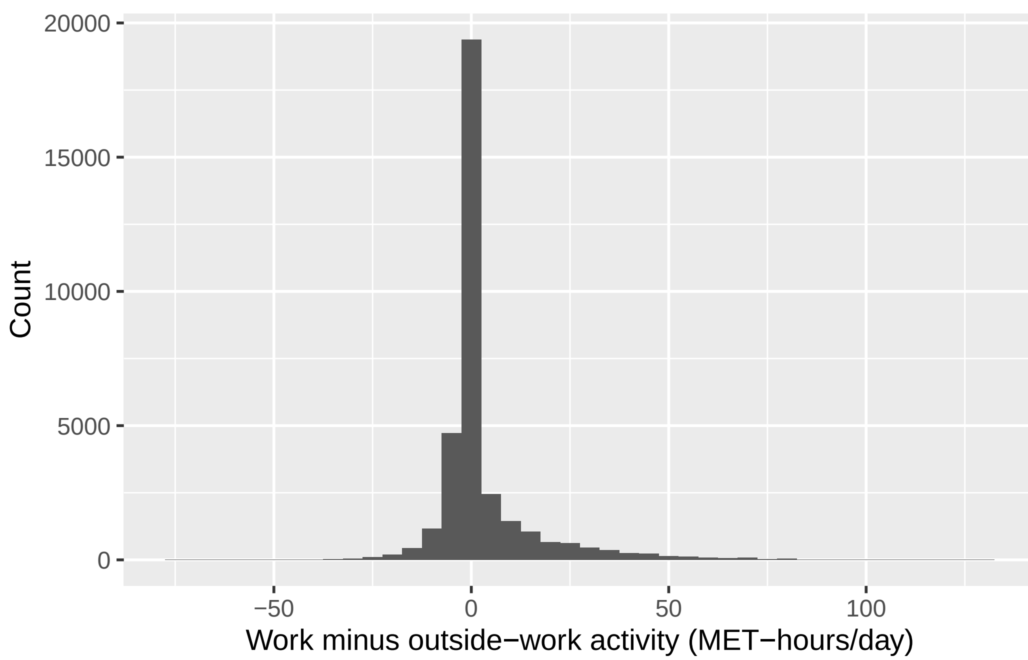
The t-statistic, degrees of freedom, and p-value are identical to the paired test above. That's not a coincidence: `paired = TRUE` is doing exactly this calculation internally, collapsing the two columns into one column of differences first.

5.6 A non-parametric alternative: the Wilcoxon signed-rank test

A paired t-test assumes the sampling distribution of the within-person difference is at least roughly normally distributed, the same kind of claim the one-sample section above explained. Let's look at the data's distribution, this time with a histogram, since the within-person difference is a new variable we haven't looked at yet:

```
ggplot(nhanes |> mutate(met_diff = met_w - met_r), aes(x = met_diff)) +
  geom_histogram(binwidth = 5) +
  labs(x = 'Work minus outside-work activity (MET-hours/day)', y = 'Count')
```

```
Warning: Removed 7480 rows containing non-finite outside the scale range
(`stat_bin()`).
```



The distribution of differences is heavily skewed, and a large share of participants sit exactly at zero. Many people report no occupational activity at all, so `met_w` and `met_r` are equal for them. The paired t-test may still be fine, because the sample size is large enough for the central limit theorem to apply, but a skew this pronounced is a good reason to check the conclusion against a test that doesn't lean on that theorem at all.

`wilcox.test()` also tests the same question as the paired t-test here, without assuming a normal distribution. It works with the ranks of the differences rather than their average, so one extreme value or an oddly-shaped distribution can't dominate the result the way it can dominate a mean.

Null hypothesis. In the population, work activity and outside-work activity are equally likely to be the larger of the pair: there's no systematic difference within the pair.

Alternative hypothesis. One is systematically larger than the other.

With `paired = TRUE`, it runs the paired version, known as the Wilcoxon signed-rank test:

```
wilcox.test(nhanes$met_w, nhanes$met_r, paired = TRUE)
```

```

	Wilcoxon signed rank test with continuity correction

data:  nhanes$met_w and nhanes$met_r
V = 169408552, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

The conclusion doesn't change here: both tests reject the null hypothesis convincingly, each with a p-value far below 0.001, and the paired t-test's own effect size above, work activity running about 3.07 MET-hours/day higher, still stands as the effect-size figure to report alongside either p-value. With a sample this large, the t-test and the Wilcoxon test agree despite the skew. That won't always be true. With a smaller sample, or a more extreme distribution, the two tests can disagree, and the non-parametric result is usually the one to trust when the normality assumption is in real doubt.

5.7 Independent-samples t-test: cholesterol by gender

Unlike the paired comparison just above, an independent-samples t-test compares two entirely separate groups of people, with no link between a value in one group and a value in the other. `t.test()` compares two independent groups directly. We already used it in Chapter 4 to build a confidence interval for the difference in total cholesterol (`chol`, in mmol/L) between men and women.

Null hypothesis. Mean total cholesterol is the same in men and women in the population.

Alternative hypothesis. Mean total cholesterol differs between men and women in the population.

```
t.test(chol ~ gender, data = nhanes |> filter(!is.na(gender)))
```

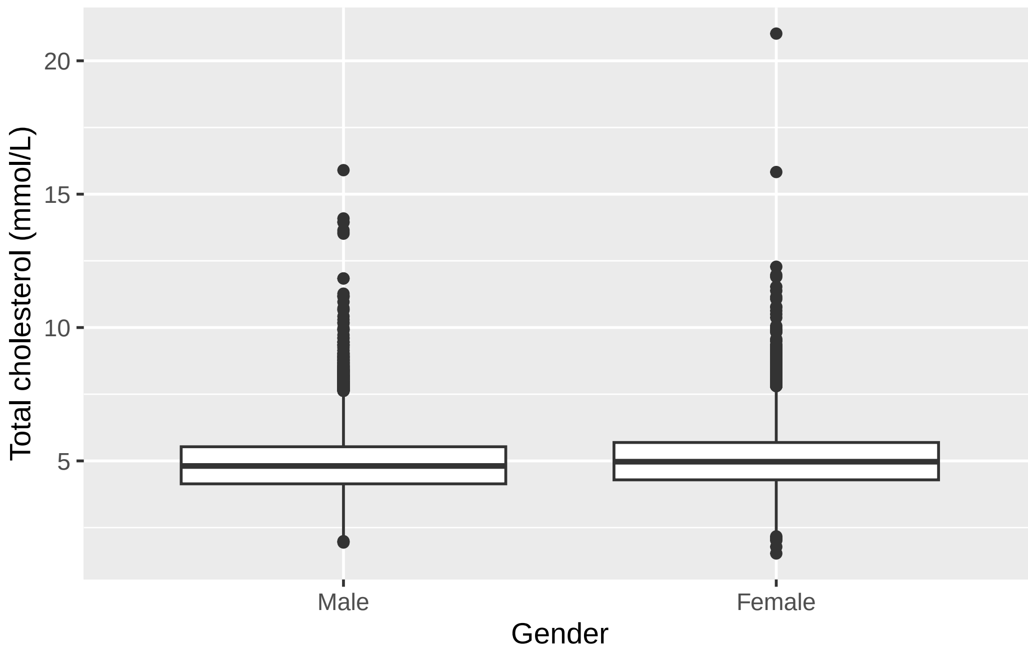
```

	Welch Two Sample t-test

data:  chol by gender
t = -15.318, df = 38908, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
 -0.1901172 -0.1469821
sample estimates:
  mean in group Male mean in group Female
            4.885939             5.054489
```

This is the exact same call as Chapter 4's. `t.test()` was already giving us a hypothesis test; Chapter 4 just hadn't introduced hypothesis testing yet, so we only looked at the confidence interval. Now we bring the rest into view: a t-statistic, degrees of freedom, and a p-value.

Before trusting a t-test, we check that the two groups aren't wildly different in spread or shape, the way Chapter 3's boxplot of `chol` by `gender` already let us do. This is a different check from the sampling-distribution question the one-sample and paired sections asked: there, the concern was whether one sampling distribution was close to normal; here, with two separate groups, the concern is whether the two groups look comparable enough in spread and shape for a difference in their means to be meaningful.

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Total cholesterol (mmol/L)')
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



The two boxplots are similar in spread and shape, just shifted slightly. Nothing here rules out a t-test. If, instead, the shape of the boxplot looks very different between the two groups, a t-test might not be appropriate, since it only compares the sample means. `t.test()` also doesn't assume equal variance between the two groups by default, which protects us even where the spreads differ more than they do here.

Before reading the interval, check the direction, exactly as Chapter 4 set out. The alternative-hypothesis line says *group Male and group Female*, because `'Male'` is the first level of `gender`, so R has printed men minus women and both ends come out negative. Reversing every sign together turns it into the more natural reading: mean total cholesterol was about 0.17 mmol/L higher in women than in men (95% CI: 0.15 to 0.19 mmol/L, $p < 0.001$). The two statements are the same result written from a different perspective. This is the reporting standard the seminar introduced: effect size, confidence interval, and p-value, together, every time, from here on. `p < 0.001` is how we write a p-value that R prints as something like `< 2.2e-16`. Convention in public health (or clinical medicine) reports a very small p-value this way rather than typing out the scientific notation.

5.8 Chi-squared test: `strep_tb`'s primary result

`strep_tb`'s primary outcome is binary: did chest X-ray improve at six months (`improved`), and does that depend on trial arm (`arm`)? `table()` builds a two-way count, the same way it did in Chapter 3, and `chisq.test()`, from base R, tests whether the two variables are associated.

Null hypothesis. The probability of improvement is the same in the population of patients receiving streptomycin and the population of patients receiving control treatment.

Alternative hypothesis. The probability of improvement is not the same in the two populations.

```
tab_arm <- table(strep_tb$arm, strep_tb$improved)
tab_arm
```

```
              FALSE TRUE
  Streptomycin    17   38
  Control         35   17
```

```
chisq.test(tab_arm, correct = FALSE)
```

```

	Pearson's Chi-squared test

data:  tab_arm
X-squared = 14.176, df = 1, p-value = 0.0001665
```

The chi-squared statistic is large and the p-value is small: the proportion improved is not the same in the two arms. A p-value alone doesn't say by how much; the risk ratio and odds ratio below put a size on that difference.

A chi-squared test also has an assumption to check. Every **expected** count should be reasonably large, conventionally at least 5. `chisq.test()` calculates these expected counts internally and stores them in `$expected`:

```
chisq.test(tab_arm, correct = FALSE)$expected
```

```
                 FALSE     TRUE
  Streptomycin 26.72897 28.27103
  Control      25.27103 26.72897
```

All four expected counts are comfortably above 5, so the test is on solid ground here. Expected counts run low typically in a small or very unevenly split sample. When that happens, `fisher.test()`, also from base R, is the alternative: it tests the same association exactly, rather than by approximation, and doesn't depend on that condition.

`correct = FALSE` switches off a specific adjustment. Left alone on a 2×2 table, `chisq.test()` applies **Yates' continuity correction** by default: it subtracts 0.5 from the absolute difference between each observed and expected count before squaring, a conservative adjustment that compensates for approximating whole-number counts with a smooth curve. It shrinks the test statistic slightly, so it makes the p-value larger.

We've already met it without being told what it was. Chapter 4's `prop.test()` on these same two arms was headed 'with continuity correction' too, and printed a p-value of 0.0003548. The uncorrected test above gives 0.000166: same data, same comparison, two different numbers, purely from this one default.

The correction earns its place when expected counts run low, typically down around 5 to 10. Ours aren't: the smallest expected count is above 25. So we switch it off here, and it's also what makes this p-value match the one `epitools` reports for the same table in the next section.

> **ℹ Try it yourself: two tests, one answer**
>
> Chapter 4's `prop.test()` compares two proportions; this section's `chisq.test()` tests association in a two-way table. Run `prop.test(tab_arm, correct = FALSE)` and compare its chi-squared statistic and p-value to `chisq.test(tab_arm, correct = FALSE)` above, digit for digit. Write one sentence on what that match tells us about how the two tests relate, for a 2×2 table like this one.
> `prop.test()` also prints something `chisq.test()` doesn't: a confidence interval for the difference between the two proportions. Read the two proportions R prints and check which category of `improved` they refer to, the same way Chapter 4 warned to check before feeding a table straight in. Reverse the signs if needed, the way the independent-samples section above reversed its own interval, and write a one-sentence results statement giving the risk difference between the two arms with its 95% confidence interval. Write a second sentence on why that statement is something `chisq.test()` alone could never have given us.

5.9 Fisher's exact test

This is the test to reach for when a chi-squared test's expected-count assumption fails, which in public health work usually means a small trial or a rare outcome, both common enough that knowing the fix in advance pays off. It tests the same null and alternative hypothesis as the chi-squared test above, independence versus dependence between arm and improvement, just by an exact calculation instead of an approximation. Let's try `fisher.test()` on this table too, even though nothing here required it. The call looks just like `chisq.test()`:

```
fisher.test(tab_arm)
```

```
    Fisher's Exact Test for Count Data

data:  tab_arm
p-value = 0.0002218
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.08866883 0.52737127
sample estimates:
odds ratio
 0.2207311
```

The p-value, 0.0002218, is close to the chi-squared test's 0.000166. Both lead to the same conclusion here, a clear association between arm and improvement. `riskratio()` and `oddsratio()`, met below, report a Fisher's exact p-value of their own for the same table, so this number will reappear.

On a larger table, with many categories instead of the 2×2 shape here, `fisher.test()` can become slow, because it enumerates every possible table with the same row and column totals to work out the exact probability. If it runs too slowly or refuses with an error on a bigger table, adding `simulate.p.value = TRUE` switches it to a Monte Carlo estimate instead of the exact calculation, trading a small amount of precision for speed. That isn't needed here.

5.10 Risk ratios and odds ratios, with confidence intervals

The epidemiology half already taught how to calculate a risk ratio and an odds ratio from a 2×2 table like `tab_arm` above, and what each one means. We don't re-derive those formulas here. What's new is attaching a confidence interval and a p-value to numbers we can already compute by hand, and checking that R's version agrees with ours.

`riskratio()` and `oddsratio()`, both from `epitools`, take a 2×2 table and compare its rows, treating the first row as the reference category everything else is measured against.

Null hypothesis. In the population, the risk ratio is 1, and the odds ratio is 1: no association between arm and improvement.

Alternative hypothesis. The risk ratio, and the odds ratio, differ from 1.

Note the null value sits at 1 for a risk ratio or an odds ratio, not 0. A ratio of 1 means the two groups' risk, or odds, are identical; a difference in means or a slope has its null at 0 instead, because there the null value means no gap rather than no ratio.

They expect the table a particular way round, and getting it wrong produces a perfectly ordinary-looking answer, not an error, so check the layout before building one. The **exposure**

goes in the rows, with the reference group first. The **outcome goes in the columns**, and the second column is taken as the event: the thing whose risk we're comparing. Our table has `arm` in the rows and `improved` in the columns. `improved` holds `TRUE`/`FALSE` values, and `table()` orders these `FALSE` then `TRUE`, so improvement lands in the second column exactly as needed. Print the table and look at it before passing it on. If the reference row or the event column is the wrong one, these functions still return a risk ratio, just an upside-down one.

`levels()`, from base R, lists a factor's categories in their current order, which is what we need to check before comparing rows:

```
levels(strep_tb$arm)
```

```
[1] "Streptomycin" "Control"
```

`Streptomycin` is listed first, which would make it the reference, backwards from what we want here. `relevel()`, met in Chapter 3, moves the level named in `ref` to the front, making it the new reference level. There we were only demonstrating it; here it changes the answer:

```
arm_control_ref <- relevel(strep_tb$arm, ref = 'Control')
levels(arm_control_ref)
```

```
[1] "Control"      "Streptomycin"
```

`Control` is now first, so it becomes the reference row: the one everything else is compared against. This is the same reference level idea we met with factors in Chapter 3, now doing real work instead of being demonstrated on a copy. Using it here, a second time before Chapter 6, is good practice: setting a sensible reference level is part of fitting a regression model there too.

We rebuild the table with the releveled factor, and pass it to `riskratio()`:

```
tab_arm_releveled <- table(arm_control_ref, strep_tb$improved)
tab_arm_releveled
```

```
arm_control_ref FALSE TRUE
   Control         35   17
   Streptomycin    17   38
```

```r
riskratio(tab_arm_releveled)
```

```
$data

arm_control_ref FALSE TRUE Total
   Control         35   17    52
   Streptomycin    17   38    55
   Total           52   55   107

$measure
               risk ratio with 95% C.I.
arm_control_ref estimate    lower    upper
   Control      1.000000       NA       NA
   Streptomycin 2.113369 1.377267 3.242893

$p.value
               two-sided
arm_control_ref   midp.exact fisher.exact  chi.square
   Control                NA           NA          NA
   Streptomycin 0.0001868959 0.0002217708 0.000166482

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

`oddsratio()` works the same way. We ask for `method = 'wald'`, which uses the same normal-approximation logic behind the $\pm$ 1.96 $\times$ SE intervals from Chapter 4. This matches the simple cross-product formula from the hand calculation directly:

```r
oddsratio(tab_arm_releveled, method = 'wald')
```

```
$data

arm_control_ref FALSE TRUE Total
   Control         35   17    52
   Streptomycin    17   38    55
   Total           52   55   107

$measure
```

```
               odds ratio with 95% C.I.
arm_control_ref estimate    lower   upper
   Control      1.000000       NA      NA
   Streptomycin 4.602076 2.038863 10.3877

$p.value
               two-sided
arm_control_ref   midp.exact fisher.exact  chi.square
   Control                NA           NA          NA
   Streptomycin 0.0001868959 0.0002217708 0.000166482

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

From the table above: 38 of 55 patients improved in the streptomycin arm (risk = 0.69), and 17 of 52 improved in the control arm (risk = 0.33). By hand, risk ratio = 0.69 / 0.33 = 2.11, and odds ratio = (38 × 35) / (17 × 17) = 4.60. Both match what `riskratio()` and `oddsratio()` just reported, with a confidence interval attached to each: a risk ratio of 2.11 (95% CI: 1.38 to 3.24) and an odds ratio of 4.60 (95% CI: 2.04 to 10.39). Neither interval comes close to including 1, the null value for both a risk ratio and an odds ratio, consistent with the chi-squared test above.

`riskratio()` and `oddsratio()` each also report three p-values alongside the estimate: a mid-p exact test, Fisher's exact test, and a chi-squared test. All three agree closely here. The chi-squared one is 0.000166, exactly the p-value `chisq.test(tab_arm, correct = FALSE)` gave us above, to every digit printed. That's only because we switched the continuity correction off. Had we left the correction on, `chisq.test()` would have reported 0.0003548, the number Chapter 4's `prop.test()` printed, while `epitools` reported 0.000166. There'd be no way to tell from the output that the two functions had simply made different default choices.

5.11 `gtsummary::tbl_summary()` with a test attached

`tbl_summary()`, already met in Chapter 3, can attach a test to a stratified table with `add_p()`, piped on afterwards:

```r
strep_tb |>
  select(arm, improved) |>
  tbl_summary(by = arm) |>
  add_p()
```

Table 5.1

Characteristic	Streptomycin N = 55	Control N = 52	p-value
improved	38 (69%)	17 (33%)	<0.001

[1] n (%)

[2] Pearson's Chi-squared test

This single call reproduces the count, the percentage, and the p-value we built by hand above, in the one-line form we'll reach for once the manual steps are familiar. The p-value matches to every digit because `add_p()` happens to default to the *uncorrected* chi-squared test here, the same choice we made by hand with `correct = FALSE`.

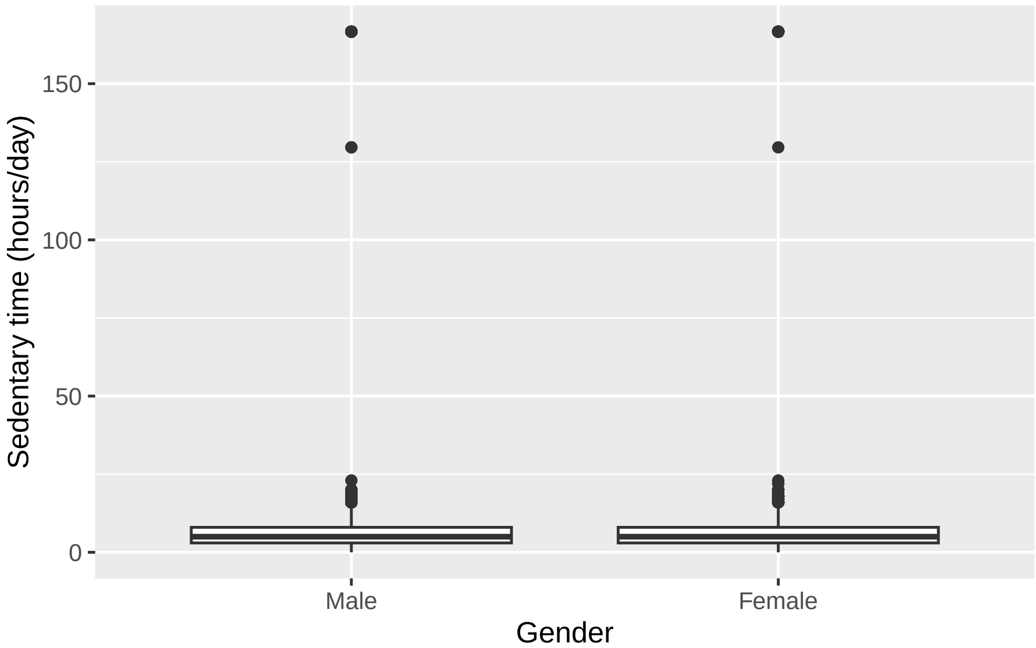
> **Try it yourself: sample size and significance**
>
> Using `slice_sample()` from Chapter 4, take a 10% random subsample of `nhanes` (remember to `set.seed()` first, so the subsample is reproducible) and re-run the `chol ~ gender` t-test from earlier in this chapter on it. Compare the p-value and the confidence interval to the ones from the full sample. Note what changed in one sentence, then explain in a second what it means for judging whether a small p-value reflects a large effect or simply a large sample.

5.12 Worked example: sedentary time by gender

Let's bring the independent-samples t-test together on a research question we haven't asked elsewhere in this chapter: does sedentary time (`sed`, in hours/day) differ between men and women?

```r
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = sed)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Sedentary time (hours/day)')
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```




```
t.test(sed ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```

	Welch Two Sample t-test

data:  sed by gender
t = -2.2193, df = 33827, p-value = 0.02648
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
 -0.48830744 -0.03028725
sample estimates:
  mean in group Male mean in group Female 
            6.430592             6.689889 
```

The boxplots overlap substantially, with similar spread in each group, so a t-test is reasonable here too. R prints the difference men minus women once again, so the interval on the page runs from -0.49 to -0.03. Put into a sentence, with the whole thing reversed: women reported slightly more sedentary time than men (mean difference about 0.26 hours/day, 95% CI: 0.03 to 0.49 hours/day, p = 0.026). The interval doesn't include zero, so the difference is statistically significant, but it's a small effect, about 15 minutes a day, on a scale of many hours a day.

5.13 Exercises

Exercise 1 (ILOs 1, 3). State the null and alternative hypotheses for a one-sample test of sedentary time (`sed`) against a reference value of 6 hours/day. Perform a one-sample t-test with `t.test()`, then repeat it with `wilcox.test()`. Report both results in one sentence each, each with its own effect size, and write a third sentence on why the mean and the median of `sed` might point to different conclusions about a typical participant's sedentary time.

Exercise 2 (ILOs 1, 3). State the null and alternative hypotheses for a paired comparison of `met_w` and `met_r`. Using `filter()`, restrict `nhanes` to participants whose sedentary time (`sed`) is above the sample median. On this subgroup, run both a paired t-test and a Wilcoxon signed-rank test comparing `met_w` and `met_r`. Write one sentence stating whether your conclusion changes in this more sedentary subgroup, and one sentence explaining why `met_w` and `met_r` are still a paired comparison here, not an independent one, even after filtering.

Exercise 3 (ILO 1). State the null and alternative hypotheses, then perform an independent-samples t-test comparing age (`age`) between men and women. Check the assumption graphically first with a boxplot. Report the result in a single sentence, and add one more sentence on whether a statistically significant result here is also a practically important one.

Exercise 4 (ILOs 2, 3). State the null and alternative hypotheses, then construct a contingency table of baseline cavitation (`baseline_cavitation`) against improvement (`improved`) in `strep_tb`. Run a chi-squared test, check the expected counts, and write one sentence interpreting the result.

Exercise 5 (ILOs 4, 5). State the null and alternative hypotheses for the risk ratio and the odds ratio, noting the scale their null value sits on. Using the same method as the `arm`/`improved` example above, check with `levels()` which category of `gender` is already the reference, then use `relevel()` to set it explicitly (even if it turns out to already be the one you want). Obtain a risk ratio and an odds ratio, each with a 95% confidence interval, for improvement in men compared with women. Reconcile the software output with a hand calculation from the same 2×2 table, and write a two-sentence results statement.

5.14 Writing exercise

Using the `arm`/`improved` result from earlier in this chapter, write two sentences for `strep_tb`'s primary outcome. State the proportion improved in each arm, and report the risk ratio or the odds ratio (your choice), its 95% confidence interval, and its p-value, together.

Solutions

Exercise 1
Null hypothesis. In the population, mean sedentary time is 6 hours/day (t-test); median sedentary time is 6 hours/day (`wilcox.test()`).
Alternative hypothesis. The mean, or the median, is not 6 hours/day.

```
t.test(nhanes$sed, mu = 6)
```

```

	One Sample t-test

data:  nhanes$sed
t = 9.6046, df = 34433, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 6
95 percent confidence interval:
 6.449124 6.679429
sample estimates:
mean of x 
 6.564277 
```

```
wilcox.test(nhanes$sed, mu = 6)
```

```

	Wilcoxon signed rank test with continuity correction

data:  nhanes$sed
V = 216498063, p-value < 2.2e-16
alternative hypothesis: true location is not equal to 6
```

R's confidence interval, 6.45 to 6.68 hours/day, is for the mean itself; subtracting the 6-hour benchmark from both ends gives the effect size: mean sedentary time is about 6.56 hours/day, 0.56 hours/day above 6 hours/day (95% CI: 0.45 to 0.68 hours/day above, $p < 0.001$). The Wilcoxon test is significant too ($p < 0.001$), but it's testing a different quantity, so its effect size is a median, not a mean: sample median 5 hours/day (IQR: 3 to 8 hours/day), *below* the 6-hour benchmark. The mean and the median disagree on which side of 6 hours 'typical' sedentary time falls. Sedentary time is right-skewed: a small number of participants report very high values, which pulls the mean up above the median, and above the benchmark, while the middle of the distribution, where most participants actually sit, remains below it. A typical participant, in the middle of the distribution, reports noticeably less sedentary time than the mean alone would suggest.
Exercise 2

Null hypothesis. In the population of participants in this more sedentary subgroup, the mean within-person difference between `met_w` and `met_r` is zero.
Alternative hypothesis. The mean within-person difference is not zero.

```
sed_above_median <- nhanes |>
  filter(!is.na(sed), sed > median(sed, na.rm = TRUE))

t.test(sed_above_median$met_w, sed_above_median$met_r, paired = TRUE)
```

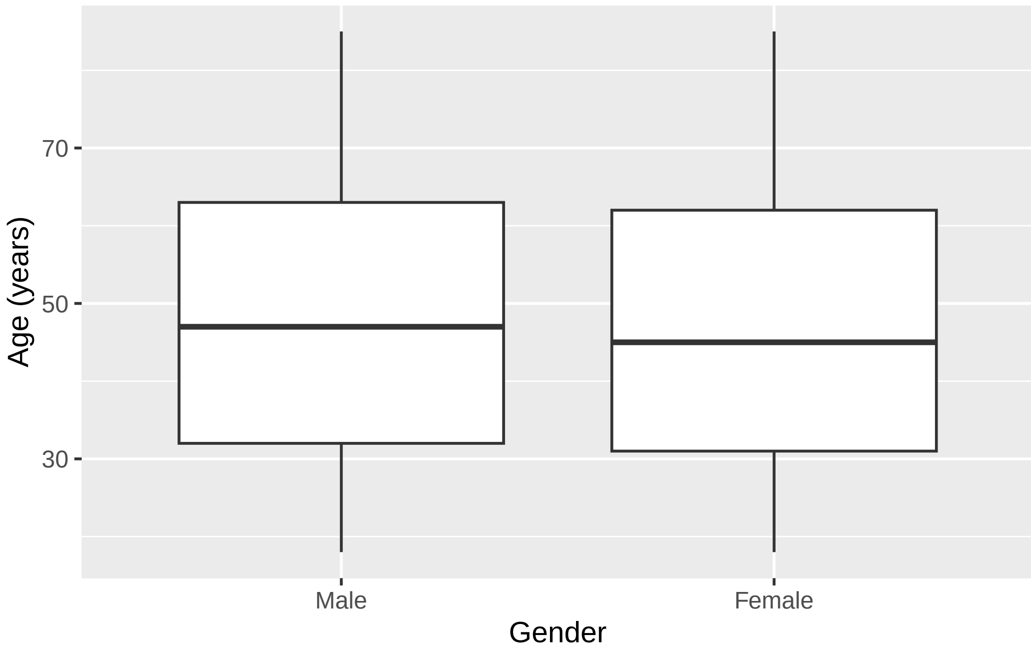
```

	Paired t-test

data:  sed_above_median$met_w and sed_above_median$met_r
t = 7.4471, df = 16724, p-value = 1e-13
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.3593059 0.6160133
sample estimates:
mean difference
      0.4876596
```

```
wilcox.test(sed_above_median$met_w, sed_above_median$met_r, paired = TRUE)
```

```

	Wilcoxon signed rank test with continuity correction

data:  sed_above_median$met_w and sed_above_median$met_r
V = 25597758, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

Both tests still reject the null hypothesis in this more sedentary subgroup, though the mean difference is much smaller than in the whole sample (about 0.49 MET-hours/day here, versus about 3.07 MET-hours/day overall). `met_w` and `met_r` remain two measurements taken on the same person at the same time, so the comparison is paired regardless of filtering.
Exercise 3
Null hypothesis. Mean age is the same in men and women in the population.
Alternative hypothesis. Mean age differs between men and women in the population.

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = age)) +
  geom_boxplot() +
  labs(x = 'Gender', y = 'Age (years)')
```




```
t.test(age ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```

	Welch Two Sample t-test

data:  age by gender
t = 4.0035, df = 41503, p-value = 6.251e-05
alternative hypothesis: true difference in means between group Male and group Female is not
95 percent confidence interval:
 0.3702765 1.0805737
sample estimates:
  mean in group Male mean in group Female 
            47.89355             47.16813 
```

This time R's interval is already positive, and that's informative: the difference is still men minus women, so a positive interval means the first group is the higher one. Men in this sample were on average about 0.7 years older than women (95% CI: 0.4 to 1.1 years, $p < 0.001$), and no signs need reversing to say so. The difference is statistically significant but tiny relative to a population averaging around 47 years old. With a sample this large, tens of thousands of participants, even a very small true difference will produce a small p-value. Statistical significance here is not the same as a practically important difference, and it doesn't really contradict Chapter 3's impression that age looked similar across the two groups.

Exercise 4

Null hypothesis. In the population, improvement is not associated with baseline cavitation.

Alternative hypothesis. Improvement depends on baseline cavitation.

```r
tab_cavitation <- table(strep_tb$baseline_cavitation, strep_tb$improved)
tab_cavitation
```

```
      FALSE TRUE
  no     17   28
  yes    35   27
```

```r
chisq.test(tab_cavitation, correct = FALSE)
```

```

	Pearson's Chi-squared test

data:  tab_cavitation
X-squared = 3.6399, df = 1, p-value = 0.05641
```

```r
chisq.test(tab_cavitation, correct = FALSE)$expected
```

```
         FALSE     TRUE
  no  21.86916 23.13084
  yes 30.13084 31.86916
```

All expected counts are comfortably above 5, the smallest being just under 22, so `correct = FALSE` is the right call here, for the same reason it was for the `arm` table.

The chi-squared test gives a p-value of about 0.056. That's above the conventional 0.05 threshold, but only just, and it would be a mistake to read much into which side of it the number landed on. The honest reading is the same either way: this is weak evidence of an association between baseline cavitation and improvement. The data don't establish a relationship, but they don't rule one out either. A trial of 107 patients simply cannot settle a question this size, and the result is best reported as inconclusive.

Exercise 5

Null hypothesis. In the population, the risk ratio is 1 and the odds ratio is 1: no association between gender and improvement.

Alternative hypothesis. The risk ratio, and the odds ratio, differ from 1. Both nulls sit at 1, not 0, since these are ratios instead of differences.

```r
levels(strep_tb$gender)
```

```
[1] "F" "M"
```

```r
gender_f_ref <- relevel(strep_tb$gender, ref = 'F')
tab_gender <- table(gender_f_ref, strep_tb$improved)
tab_gender
```

```

gender_f_ref FALSE TRUE
           F    31   28
           M    21   27
```

```r
riskratio(tab_gender)
```

```
$data

gender_f_ref FALSE TRUE Total
       F        31   28    59
       M        21   27    48
       Total    52   55   107

$measure
            risk ratio with 95% C.I.
gender_f_ref estimate     lower    upper
           F 1.000000        NA       NA
           M 1.185268 0.8215655 1.709979

$p.value
            two-sided
gender_f_ref midp.exact fisher.exact chi.square
           F         NA           NA         NA
           M  0.3745488    0.4378977   0.365451

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

```r
oddsratio(tab_gender, method = 'wald')
```

```
$data

gender_f_ref FALSE TRUE Total
       F        31   28    59
       M        21   27    48
       Total    52   55   107

$measure
            odds ratio with 95% C.I.
gender_f_ref estimate     lower    upper
           F 1.000000        NA       NA
           M 1.423469 0.6619167 3.061208

$p.value
            two-sided
gender_f_ref midp.exact fisher.exact chi.square
           F         NA           NA         NA
           M  0.3745488    0.4378977   0.365451

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

`levels()` shows that `'F'` is already listed first, so `relevel()` here confirms that choice explicitly. By hand, from the table: 27 of 48 men improved (risk = 0.5625) and 28 of 59 women improved (risk = 0.4746), giving a risk ratio of 0.5625 / 0.4746 = 1.19, and an odds ratio of $(27 \times 31) / (21 \times 28) = 1.42$. Both match the software output. The proportion improved was slightly higher in men than in women (risk ratio 1.19, 95% CI: 0.82 to 1.71; odds ratio 1.42, 95% CI: 0.66 to 3.06, chi-squared $p = 0.37$). But both confidence intervals include the null value of 1, so this trial gives no good evidence of a difference in improvement by gender. `oddsratio()` prints three p-values, as it did for the `arm` table; the chi-squared one is quoted here because that's the one that reconciles with `chisq.test()`.

Writing exercise (sample answer)

Among patients receiving streptomycin, 69% (38 of 55) showed radiological improvement at six months, compared with 33% (17 of 52) among those receiving the control treatment. This corresponds to a risk ratio of 2.11 (95% CI: 1.38 to 3.24, $p < 0.001$): about twice the proportion improved with streptomycin compared with control in this trial.

6 Correlation and simple linear regression (Week 10)

Chapter 5 compared two groups directly against each other, with cholesterol sorted into one of two buckets. Age and hours of sedentary time don't come in buckets: they're continuous, and the question about them isn't whether a group differs but whether the outcome moves as they move. Establishing that a straight line is a fair way to describe that movement, and not just a shape imposed on the data, is what this chapter builds toward.

We describe the relationship before modelling it, with a scatterplot and Pearson's correlation coefficient, plus Spearman's rank correlation for the case where one variable is ordered categories rather than a measurement. The model itself is a simple linear regression, $Y = \ + \ X + $: an outcome Y as an intercept plus a slope times a predictor X, with standing for the error the line doesn't capture, giving us an intercept, a slope with a confidence interval, and an R^2 saying how much of the outcome's variation was accounted for. Fitting that same model with a two-category predictor instead of a continuous one shows that Chapter 5's independent-samples t-test was this model all along, seen from a different angle. Residual and QQ plots check whether the assumptions behind the line hold up, and once we trust it, the model can predict: ask it about an input like the ones it was built on and the prediction means something, ask it about one far outside that range and it will still answer, just not reliably.

6.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Produce and interpret a scatterplot with a fitted regression line using `ggplot2`
2. Fit a simple linear regression with `lm()`, using a continuous, binary, or multi-category predictor
3. Explain why an independent-samples t-test and a simple linear regression on a binary predictor give equivalent results
4. Generate and interpret standard diagnostic plots
5. Generate predictions and identify when a prediction constitutes extrapolation
6. Extract coefficients and confidence intervals, and report a simple regression result in an appropriate academic style

6.2 Setup

As in Chapters 3 to 5, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary.

This chapter adds one new package, `broom`. If `broom` isn't already on your machine, run `install.packages('broom')` once in the console.

```
library(readr)
library(dplyr)
library(ggplot2)
library(here)
library(broom)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c('Male', 'Female')),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c('Mexican American', 'Other Hispanic', 'White', 'Black', 'Others')),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c('Less than high school', 'High school graduate', 'Above high school')),
    smoking = factor(smoking,
      levels = c('Never', 'Previous', 'Current')),
    phq.c = factor(phq.c,
      levels = c('0 Normal', '1 Mild symptoms', '2 Possible depression'),
      labels = c('Normal', 'Mild symptoms', 'Possible depression'),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c('No', 'Yes'))
  )
```

This is the same recode we introduced in Chapter 3 and repeated in Chapters 4 and 5. See Chapter 3 for the reasoning behind each variable and each choice of `ordered = TRUE`. We repeat it here, unchanged, so this chapter stands on its own too.

6.3 Correlation: Pearson's r and Spearman's rho

A correlation coefficient examines whether and to what extent two variables move together proportionally. The two variables are symmetric in a correlation: neither is assumed to be an 'outcome'.

`cor()`, from base R, calculates a correlation coefficient between two continuous variables. Its default method is Pearson's r, which measures the strength of a linear, or straight-line, association. We'll use the pairing this chapter's first model uses, `age` and `chol`:

```
cor(nhanes$age, nhanes$chol, use = 'complete.obs')
```

```
[1] 0.09398053
```

`use = 'complete.obs'` tells `cor()` to drop any participant missing either variable first; without it, a single missing value makes the whole calculation return `NA`. `cor.test()` adds a confidence interval and a p-value, and drops incomplete pairs automatically.

Null hypothesis. In the population, the true correlation between age and cholesterol is zero.

Alternative hypothesis. The true correlation is not zero.

```
cor.test(nhanes$age, nhanes$chol)
```

```

	Pearson's product-moment correlation

data:  nhanes$age and nhanes$chol
t = 18.684, df = 39174, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.08415645 0.10378634
sample estimates:
       cor 
0.09398053 
```

Pearson's r here is about 0.094 (95% CI 0.084 to 0.104, $p < 0.001$): a weak but non-zero positive correlation between age and cholesterol.

Pearson's r assumes both variables sit on a scale where equal units of increase or decrease mean the same thing throughout, i.e. an interval or ratio scale. `phq.c` doesn't fit with this assumption as ‘Possible depression’ isn’t a fixed distance from ‘Mild symptoms’ the way one extra year of age is from another, it’s an ordered category, not a measurement. Spearman’s rank correlation is the alternative, correlating ranks rather than raw values, so it only needs observations put in order, not a distance measured between them. `phq.c` is already an ordered factor, so `as.numeric()` from base R turns it into the numbers 1, 2, and 3 in the order the factor was built, the ranking Spearman’s correlation needs.

Null hypothesis. In the population, the true rank correlation between age and depression symptom category is zero.

Alternative hypothesis. The true rank correlation is not zero.

```
phq_numeric <- as.numeric(nhanes$phq.c)
cor.test(nhanes$age, phq_numeric, method = 'spearman')
```

```
Warning in cor.test.default(nhanes$age, phq_numeric, method = "spearman"):
cannot compute exact p-value with ties
```

```

	Spearman's rank correlation rho

data:  nhanes$age and phq_numeric
S = 8.5857e+12, p-value = 0.3725
alternative hypothesis: true rho is not equal to 0
sample estimates:
        rho 
0.004619514 
```

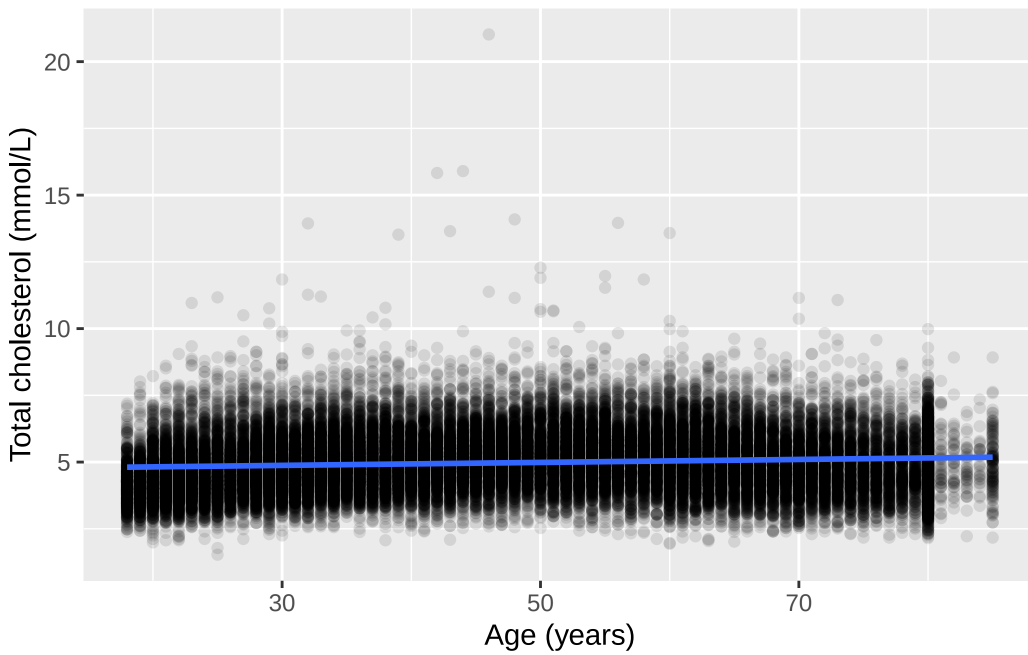
Spearman’s rho here is about 0.005, p = 0.37: close enough to zero, and far from significant, so age and depression symptom category show no clear monotonic relationship. `cor.test()` doesn’t report a confidence interval for Spearman’s rho the way it did for Pearson’s r above; rho has no equivalent simple formula for one. The warning above, ‘cannot compute exact p-value with ties,’ isn’t a problem: with tens of thousands of participants spread across only three `phq.c` categories, many necessarily share a rank, so R falls back on an approximate p-value, which is still valid.

6.4 A scatterplot with a fitted line: cholesterol by age

Chapter 3 plotted total cholesterol (`chol`, in mmol/L) against age (in years) as a plain scatterplot, with no line through it. `geom_smooth()`, from `ggplot2`, adds one. With `method = 'lm'`, it fits exactly the simple linear regression this chapter is about, and draws the result:

```r
ggplot(nhanes |> filter(!is.na(age), !is.na(chol)), aes(x = age, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = 'lm') +
  labs(x = 'Age (years)', y = 'Total cholesterol (mmol/L)')
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Above the plot, ggplot2 printed `` `geom_smooth()` using formula = 'y ~ x' ``. That's a message, not a warning: it's just reporting which formula it fitted, since `geom_smooth()` picks one for us if we don't say. Here it chose a straight line in `x`, exactly the simple linear regression we asked for with `method = 'lm'`.

The shaded band around the line is a 95% confidence interval for the line itself, the same idea as every other confidence interval in this course, just drawn as a band rather than printed as two numbers. The line slopes gently upward: cholesterol tends to be a little higher in older participants. The cloud of points around it is wide, though. Age doesn't determine cholesterol on its own, and the next section puts a number on exactly how much of the variation in cholesterol age can explain.

Try it yourself: what the band shows

Re-run the plot above with `geom_smooth(method = 'lm', se = FALSE)`. Then try `geom_smooth(method = 'lm', level = 0.80)`. Describe what disappeared in the first change, then what changed in the second, one sentence each.

6.5 Fitting the model with `lm()`

`lm()`, from base R, fits a linear model. Its first argument is a formula, in the same `y ~ x` form `t.test()` used in Chapter 4: the variable on the left is what we're modelling, and the variable on the right is what we're modelling it with. We drop the rows where either variable is missing first, the same habit we've used for every model and test so far.

Null hypothesis. In the population, the slope of cholesterol on age is zero: age carries no linear association with cholesterol.

Alternative hypothesis. The slope is not zero.

```r
age_complete <- nhanes |> filter(!is.na(age), !is.na(chol))
m_age <- lm(chol ~ age, data = age_complete)
summary(m_age)
```

```

Call:
lm(formula = chol ~ age, data = age_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.3177 -0.7469 -0.0877  0.6401 16.0554 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 4.7085437  0.0151970  309.83   <2e-16 ***
age         0.0055658  0.0002979   18.68   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.086 on 39174 degrees of freedom
Multiple R-squared:  0.008832,	Adjusted R-squared:  0.008807 
F-statistic: 349.1 on 1 and 39174 DF,  p-value: < 2.2e-16
```

As with `t.test()` in Chapter 4, there's no `$` anywhere in that call. `lm()`'s `data =` argument tells it to look for `chol` and `age` inside `age_complete`, so it finds them there directly instead of us pulling either one out by hand first. Every model in this course, `lm()` and `glm()` alike, follows this same pattern: name the variables bare in the formula, and point `data =` at wherever they live.

`summary()` prints a lot at once. The `Coefficients` table has the two numbers that define the fitted line: the `(Intercept)`, the model's predicted cholesterol at age zero, and the slope on `age`, the predicted change in cholesterol for each additional year of age. Each has a standard error, a t-value, and a p-value attached, exactly the reporting standard from Chapter 5. The slope here is small but non-zero: cholesterol tends to be about 0.0056 mmol/L higher for every extra year of age, and the p-value is far below 0.001.

`Multiple R-squared`, R^2, says what fraction of the variation in cholesterol this model accounts for. Here it's under 0.01: age explains under 1% of the variation in cholesterol between participants. That isn't a contradiction of the significant slope above. A relationship can be real, precisely estimated, and still leave most of the variation in the outcome unexplained. For a simple regression like this one, with a single predictor, the square of Pearson's r is exactly the same number as the R^2 above, and the hypothesis test for Pearson's r is equivalent to the test of the slope coefficient.

A low R^2 would mean a poorer explanation or prediction of the variation of the outcome. However, that isn't our goal here. Instead, we're estimating how cholesterol varies with age, as precisely as the data allow. Chapter 7 returns to this distinction, between explaining an association and predicting an outcome, when it looks at what a regression model can and can't claim to establish.

`coef()` and `confint()`, both base R, pull out just the coefficients and just their confidence intervals, without the rest of the printout:

```r
coef(m_age)
```

```
(Intercept)         age
4.708543749 0.005565764
```

```r
confint(m_age)
```

```
                  2.5 %      97.5 %
(Intercept) 4.678757321 4.738330177
age         0.004981885 0.006149643
```

`broom::tidy()` puts the same information into a tidy data frame, one row per coefficient, which is easier to work with in code than `summary()`'s printed output. Adding `conf.int = TRUE` includes the confidence interval as two extra columns:

```r
tidy(m_age, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)  4.71     0.0152       310.  0          4.68      4.74
2 age          0.00557  0.000298      18.7 1.46e-77   0.00498   0.00615
```

Reported in full, that reads: total cholesterol was on average 0.0056 mmol/L higher for each additional year of age (95% CI: 0.0050 to 0.0061 mmol/L, $p < 0.001$), although age accounted for less than 1% of the variation in cholesterol between participants.

6.6 A binary predictor: cholesterol by gender

`age` is a continuous predictor, taking any value across a range. `lm()` doesn't need that. A predictor on the right of the formula can be a two-category factor instead, and Chapter 5 already asked exactly this question with a t-test: does mean total cholesterol differ between men and women? Let's fit it as a regression and put the two answers side by side.

Null hypothesis. In the population, mean total cholesterol is the same in men and women.

Alternative hypothesis. Mean total cholesterol differs between men and women.

```r
gender_chol_complete <- nhanes |> filter(!is.na(gender), !is.na(chol))
m_gender <- lm(chol ~ gender, data = gender_chol_complete)
summary(m_gender)
```

```

Call:
lm(formula = chol ~ gender, data = gender_chol_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.5245 -0.7645 -0.0845  0.6441 15.9655 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)   4.88594    0.00792  616.90   <2e-16 ***
genderFemale  0.16855    0.01100   15.32   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.088 on 39174 degrees of freedom
Multiple R-squared:  0.005958,  Adjusted R-squared:  0.005932 
F-statistic: 234.8 on 1 and 39174 DF,  p-value: < 2.2e-16
```

```r
confint(m_gender)
```

```
                2.5 %    97.5 %
(Intercept)  4.8704159 4.9014630
genderFemale 0.1469893 0.1901101
```

With a continuous predictor like `age`, the `(Intercept)` was an abstract number, cholesterol at age zero, nobody's actual cholesterol. With a two-category predictor it's concrete: `gender`'s first level is `'Male'`, so `(Intercept)`, 4.8859 mmol/L, is simply mean cholesterol in men. This is the reference level again, the same idea Chapter 3 introduced and Chapter 5's `relevel()` work used a second time: whichever category is listed first becomes the one everything else in the model is compared against, and here that comparison is the whole model.

The slope, on `genderFemale`, is the difference between the two group means: 0.1685 mmol/L, women minus men, with its own standard error, t-value, and p-value, and a 95% confidence interval of 0.1470 to 0.1901 mmol/L. Chapter 5 reported the same comparison as 0.17 mmol/L higher in women (95% CI: 0.15 to 0.19 mmol/L), but only after reversing the signs on what `t.test()` printed, because `t.test()` reports Male minus Female by default. `lm()` reports the comparison the other way round, reference category first, so the sign comes out already the way we want to read it: the same underlying result, with no sign-flipping needed here.

The two results agree this closely because they're close to the same calculation, but not identically the same one. `t.test()`'s default doesn't assume the two groups share one variance, as Chapter 5's independent-samples section said explicitly; `lm()` does assume it, one shared residual variance for the whole model. Telling `t.test()` to make that same assumption, with `var.equal = TRUE`, closes the gap completely:

```r
t.test(chol ~ gender, data = nhanes |> filter(!is.na(gender)), var.equal = TRUE)
```

```

	Two Sample t-test

data:  chol by gender
t = -15.323, df = 39174, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
```

```
 -0.1901101 -0.1469893
sample estimates:
  mean in group Male mean in group Female 
            4.885939             5.054489 
```

`t = -15.323` on 39174 degrees of freedom, the same degrees of freedom `summary(m_gender)` reported for the regression, and an interval of -0.1901 to -0.1470, the exact mirror of `lm()`'s 0.1470 to 0.1901. `var.equal = TRUE` isn't a better version of the default. Chapter 5's default, unequal-variance form is the safer everyday choice, since it stays valid whether or not the two groups' spreads match. Setting `var.equal = TRUE` here is only to show that a t-test *is* this regression, once both make the same assumption.

An independent-samples t-test on cholesterol by gender, and a simple linear regression of cholesterol on the two-category factor `gender`, are the same model, read two ways. The diagnostic plots in the next section apply here too, and are worth trying on `m_gender`: with only two possible fitted values, the residuals-vs-fitted plot collapses into two vertical columns of points instead of the scattered cloud `m_age` produced, which is exactly what a two-category predictor should look like, not a sign that something has gone wrong.

6.7 A multi-level categorical predictor: cholesterol by education

`gender` has two categories, so `lm()` needed one extra row to describe it. A factor with more categories just needs more rows, one per non-reference level. `edu` has three: does mean cholesterol differ by education?

Null hypothesis. In the population, a non-reference education category's mean cholesterol equals the reference category's.

Alternative hypothesis. It differs. Each non-reference row in the coefficients table tests this comparison separately, one category against the reference.

```r
edu_chol_complete <- nhanes |> filter(!is.na(edu), !is.na(chol))
m_edu <- lm(chol ~ edu, data = edu_chol_complete)
summary(m_edu)
```

```

Call:
lm(formula = chol ~ edu, data = edu_chol_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4623 -0.7523 -0.0946  0.6477 16.0277 
```

```
Coefficients:
                         Estimate Std. Error t value Pr(>|t|)
(Intercept)               5.00462    0.01113 449.488   <2e-16 ***
eduHigh school graduate -0.01569    0.01610  -0.974    0.330
eduAbove high school    -0.01235    0.01357  -0.910    0.363
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.09 on 38080 degrees of freedom
Multiple R-squared:  2.988e-05, Adjusted R-squared:  -2.264e-05
F-statistic: 0.569 on 2 and 38080 DF,  p-value: 0.5661
```

```r
confint(m_edu)
```

```
                              2.5 %     97.5 %
(Intercept)               4.98280100 5.02644712
eduHigh school graduate -0.04724638 0.01586769
eduAbove high school    -0.03895027 0.01424346
```

Two rows this time, `eduHigh school graduate` and `eduAbove high school`, each a difference from whichever category is first in `levels(edu_chol_complete$edu)`. Neither is anywhere near significant here: on this evidence, education isn't associated with cholesterol in this sample.

`levels()` confirms the reference is `'Less than high school'`. `relevel()`, already met in Chapter 3, moves a different category to the front. This time we write the releveled factor back into a data frame, rather than working on a standalone copy the way Chapter 3 did: `lm()` needs to find `edu` already releveled inside whatever `data =` points at, so it goes into a new object, `edu_relevelled`, leaving `edu_chol_complete` and the model fitted on it untouched:

```r
edu_relevelled <- edu_chol_complete |>
  mutate(edu = relevel(edu, ref = 'Above high school'))
m_edu_relevel <- lm(chol ~ edu, data = edu_relevelled)
summary(m_edu_relevel)
```

```
Call:
lm(formula = chol ~ edu, data = edu_relevelled)

Residuals:
```

```
    Min      1Q  Median      3Q     Max 
-3.4623 -0.7523 -0.0946  0.6477 16.0277 

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)    
(Intercept)               4.992271   0.007757 643.600   <2e-16 ***
eduLess than high school  0.012353   0.013570   0.910    0.363    
eduHigh school graduate  -0.003336   0.013979  -0.239    0.811    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.09 on 38080 degrees of freedom
Multiple R-squared:  2.988e-05, Adjusted R-squared:  -2.264e-05 
F-statistic: 0.569 on 2 and 38080 DF,  p-value: 0.5661
```

```r
confint(m_edu_relevel)
```

```
                               2.5 %     97.5 %
(Intercept)               4.97706713 5.00747418
eduLess than high school -0.01424346 0.03895027
eduHigh school graduate  -0.03073571 0.02406383
```

Both rows changed, since they're now contrasts against a different category. But look at `Multiple R-squared` and `Residual standard error` in the two printouts: identical, 2.988e-05 and 1.09, to every digit. So is the F-statistic. Releveling didn't change what the model fits, only which comparisons it prints. The fitted values are the same for every participant either way; relabelling the reference category just changes which category each row is measured against, nothing about the cholesterol the model predicts for anyone.

Keep that distinct from a different invariant met again later, in Chapter 7: there, releveling one predictor inside a multivariable model doesn't move a *different* predictor's own coefficient. Both are true, and neither licenses reading a releveled row as if it were a causal effect on its own; a coefficient's meaning still depends on what it's being compared against and what else is in the model.

6.8 Checking assumptions: diagnostic plots

A fitted line is only trustworthy if the model's assumptions hold: linearity (a straight line is the right shape), constant variance (the scatter around the line doesn't change as we move along it), and normally distributed residuals (the vertical distances between the points and the line). A fourth assumption, independence between observations, isn't something a plot from one fitted model can check. It's a property of how the data were collected, and NHANES's sampling design is what would need inspecting for that, not this model's output.

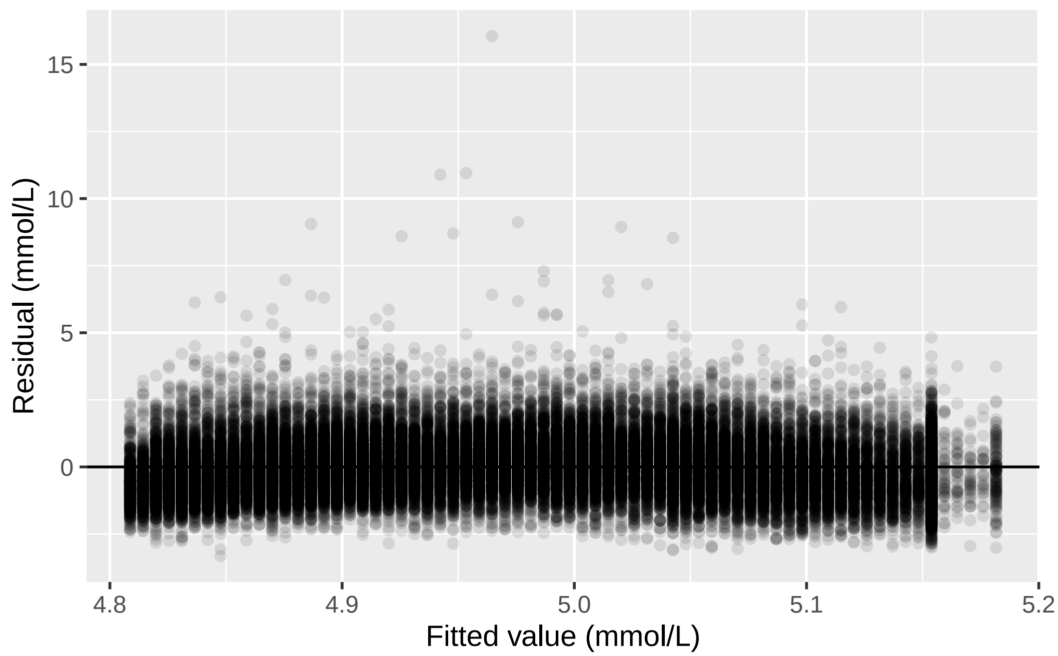
`broom::augment()` adds each observation's fitted value and residual back onto the original data, in columns named `.fitted` and `.resid`:

```
age_aug <- augment(m_age)
age_aug
```

```
# A tibble: 39,176 x 8
    chol   age .fitted .resid       .hat .sigma     .cooksd .std.resid
   <dbl> <dbl>   <dbl>  <dbl>      <dbl>  <dbl>       <dbl>      <dbl>
 1  4.03    37    4.91 -0.884 0.0000339   1.09 0.0000112       -0.814
 2  3.75    23    4.84 -1.09  0.0000709   1.09 0.0000355       -1.00
 3  5.04    38    4.92  0.120 0.0000324   1.09 0.000000198      0.110
 4  5.56    37    4.91  0.646 0.0000339   1.09 0.00000599       0.594
 5  5.66    27    4.86  0.801 0.0000573   1.09 0.0000156        0.738
 6  8.9     30    4.88  4.02  0.0000487   1.09 0.000335         3.70
 7  3.57    30    4.88 -1.31  0.0000487   1.09 0.0000352       -1.20
 8  6.57    30    4.88  1.69  0.0000487   1.09 0.0000593        1.56
 9  6.67    22    4.83  1.84  0.0000747   1.09 0.000107         1.69
10  5.9     29    4.87  1.03  0.0000515   1.09 0.0000231        0.948
# i 39,166 more rows
```

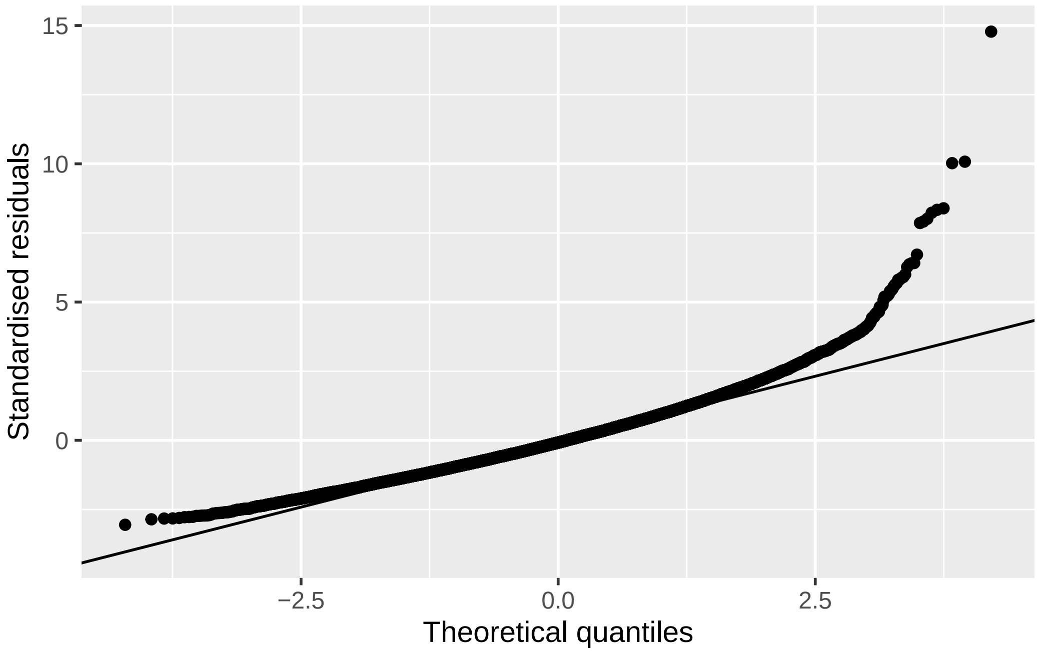
Plotting residuals against fitted values checks linearity and constant variance together. If the model is a good fit, the points should scatter evenly above and below zero, with no obvious curve and no obvious change in spread from left to right. `geom_hline()`, from `ggplot2`, draws a single horizontal line across the plot at whatever height `yintercept` names, and we put one at zero to make 'above and below' easy to judge by eye:

```
ggplot(age_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = 'Fitted value (mmol/L)', y = 'Residual (mmol/L)')
```



Most points sit in a fairly even band around zero, consistent with linearity and roughly constant variance. A small number of participants with very high cholesterol readings sit well above the line, though, visible here as points a long way above zero. Those same participants are worth keeping in mind for the QQ plot, which checks whether the residuals are approximately normally distributed. `.std.resid`, also added by `augment()`, standardises each residual to the same scale a normal distribution would use. `stat_qq()` and `stat_qq_line()`, both from `ggplot2`, plot the standardised residuals against what a perfect normal distribution would produce, with a reference line for comparison:

```
ggplot(age_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = 'Theoretical quantiles', y = 'Standardised residuals')
```



Points that fall on the line are consistent with normally distributed residuals. Here, the middle of the distribution sits close to the line, but the upper tail pulls away from it. That's the same small group of participants with unusually high cholesterol showing up again: the residuals are a little more heavy-tailed than a normal distribution would predict. With a sample this large, the model's estimates are still reliable despite this. With a much smaller sample, tail behaviour like this would be more of a concern.

6.9 Prediction and extrapolation from the model

`predict()`, from base R, uses a fitted model to generate a predicted value for a new set of inputs. Those inputs need to arrive as a data frame, with a column named to match each predictor in the model. `data.frame()`, also from base R, builds one directly from the values we supply, instead of reading one in from a file:

```
predict(m_age, newdata = data.frame(age = c(45, 150)), interval = 'confidence')
```

```
       fit      lwr      upr
1 4.959003 4.948142 4.969865
2 5.543408 5.482643 5.604174
```

`interval = 'confidence'` adds a confidence interval around each prediction. This confidence interval describes the precision of the predicted **mean** value, not the precision for an **individual**. The precision around an individual prediction is called a **prediction interval**. It's always wider than the confidence interval around the mean, since predicting one person's value is less certain than predicting the average across many. A prediction interval can be requested by stating `interval = 'prediction'` instead.

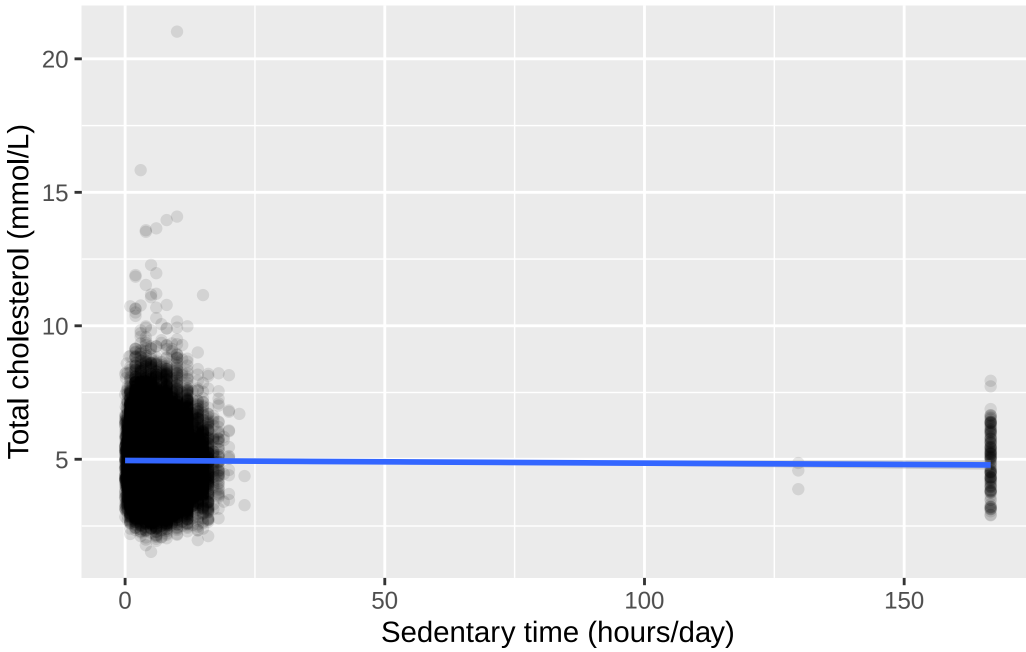
The first row, age 45, sits comfortably inside the range of ages actually observed in this sample (18 to 85), so this prediction is supported by data the model was fitted on. The second row, age 150, is not. Nobody in this dataset, or in any verified human population, is 150 years old. The model has no information about what happens at that age, and fitting a straight line through the data we do have says nothing reliable about what lies far outside it. The predicted value it returns looks like an ordinary number, which is exactly what makes extrapolation dangerous: a plausible-looking output doesn't mean the input that produced it was reasonable. Always check a prediction's input against the range the model was actually fitted on before trusting its output.

6.10 Worked example: cholesterol by sedentary time

Let's bring the whole workflow together on another research question: is sedentary time (`sed`, in hours/day) associated with total cholesterol (`chol`)?

```
ggplot(nhanes |> filter(!is.na(sed), !is.na(chol)), aes(x = sed, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = 'lm') +
  labs(x = 'Sedentary time (hours/day)', y = 'Total cholesterol (mmol/L)')
```

```
`geom_smooth()` using formula = 'y ~ x'
```



The fitted line is nearly flat, and the confidence band around it is wide enough to include a flat, horizontal line, that is, no relationship at all. Fitting the model directly confirms what the plot suggests:

```r
sed_complete <- nhanes |> filter(!is.na(sed), !is.na(chol))
m_sed <- lm(chol ~ sed, data = sed_complete)
summary(m_sed)
```

```
Call:
lm(formula = chol ~ sed, data = sed_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4193 -0.7563 -0.0893  0.6467 16.0757 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)  4.9543425  0.0070430 703.445   <2e-16 ***
sed         -0.0010015  0.0005582  -1.794   0.0728 .  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.083 on 32307 degrees of freedom
Multiple R-squared:  9.961e-05, Adjusted R-squared:  6.866e-05
F-statistic: 3.218 on 1 and 32307 DF,  p-value: 0.07282
```

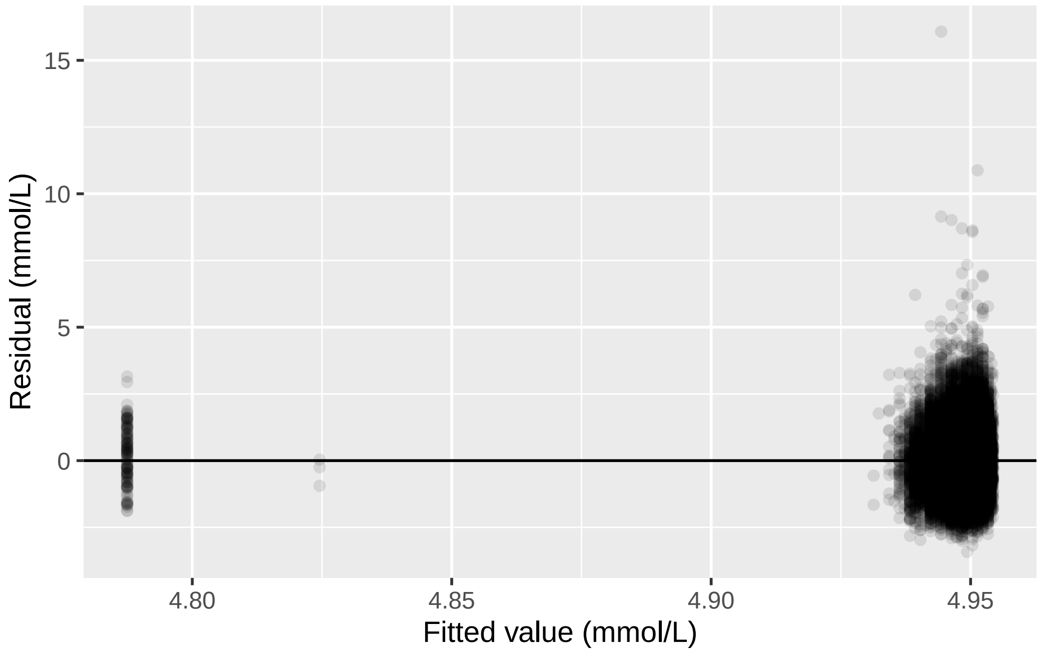
```r
confint(m_sed)
```

```
                   2.5 %       97.5 %
(Intercept)  4.940538020 4.968147e+00
sed         -0.002095651 9.269529e-05
```

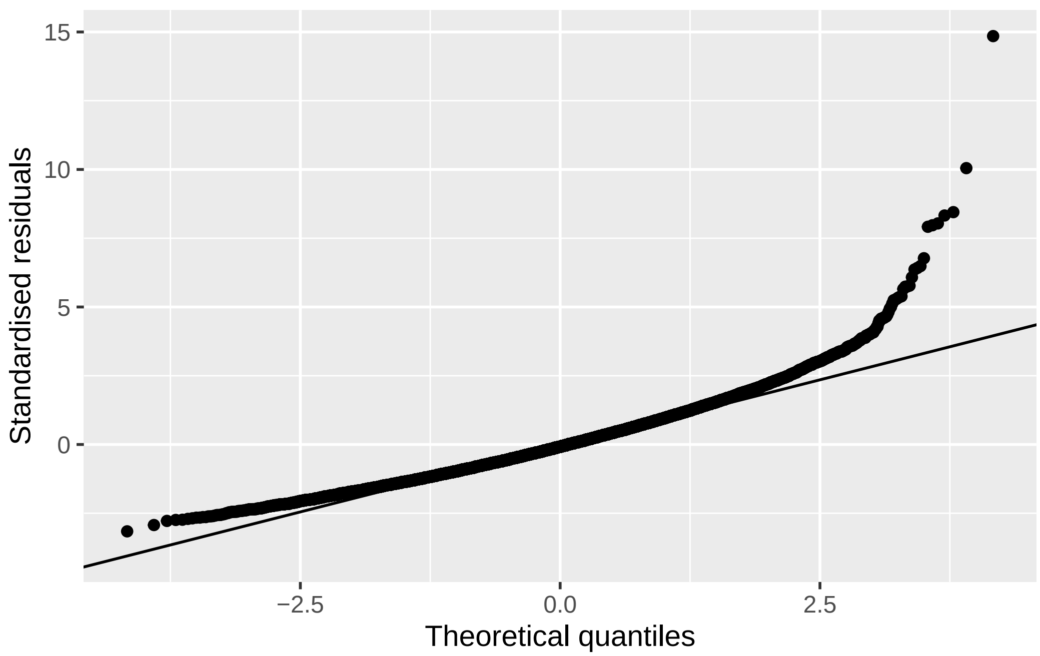
The slope is small and negative, about -0.0010 mmol/L for each additional hour/day of sedentary time, and the p-value is 0.073: above the conventional 0.05 threshold. The 95% confidence interval for the slope, -0.0021 to 0.0001 mmol/L, includes zero. R^2 is essentially zero. On this evidence alone, sedentary time and cholesterol show no clear linear relationship in this sample.

We check the diagnostic plots anyway, since a weak result could be caused by violated assumptions:

```r
sed_aug <- augment(m_sed)

ggplot(sed_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = 'Fitted value (mmol/L)', y = 'Residual (mmol/L)')
```




```
ggplot(sed_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = 'Theoretical quantiles', y = 'Standardised residuals')
```



The same pattern from the age model shows up again, more visibly. A handful of participants sit far from the fitted line, and `sed` itself carries a long right tail. Some participants recorded well over 24 hours of sedentary time a day, which cannot be literally true, and points to some noise in how this variable was recorded or reported. None of this overturns the conclusion above. It's a reason to hold the result loosely rather than to distrust the direction of the estimate.

In an academic report this becomes: sedentary time was not associated with total cholesterol in this sample (slope -0.0010 mmol/L per hour/day, 95% CI: -0.0021 to 0.0001 mmol/L, p = 0.073). But it would be a mistake to conclude from this alone that sedentary time has no bearing on cholesterol. This model adjusts for nothing else: age, gender, and other characteristics could still be masking a real association. Chapter 7 revisits this same pairing with those factors added back in.

6.11 Exercises

Exercise 1 (ILOs 1, 2, 6). State the null and alternative hypotheses for the slope. Using `vpa_r`, vigorous physical activity outside work (hours/day), produce a scatterplot of `chol` against `vpa_r` with a fitted regression line. Fit the model with `lm()`, and extract its coefficients and their 95% confidence intervals with `confint()` and `broom::tidy()`. Report the slope in one sentence, in the same style used above.

Exercise 2 (ILO 4). For the model fitted in Exercise 1, produce a residuals-vs-fitted plot and a QQ plot. Write one sentence describing what each plot shows, and one sentence commenting on anything the plots reveal about the shape of `vpa_r` itself.

Exercise 3 (ILO 5). Using the model fitted in Exercise 1, predict total cholesterol at `vpa_r` = 1 hour/day and at `vpa_r` = 24 hours/day. Write one sentence stating which of the two predictions can be trusted, and one sentence explaining why the other cannot, referring to the range of `vpa_r` actually observed in the data.

Exercise 4 (ILOs 2, 3). State the null and alternative hypotheses, then reproduce Chapter 5's worked example, sedentary time (`sed`) by gender, with `lm()` instead of `t.test()`. Compare the confidence interval `lm()` gives you to the one Chapter 5 reported. Write one sentence on what you find, and a second sentence explaining it, using `var.equal = TRUE` to check your explanation.

Exercise 5 (ILOs 2, 6). State the null and alternative hypotheses, then fit `chol ~ smoking`. Use `levels()` to check which category of `smoking` the model used as its reference, then refit with `relevel()` using `'Current'` as the reference instead. Report the first model's coefficients in one sentence, then write a second sentence stating what changed between the two model fits and a third stating what stayed exactly the same, and why.

6.12 Writing exercise

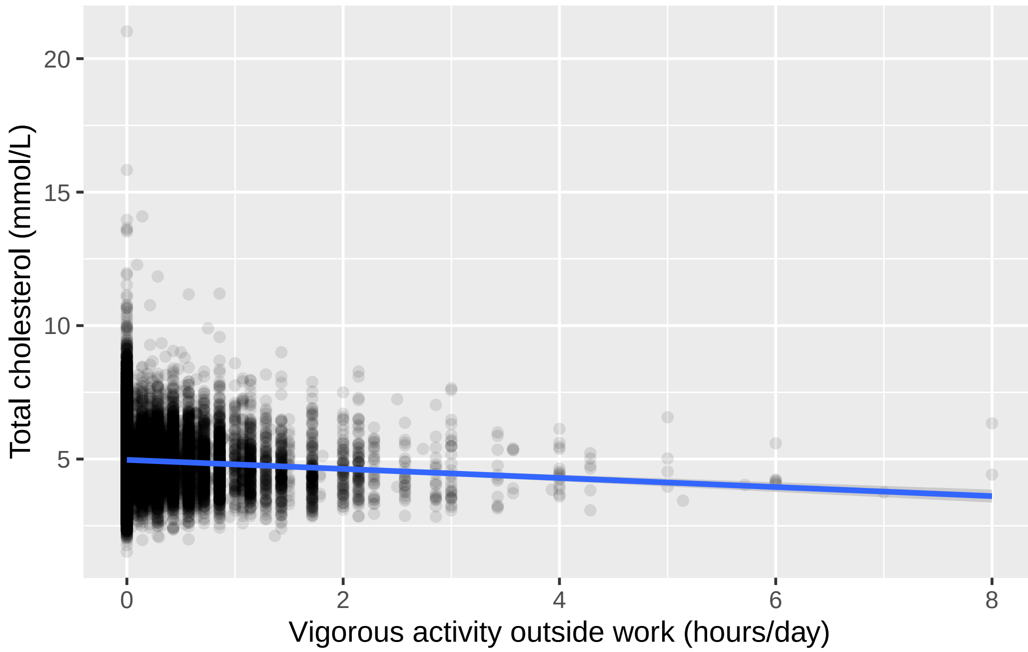
Using the `chol ~ sed` result from the worked example, write a short results paragraph of two to three sentences, as it would appear in a report. State the estimated slope, its 95% confidence interval, and its p-value, and say plainly what this evidence does and does not support. Avoid two common mistakes: don't describe a non-significant result as proof there is no relationship at all, and don't write around the p-value without stating it.

Solutions

Exercise 1
Null hypothesis. In the population, the slope of cholesterol on `vpa_r` is zero.
Alternative hypothesis. The slope is not zero.

```
vpa_r_complete <- nhanes |> filter(!is.na(vpa_r), !is.na(chol))

ggplot(vpa_r_complete, aes(x = vpa_r, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = 'lm') +
  labs(x = 'Vigorous activity outside work (hours/day)', y = 'Total cholesterol (mmol/L)')
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
m_vpa_r <- lm(chol ~ vpa_r, data = vpa_r_complete)
summary(m_vpa_r)
```

```

Call:
lm(formula = chol ~ vpa_r, data = vpa_r_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4408 -0.7508 -0.0808  0.6392 16.0492 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  4.970791   0.006375  779.74   <2e-16 ***
vpa_r       -0.169796   0.015756  -10.78   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.08 on 32329 degrees of freedom
Multiple R-squared:  0.003579,	Adjusted R-squared:  0.003549 
F-statistic: 116.1 on 1 and 32329 DF,  p-value: < 2.2e-16
```

```r
confint(m_vpa_r)
```

```
                 2.5 %     97.5 %
(Intercept)  4.9582959  4.9832860
vpa_r       -0.2006789 -0.1389139
```

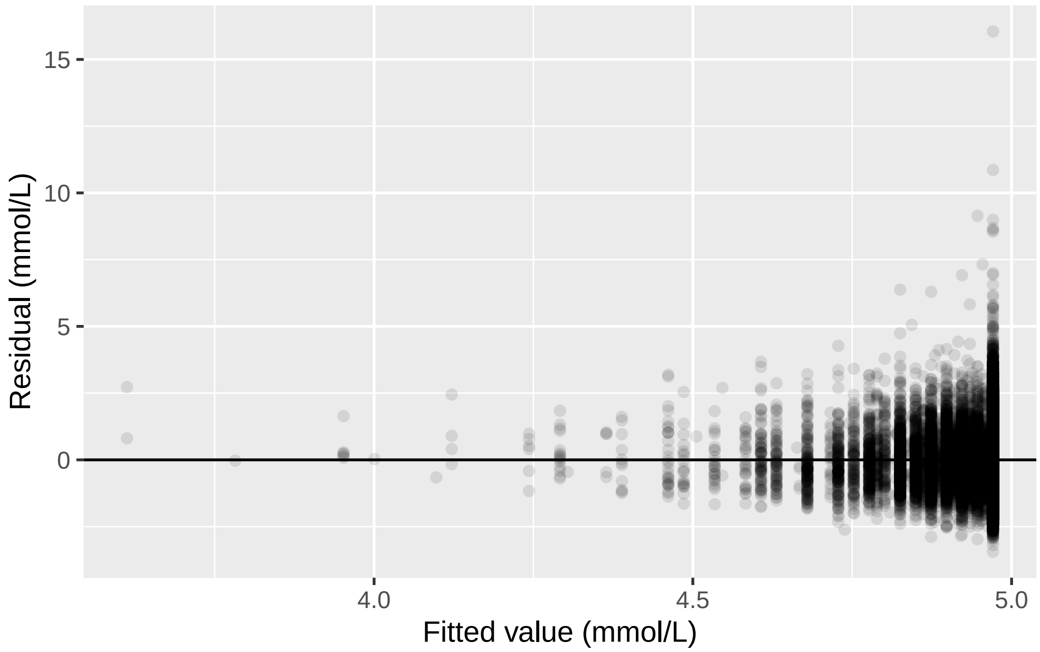
```r
tidy(m_vpa_r, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)    4.97    0.00637     780.  0           4.96      4.98
2 vpa_r         -0.170   0.0158      -10.8 4.94e-27   -0.201    -0.139
```

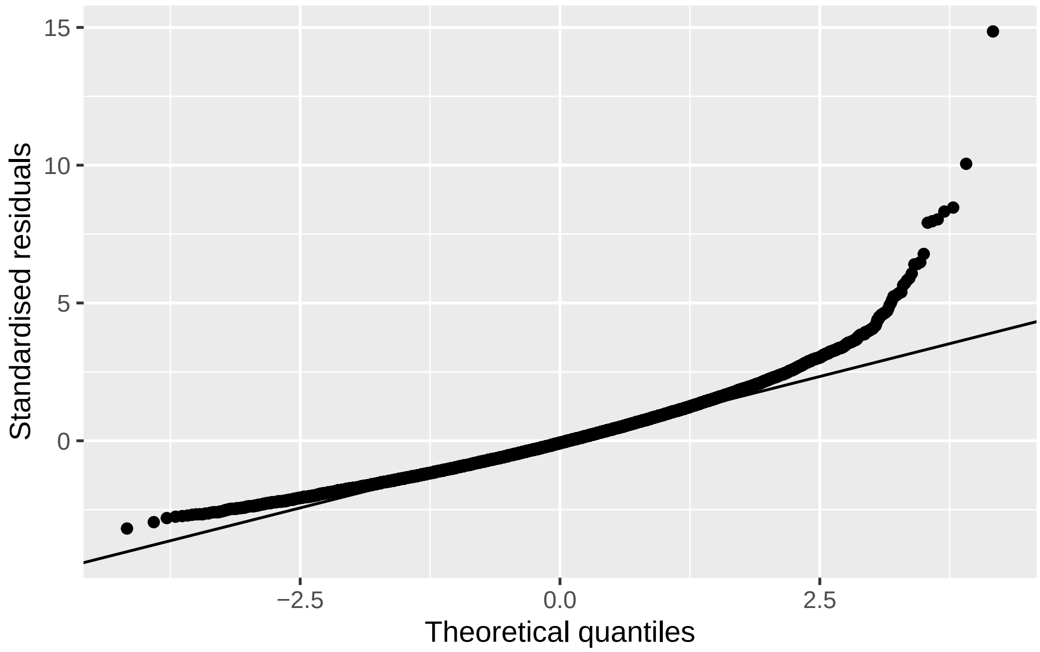
Total cholesterol was about 0.170 mmol/L lower for each additional hour/day of vigorous activity outside work (95% CI: -0.201 to -0.139 mmol/L, $p < 0.001$).

Exercise 2

```r
vpa_r_aug <- augment(m_vpa_r)

ggplot(vpa_r_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = 'Fitted value (mmol/L)', y = 'Residual (mmol/L)')
```




```
ggplot(vpa_r_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = 'Theoretical quantiles', y = 'Standardised residuals')
```



The residuals-vs-fitted plot checks linearity and constant variance; the QQ plot checks whether the residuals are approximately normally distributed. Most points in the residual plot fall into a small number of vertical strips rather than scattering freely. That's because `vpa_r` itself only takes a small number of distinct values: most participants report exactly zero hours/day of vigorous activity outside work. Their fitted value and residual pattern is shared, not spread out. This is a feature of the predictor's own distribution, not a sign that the model is wrongly specified.

Exercise 3

```
predict(m_vpa_r, newdata = data.frame(vpa_r = c(1, 24)), interval = 'confidence')
```

```
        fit       lwr      upr
1 4.8009945 4.7718047 4.830184
2 0.8956766 0.1585768 1.632776
```

The prediction at `vpa_r` = 1 hour/day (about 4.80 mmol/L) can be trusted, because an hour of vigorous activity outside work a day is well within the range observed in this sample (0 to 8 hours/day). The prediction at `vpa_r` = 24 hours/day (about 0.90 mmol/L) cannot: 24 hours/day of vigorous activity is not physically possible to sustain, it is far outside the observed range, and the resulting prediction, a cholesterol level barely above zero, is not a value any living person could have. It is a consequence of extending the fitted line somewhere the data give it no support, not a real finding.

Exercise 4

Null hypothesis. In the population, mean sedentary time is the same in men and women.
Alternative hypothesis. Mean sedentary time differs between men and women.

```
sed_gender_complete <- nhanes |> filter(!is.na(gender), !is.na(sed))

m_sed_gender <- lm(sed ~ gender, data = sed_gender_complete)
summary(m_sed_gender)
```

```
Call:
lm(formula = sed ~ gender, data = sed_gender_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
 -6.690  -3.431  -1.431   1.569 160.219 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)   6.43059    0.08441  76.187   <2e-16 ***
genderFemale  0.25930    0.11755   2.206   0.0274 *  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 10.9 on 34432 degrees of freedom
Multiple R-squared:  0.0001413, Adjusted R-squared:  0.0001123 
F-statistic: 4.866 on 1 and 34432 DF,  p-value: 0.0274
```

```
confint(m_sed_gender)
```

```
                  2.5 %    97.5 %
(Intercept)  6.26515458 6.5960288
genderFemale 0.02889292 0.4897018
```

`lm()` gives a slope of 0.259 hours/day (95% CI: 0.029 to 0.490 hours/day, p = 0.027), close to Chapter 5's t-test result but not identical: Chapter 5 reported a mean difference of about 0.26 hours/day (95% CI: 0.03 to 0.49 hours/day, p = 0.026). The gap is small, but it's real, in the third decimal of the p-value. It happens because `t.test()`'s default doesn't assume the two groups share one variance and `lm()` does, and here that assumption isn't quite free: `sed`'s spread genuinely differs by gender, a standard deviation of about 9.75 hours/day in men against about 11.9 in women. Setting `var.equal = TRUE` on the t-test confirms it:

```
t.test(sed ~ gender, data = sed_gender_complete, var.equal = TRUE)
```

```

	Two Sample t-test

data:  sed by gender
t = -2.2058, df = 34432, p-value = 0.0274
alternative hypothesis: true difference in means between group Male and group Female is not
95 percent confidence interval:
 -0.48970176 -0.02889292
sample estimates:
  mean in group Male mean in group Female
            6.430592             6.689889
```

With `var.equal = TRUE`, the t-test's p-value becomes 0.027 and its interval 0.029 to 0.490 once the sign is reversed, matching `lm()` exactly. The `chol` model earlier in this chapter agreed with its t-test to four decimal places because the two groups' spreads there were nearly equal; here they aren't, so the equal-variance assumption `lm()` makes actually changes the answer, by a small amount.

Exercise 5

Null hypothesis. In the population, a non-reference smoking category's mean cholesterol equals the reference category's.

Alternative hypothesis. It differs. Each non-reference row tests this comparison separately.

```
smoking_chol_complete <- nhanes |> filter(!is.na(smoking), !is.na(chol))

m_smoking <- lm(chol ~ smoking, data = smoking_chol_complete)
summary(m_smoking)
```

```

Call:
lm(formula = chol ~ smoking, data = smoking_chol_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4658 -0.7496 -0.0858  0.6442 16.0304 

Coefficients:
                Estimate Std. Error t value Pr(>|t|)    
(Intercept)     4.967995   0.007375 673.672   <2e-16 ***
```

```
smokingPrevious 0.021601   0.013671   1.580   0.1141
smokingCurrent  0.027794   0.014238   1.952   0.0509 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.091 on 38874 degrees of freedom
Multiple R-squared:  0.0001279, Adjusted R-squared:  7.644e-05
F-statistic: 2.486 on 2 and 38874 DF,  p-value: 0.08327
```

```
confint(m_smoking)
```

```
                        2.5 %     97.5 %
(Intercept)      4.9535405752 4.98244900
smokingPrevious -0.0051953902 0.04839717
smokingCurrent  -0.0001129389 0.05570047
```

```
smoking_relevelled <- smoking_chol_complete |>
  mutate(smoking = relevel(smoking, ref = 'Current'))
m_smoking_relevel <- lm(chol ~ smoking, data = smoking_relevelled)
summary(m_smoking_relevel)
```

```
Call:
lm(formula = chol ~ smoking, data = smoking_relevelled)

Residuals:
    Min      1Q  Median      3Q     Max
-3.4658 -0.7496 -0.0858  0.6442 16.0304

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      4.995789   0.012179 410.187   <2e-16 ***
smokingNever    -0.027794   0.014238  -1.952   0.0509 .
smokingPrevious -0.006193   0.016759  -0.370   0.7117
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.091 on 38874 degrees of freedom
Multiple R-squared:  0.0001279, Adjusted R-squared:  7.644e-05
F-statistic: 2.486 on 2 and 38874 DF,  p-value: 0.08327
```

```
confint(m_smoking_relevel)
```

```
                    2.5 %       97.5 %
(Intercept)      4.97191682 5.0196602927
smokingNever    -0.05570047 0.0001129389
smokingPrevious -0.03904065 0.0266548955
```

`levels()` on the original factor shows `'Never'` first, so that's the reference the first model used: mean cholesterol is about 4.97 mmol/L in never-smokers, 0.022 mmol/L higher in previous smokers (95% CI -0.005 to 0.048, $p = 0.114$), and 0.028 mmol/L higher in current smokers (95% CI 0.000 to 0.056, $p = 0.051$), neither significant at the conventional 5% level. Releveled to `'Current'`, both rows change: the intercept becomes current smokers' own mean, and the two remaining rows compare `'Never'` and `'Previous'` against `'Current'` instead. `Multiple R-squared` and `Residual standard error` are identical in both printouts, 0.0001279 and 1.091. Which category anchors the comparison changed; what the model actually fits, and predicts for any given participant, didn't.

Writing exercise (sample answer)

Sedentary time was not significantly associated with total cholesterol in this sample (slope -0.0010 mmol/L per hour/day, 95% CI: -0.0021 to 0.0001 mmol/L, $p = 0.073$). The confidence interval includes zero, so these data are consistent with no true association. This model adjusts for nothing else, so it cannot say whether age, gender, or other characteristics are masking or accounting for this result.

7 Multiple linear and logistic regression (Week 11)

A confounder is a variable that lies behind an association and distorts it, and Chapter 6's single-predictor model has no way to hold one still. Baseline cardiovascular disease in our data is a real example of the problem, not just a definition of it: it's measured at the same visit as the exposure, so nothing in the data says whether it sits upstream of sedentary behaviour or downstream of it. Whether it belongs in the model as a confounder to adjust for, or should stay out because it's actually a mediator, is a judgement the study design can't make for us, and this chapter fits the model both ways and reports both.

Getting there means extending linear regression to hold several predictors at once, so an adjusted coefficient can sit next to the crude one and the gap between them read as an estimate of confounding bias. The same extension handles an outcome that isn't continuous at all: whether someone died within ten years of their baseline visit becomes the input to a logistic regression, with odds ratios read straight off its exponentiated coefficients.

7.1 Chapter intended learning outcomes

By the end of this chapter you will be able to:

1. Fit a multiple linear regression using `lm()`, including categorical predictors with a stated reference level
2. Compare crude and adjusted estimates and quantify the change
3. Fit a logistic regression with `glm(family = binomial)`, and interpret an exponentiated coefficient as an adjusted odds ratio with a confidence interval
4. Compare nested models as a sensitivity analysis, and appraise what each assumes about `blm.cvd`
5. Write an interpretation of an adjusted association in an appropriate academic style

7.2 Setup

As in Chapters 3 to 6, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary.

This chapter adds one new package underneath the scenes: `gtsummary`'s `tbl_regression()`, which we use for the first time here, depends on `broom.helpers`. If `broom.helpers` isn't already on your machine, install it the usual way, in the console, not in the script. We don't `library()` it directly; `gtsummary` finds it automatically once it's installed.

```
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)
library(broom)

nhanes <- read_csv(here('data', 'mph_nhanes_20241107.csv'))
dictionary <- read_csv(here('data', 'mph_nhanes_20241107_dictionary.csv'))
```

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c('Male', 'Female')),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c('Mexican American', 'Other Hispanic', 'White', 'Black', 'Others')),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c('Less than high school', 'High school graduate', 'Above high school')),
    smoking = factor(smoking,
      levels = c('Never', 'Previous', 'Current')),
    phq.c = factor(phq.c,
      levels = c('0 Normal', '1 Mild symptoms', '2 Possible depression'),
      labels = c('Normal', 'Mild symptoms', 'Possible depression'),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c('No', 'Yes'))
  )
```

This is the same recode we introduced in Chapter 3 and repeated in Chapters 4 to 6. See Chapter 3 for the reasoning behind each variable.

7.3 Opening exercise: building a ten-year mortality outcome

nhanes_mph records whether each participant died during follow-up (dth: 1=died; 0=alive) and how long they were followed for (fu_month, in months). Neither column on its own answers the question this chapter needs: did a participant die *within ten years*? 'Died' isn't a question we can answer until we say 'by when.'

Three groups exist in this data:

- Participants who died, and whose death happened within 120 months (10 years) of their follow-up starting. These count as an event: 1.
- Participants known to be alive at 120 months. This includes everyone followed at least that long without dying, but it also includes anyone who died *after* 120 months: surviving to month 150 and then dying still means they were alive at month 120, so this group counts as a non-event: 0, regardless of whether death eventually happened later on.
- Participants who did not die, but whose recorded follow-up ended before 120 months. These cannot be classified either way. They might have died the month after their follow-up ended, or lived another thirty years. Calling them 'survivors' would be a guess, so they are excluded rather than coded as 0.

Two new tools here: `case_when()`, from `dplyr`, checks a series of conditions in order and returns the first matching value, which is exactly what a three-way rule needs. Each line pairs a condition on the left of the `~` with the value to return on the right. The last line is usually a catch-all: `TRUE` is a condition that is always met, so anything reaching it without matching a line above gets whatever that line returns. Order matters, because `case_when()` stops at the first condition a participant matches. The second tool is an argument rather than a function. `table()` normally leaves missing values out of its count, the way it did in Chapter 3. `useNA = 'always'` tells it to report them as their own category instead. That's what we want here, since the participants we exclude are the point, not a footnote.

Warm-up exercise

Exercise. Write the `mutate()` yourself. Using `case_when()`, build a new column called `dead_10yr` in `nhanes` that returns `1` for the first group above, `0` for the second, and `NA` for the third, then count the three groups with `table(nhanes$dead_10yr, useNA = 'always')`.
Two things to work out as you write it, which matter more than the syntax:

1. The rule for the second group needs no mention of `dth` at all. Why not? Think about who is in that group.
2. Once you have the counts, be ready to say why the third group can't simply be added to the `0`s and called survivors.

Have a go before opening the answer. Everything after this section runs on the column you're about to build, so build it yourself instead of reading someone else's version of it.

Answer: building `dead_10yr`

```r
nhanes <- nhanes |>
  mutate(
    dead_10yr = case_when(
      dth == 1 & fu_month <= 120 ~ 1,
      fu_month >= 120 ~ 0,
      TRUE ~ NA
    )
  )

table(nhanes$dead_10yr, useNA = 'always')
```

```

    0     1  <NA>
14275  3748 23742
```

The second condition is written as `fu_month >= 120` alone, with no condition on `dth`, precisely to catch both parts of that second group: `dth == 1` (died, but after month 120) and `dth == 0` (never died, followed at least that long) both belong in it. Anyone who was still being observed at month 120 was alive at month 120, and that is the whole question. The catch-all comes last and returns `NA`. That's the third group: alive, but followed for less than ten years. Had it been written first, it would have swallowed everybody.

3,748 participants died within the ten-year window, 14,275 are known to have been alive at ten years (whether or not they later died), and 23,742 are excluded because their follow-up ended too early to classify them either way. That's a large number excluded, but it's a cost of a fixed-window outcome. We're deliberately not using information about *when* someone died, only whether it was inside or outside our chosen window. The 3,748 events that remain are plenty to fit the regression models this chapter builds.

There is a family of methods, survival analysis, built precisely to use the follow-up time we've just thrown away and to keep the participants we've just excluded. Kaplan-Meier curves and Cox regression are its familiar tools. They're taught in Advanced Statistics.

> **Try it yourself: a different window**
>
> Rebuild `dead_10yr` using a five-year window (60 months) instead of ten. Report the new event and exclusion counts, then explain in one sentence why a shorter window moves both numbers in the direction it does.

7.4 Extending the Chapter 6 model

This section adjusts for known confounders: age, gender, and education. Chapter 6 fitted total cholesterol (`chol`, in mmol/L) against sedentary time alone (`sed`, in hours/day), `chol ~ sed`, and found a small, non-significant slope: it adjusted for nothing else, so it couldn't rule out age, gender, and education masking the association. We now add them using `+` to an `lm()` formula.

Null hypothesis. In the population, the adjusted slope of cholesterol on `sed` is zero, once age, gender, and education are held constant.

Alternative hypothesis. The adjusted slope is not zero.

```r
chol_adj_complete <- nhanes |>
  filter(!is.na(sed), !is.na(chol), !is.na(age), !is.na(gender), !is.na(edu))

m_adj <- lm(chol ~ sed + age + gender + edu, data = chol_adj_complete)
summary(m_adj)
```

```
Call:
lm(formula = chol ~ sed + age + gender + edu, data = chol_adj_complete)

Residuals:
    Min      1Q  Median      3Q     Max
-3.4095 -0.7426 -0.0814  0.6424 15.9864
```

```
Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)              4.6614324  0.0230949 201.838   <2e-16 ***
sed                     -0.0011304  0.0005678  -1.991   0.0465 *
age                      0.0047488  0.0003467  13.696   <2e-16 ***
genderFemale             0.1605092  0.0121807  13.177   <2e-16 ***
eduHigh school graduate -0.0130321  0.0177073  -0.736   0.4618
eduAbove high school     0.0045031  0.0149561   0.301   0.7633
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.075 on 31222 degrees of freedom
Multiple R-squared:  0.01147,	Adjusted R-squared:  0.01131
F-statistic: 72.43 on 5 and 31222 DF,  p-value: < 2.2e-16
```

```r
confint(m_adj)
```

```
                               2.5 %        97.5 %
(Intercept)              4.616165428  4.706699e+00
sed                     -0.002243383 -1.741808e-05
age                      0.004069176  5.428422e-03
genderFemale             0.136634488  1.843839e-01
eduHigh school graduate -0.047739058  2.167485e-02
eduAbove high school    -0.024811500  3.381773e-02
```

The coefficient on `sed` is now what this model calls the *adjusted* association between sedentary time and cholesterol: the association after holding age, gender, and education constant. 'Adjustment' here means the model assumes the effect of an extra hour of sedentary time is the same regardless of a participant's age, gender, or education, and estimates that one shared slope from everybody at once.

`gender` and `edu` entered that model as factors, and each one contributes a row per non-reference level rather than one row for the whole variable. `levels()`, from base R, shows a factor's categories in order; the first one listed is the reference level, the category every other row is compared against, the same idea Chapter 6 covered in full, including what `relevel()` does and doesn't change. We don't interpret `gender`'s or `edu`'s own coefficients here the way we interpret `sed`'s: neither was the exposure this model was built to assess, so their adjustment sets weren't chosen with that in mind. More on exactly why, below.

7.5 Crude versus adjusted: quantifying the change

Chapter 6's crude result and this chapter's adjusted one belong side by side. We refit the crude model here instead of reusing Chapter 6's, so both models run on `chol_adj_complete`, the same set of participants. Without aligning the samples, any difference between the two estimates could be attributed to either the adjustment or a difference in sample size, and we couldn't tell which. This is also why the crude figure below isn't identical to the one Chapter 6 reported.

```
m_crude <- lm(chol ~ sed, data = chol_adj_complete)
tidy(m_crude, conf.int = TRUE) |> filter(term == 'sed')
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>   <dbl>    <dbl>     <dbl>
1 sed   -0.000738  0.000570     -1.29   0.195 -0.00186  0.000379
```

```
tidy(m_adj, conf.int = TRUE) |> filter(term == 'sed')
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic p.value conf.low  conf.high
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>      <dbl>
1 sed   -0.00113  0.000568     -1.99  0.0465 -0.00224 -0.0000174
```

Crude, the slope on `sed` is about -0.00074 mmol/L per hour/day (95% CI -0.00186 to 0.00038, $p = 0.195$). Adjusted for age, gender, and education, it moves to about -0.00113 mmol/L per hour/day (95% CI -0.00224 to -0.00002, $p = 0.047$).

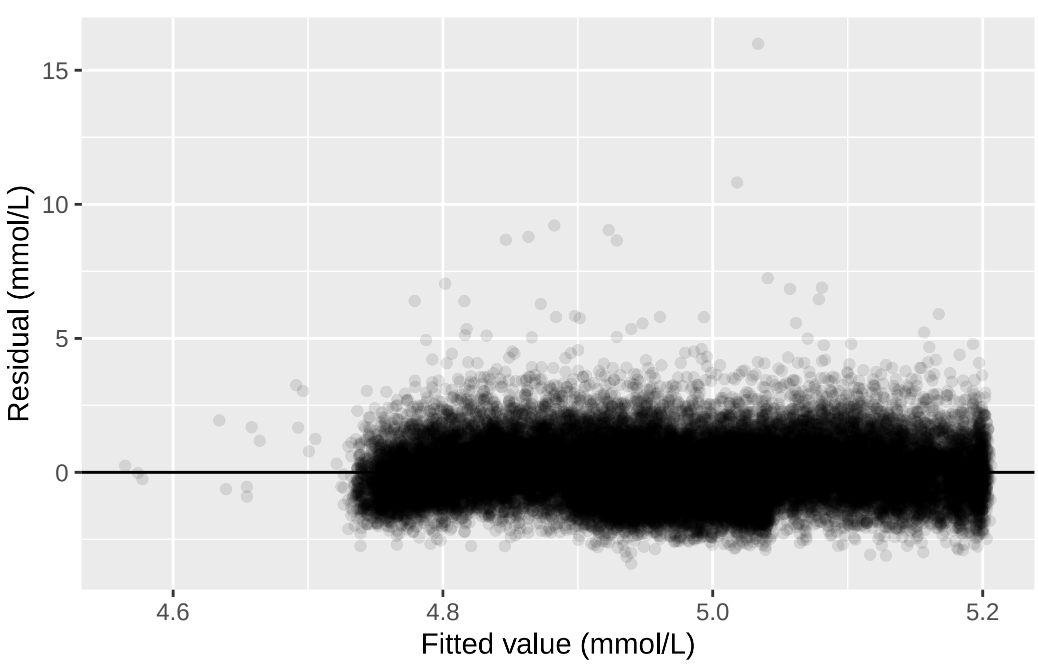
Look at the estimate first, not the p-value. The adjusted slope is about 1.5 times the size of the crude one: holding age, gender, and education constant pulls the association between sedentary time and cholesterol substantially further from zero. That movement is what confounding looks like. Age and gender were masking part of a real association, and the model that ignores them understates it. The p-value crossing 0.05 is a consequence of the estimate moving away from zero, not a separate finding.

A caution about reading the other rows of this table: `age`'s coefficient here is adjusted for `sed` too, the same as `sed`'s own coefficient is. If sedentary time sits on the path from age to cholesterol, part of age's true association runs through it, and `age`'s row only captures the part that doesn't. Treating every coefficient in a regression table as though it carried the same interpretation as the one we set out to estimate is called the *Table 2 fallacy*. `sed` is the exposure this model was built to assess, and its adjustment set was chosen with that in mind; `age`, `gender`, and `edu` weren't, so their coefficients shouldn't be read as if they were each other's crude-versus-adjusted comparisons in miniature. This is the same logic the sensitivity analysis later in this chapter applies to `blm.cvd`, just stated as a general caution rather than worked through on one variable.

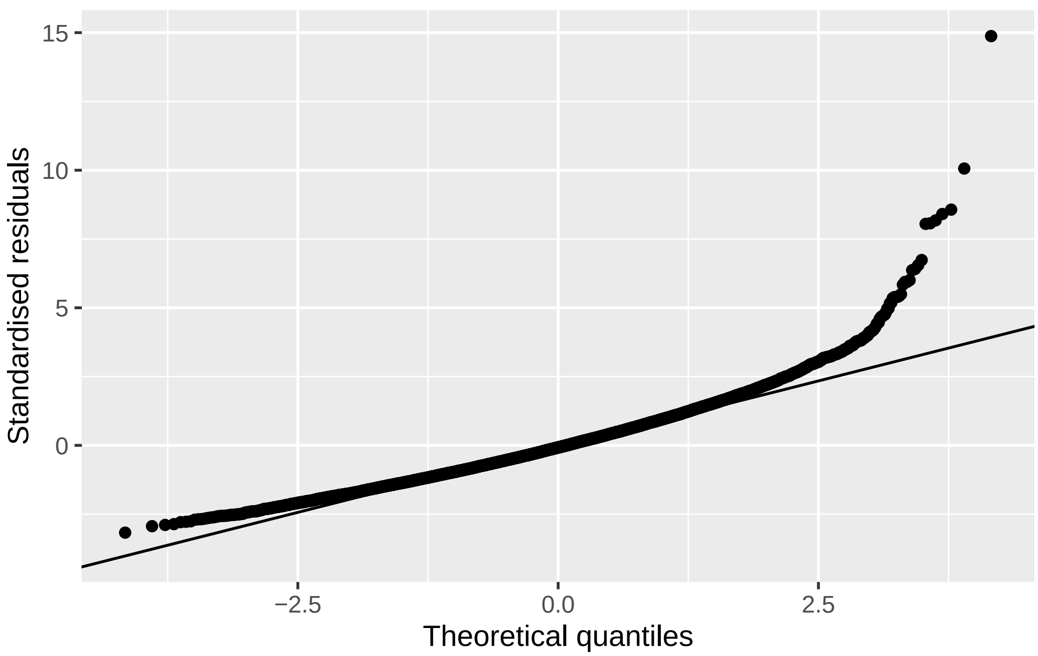
7.6 Diagnostics for the multiple regression model

The same diagnostic plots Chapter 6 used still apply here, checked against the adjusted model rather than the simple one:

```
adj_aug <- augment(m_adj)

ggplot(adj_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = 'Fitted value (mmol/L)', y = 'Residual (mmol/L)')
```




```
ggplot(adj_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = 'Theoretical quantiles', y = 'Standardised residuals')
```



The same pattern Chapter 6 found reappears: a fairly even scatter around zero for most of the range, with a small group of participants with unusually high cholesterol pulling the upper tail away from the reference line in the QQ plot.

7.7 When the outcome is binary: logistic regression

The question for the rest of this chapter: is sedentary time associated with ten-year mortality, and does that association hold up once potential confounders are adjusted for? `chol` is continuous, so `lm()` was the right tool. But `dead_10yr` is 0 or 1, and we can't model it the same way. An `lm()` produces a straight line, and a straight line fitted to a 0/1 outcome can predict values below 0 or above 1, which aren't valid probabilities. `glm()`, base R's function for generalised linear models (a family of regression models taught further in Advanced Statistics), fixes this by modelling the log odds of the outcome instead of the outcome directly.

We build the complete-case sample first and fit both models on it, for the same reason the linear models above shared one sample: the crude and adjusted estimates should differ because of the adjustment, not because they were calculated on different people.

```
dead_complete <- nhanes |>
  filter(!is.na(dead_10yr), !is.na(sed), !is.na(age), !is.na(gender), !is.na(edu))
```

Fitting one looks almost identical to fitting an `lm()`. The formula is the same shape, and the only addition is `family = binomial`, which tells `glm()` the outcome is binary.

Null hypothesis. In the population, sedentary time carries no association with ten-year mortality: on the odds ratio scale, met below, an odds ratio of 1.

Alternative hypothesis. Sedentary time is associated with ten-year mortality, an odds ratio away from 1.

```
m_dead_crude <- glm(dead_10yr ~ sed, data = dead_complete, family = binomial)
summary(m_dead_crude)
```

```

Call:
glm(formula = dead_10yr ~ sed, family = binomial, data = dead_complete)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.162367   0.039802 -29.204  < 2e-16 ***
sed          0.039638   0.005826   6.803 1.02e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12747  on 10695  degrees of freedom
Residual deviance: 12632  on 10694  degrees of freedom
AIC: 12636

Number of Fisher Scoring iterations: 5
```

```
m_dead_adj <- glm(dead_10yr ~ sed + age + gender + edu,
  data = dead_complete, family = binomial)
summary(m_dead_adj)
```

```

Call:
glm(formula = dead_10yr ~ sed + age + gender + edu, family = binomial,
    data = dead_complete)

Coefficients:
                         Estimate Std. Error z value Pr(>|z|)
```

```
(Intercept)             -6.143091   0.138353 -44.402  < 2e-16 ***
sed                      0.019971   0.003697   5.402 6.59e-08 ***
age                      0.092216   0.001993  46.273  < 2e-16 ***
genderFemale            -0.514661   0.052973  -9.716  < 2e-16 ***
eduHigh school graduate -0.089589   0.069576  -1.288    0.198
eduAbove high school    -0.308002   0.061239  -5.030 4.92e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12746.5  on 10695  degrees of freedom
Residual deviance:  8905.5  on 10690  degrees of freedom
AIC: 8917.5

Number of Fisher Scoring iterations: 5
```

We keep this adjustment set deliberately small. `dead_complete` holds 10,696 participants, of whom 3,028 died within ten years. That's fewer events than the 3,748 we counted at the start of the chapter, because requiring a recorded value for every predictor drops participants of its own. Those 3,028 events are what the model actually has to work with, and there isn't room to add many more predictors before it runs short of events per predictor.

7.8 Odds ratios: exponentiating the coefficients

A logistic regression's coefficients are on the log-odds scale, which isn't how the epidemiology half reported odds ratios. `exp()`, base R's exponential function, undoes the log and converts a coefficient back to an odds ratio. `coef()` and `confint()` pull the coefficients and their confidence intervals the same way they did for `lm()`:

```r
exp(coef(m_dead_crude))
```

```
(Intercept)         sed
   0.312745    1.040434
```

```r
exp(confint(m_dead_crude))
```

```
Waiting for profiling to be done...
```

```
                 2.5 %    97.5 %
(Intercept) 0.2888699 0.3373731
sed         1.0291891 1.0526653
```

`confint()` printed `Waiting for profiling to be done...` before the answer. That's a message, not a warning, and it marks a real difference between `glm()` and `lm()`.

For a linear model, `confint()` builds each interval the way Chapter 4 built one by hand: an estimate, plus or minus a multiplier times a standard error. For a logistic model that shortcut is only an approximation, so `confint()` does something better instead. It works along each coefficient, refitting the model repeatedly to find the range of values the data are consistent with. That search is the 'profiling', and the message is `glm()` saying it's running. On this dataset it takes a second or two. On a much larger model it can take noticeably longer, which is the only reason the message exists at all.

```r
exp(coef(m_dead_adj))
```

```
             (Intercept)                      sed                      age
             0.002148272              1.020171639              1.096602080
            genderFemale eduHigh school graduate    eduAbove high school
             0.597703435              0.914306766              0.734914077
```

```r
exp(confint(m_dead_adj))
```

```
Waiting for profiling to be done...
```

```
                              2.5 %      97.5 %
(Intercept)             0.001633024 0.002809059
sed                     1.013277999 1.028199075
age                     1.092371089 1.100939444
genderFemale            0.538655006 0.662981437
eduHigh school graduate 0.797687946 1.047837119
eduAbove high school    0.651773505 0.828632927
```

The same message appears again above, for the same reason.

`broom::tidy()` does the same conversion in one step with `exponentiate = TRUE`:

```r
tidy(m_dead_adj, exponentiate = TRUE, conf.int = TRUE) |> filter(term == 'sed')
```

Table 7.1

Characteristic	Beta	95% CI	p-value
sed	0.00	0.00, 0.00	0.047
age	0.00	0.00, 0.01	<0.001
gender			
Male	—	—	
Female	0.16	0.14, 0.18	<0.001
edu			
Less than high school	—	—	
High school graduate	-0.01	-0.05, 0.02	0.5
Above high school	0.00	-0.02, 0.03	0.8

Abbreviation: CI = Confidence Interval

```
# A tibble: 1 x 7
  term  estimate std.error statistic      p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>        <dbl>    <dbl>     <dbl>
1 sed       1.02   0.00370      5.40 0.0000000659     1.01      1.03
```

Crude, each additional hour/day of sedentary time is associated with an odds ratio of about 1.040 for ten-year mortality (95% CI: 1.029 to 1.053, $p < 0.001$). Adjusted for age, gender, and education, the odds ratio attenuates to about 1.020 (95% CI: 1.013 to 1.028, $p < 0.001$). Both intervals sit entirely above 1, the null value for an odds ratio, so both are statistically significant. But the adjusted estimate is noticeably smaller than the crude one. Part of the crude association was really age. Older participants are both more sedentary and more likely to die within ten years, for reasons that have nothing to do with sedentary behaviour itself.

Both odds ratios sit close to 1, so confounding matters less here than it will in Exercise 2, where the same mechanism, age, moves the estimate by a factor of about six instead of a couple of percent. Do that exercise before deciding how seriously to take adjustment.

7.9 Regression tables with `gtsummary`

`tbl_regression()`, from `gtsummary`, builds a publication-style table directly from a fitted model, the regression equivalent of `tbl_summary()`. For the linear model, no extra argument is needed:

```r
tbl_regression(m_adj)
```

For the logistic model, `exponentiate = TRUE` reports odds ratios instead of log-odds coefficients:

Table 7.2

Characteristic	**OR**	**95% CI**	**p-value**
sed	1.02	1.01, 1.03	<0.001
age	1.10	1.09, 1.10	<0.001
gender			
Male	—	—	
Female	0.60	0.54, 0.66	<0.001
edu			
Less than high school	—	—	
High school graduate	0.91	0.80, 1.05	0.2
Above high school	0.73	0.65, 0.83	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

Table 7.3

	Crude			Adjusted		
Characteristic	**OR**	**95% CI**	**p-value**	**OR**	**95% CI**	**p-value**
sed	1.04	1.03, 1.05	<0.001	1.02	1.01, 1.03	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```r
tbl_regression(m_dead_adj, exponentiate = TRUE)
```

`tbl_merge()`, also from `gtsummary`, places two tables side by side under named headers, which is exactly what a crude-vs-adjusted comparison needs:

```r
tbl_crude <- tbl_regression(m_dead_crude, exponentiate = TRUE, include = 1)
tbl_adj <- tbl_regression(m_dead_adj, exponentiate = TRUE, include = 1)

tbl_merge(list(tbl_crude, tbl_adj),
  tab_spanner = c('Crude', 'Adjusted'))
```

`include = 1` keeps only the `sed` row, the estimate we're interested in, and drops the adjustment variables' own rows. That's fine here: the Table 2 fallacy above is exactly why we weren't going to interpret those rows anyway. The table still needs a caption or footnote naming which variables were adjusted for, since that information doesn't disappear just because the rows do.

7.10 The sensitivity analysis: is `blm.cvd` a confounder or a mediator?

The seminar raised a genuine difficulty with `blm.cvd`, baseline cardiovascular disease. The two models below are **nested**: one model's predictors, `sed + age + gender + edu`, are a subset of the other's, which adds only `blm.cvd`. Comparing a nested pair like this isolates the effect of the one variable that differs between them. It's plausible as a confounder. Existing heart disease could make someone both less physically active and more likely to die, and adjusting for it would then remove bias. It's equally plausible as a mediator. Sedentary behaviour could contribute to cardiovascular disease, which then contributes to death, and adjusting for it would then remove part of the total effect we're trying to estimate. Baseline CVD and sedentary time were both measured at the same study visit, so nothing in the data itself establishes which came first. That's not a gap in this analysis. It's a limit on what an observational snapshot can ever tell us. The honest response is to report the estimate both ways instead of picking one.

```
dead_cvd_complete <- dead_complete |> filter(!is.na(blm.cvd))

m_dead_without <- glm(dead_10yr ~ sed + age + gender + edu,
  data = dead_cvd_complete, family = binomial)
m_dead_with <- glm(dead_10yr ~ sed + age + gender + edu + blm.cvd,
  data = dead_cvd_complete, family = binomial)

tbl_without <- tbl_regression(m_dead_without, exponentiate = TRUE, include = 1)
tbl_with <- tbl_regression(m_dead_with, exponentiate = TRUE, include = 1)

tbl_merge(list(tbl_without, tbl_with),
  tab_spanner = c('Without baseline CVD', 'With baseline CVD'))
```

	Without baseline CVD			With baseline CVD		
Characteristic	**OR**	**95% CI**	**p-value**	**OR**	**95% CI**	**p-value**
sed	1.02	1.01, 1.03	<0.001	1.02	1.01, 1.03	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

The odds ratio for `sed` moves only a little between the two models: 1.020 (95% CI 1.013 to 1.028, $p < 0.001$) without `blm.cvd`, and 1.019 (95% CI 1.012 to 1.026, $p < 0.001$) with it. `blm.cvd` itself is a strong predictor of ten-year mortality, but adjusting for it changes almost nothing about the sedentary time estimate. Whichever role `blm.cvd` plays, confounder or mediator, the conclusion about sedentary time and mortality in this dataset doesn't depend much on which one it is.

7.11 Exercises

Exercise 1 (ILOs 1, 2). State the null and alternative hypotheses for the adjusted slope. Fit `chol ~ met_t + age + gender + edu`, where `met_t` is total physical activity in MET-hours/day, and compare it with the crude model `chol ~ met_t`. Report both slopes for `met_t` with their 95% confidence intervals and p-values, and write one sentence describing how the adjusted estimate differs from the crude one, including its direction.

Exercise 2 (ILOs 3, 5). State the null and alternative hypotheses for the odds ratio, noting the scale its null value sits on. Fit `dead_10yr ~ vpa_r + age + gender + edu`, where `vpa_r` is vigorous physical activity outside work in hours/day, and compare it with the crude model `dead_10yr ~ vpa_r`. Exponentiate both models' coefficients to obtain odds ratios with confidence intervals, and build a `tbl_regression()` table for the adjusted model. Write one sentence interpreting the adjusted odds ratio for `vpa_r` in context.

Exercise 3 (ILO 4). Using the adjusted model from Exercise 2, fit a second version that also adjusts for `blm.cvd`. Combine both models into one table with `tbl_merge()`, and write one sentence comparing the two odds ratios for `vpa_r` and one sentence stating what each model assumes about `blm.cvd`.

7.12 Writing exercise

Write two short paragraphs, as they would appear in a results section.

In the first, report the adjusted association between sedentary time and cholesterol from this chapter's linear model, and the adjusted odds ratio for sedentary time and ten-year mortality from the logistic model, each with its 95% confidence interval and p-value. Add one sentence on what this analysis cannot establish: an observational adjustment reduces confounding by the variables actually included, but it cannot rule out confounding by variables that were never measured.

In the second, separate paragraph, report both odds ratios for `sed` from the sensitivity analysis, with and without `blm.cvd`, and state plainly what each model assumes about baseline CVD's role and why the data available here cannot decide between those assumptions.

> **i** Solutions
>
> **Exercise 1**
> **Null hypothesis.** In the population, the adjusted slope of cholesterol on `met_t` is zero, once age, gender, and education are held constant.
> **Alternative hypothesis.** The adjusted slope is not zero.

```
met_t_complete <- nhanes |>
  filter(!is.na(met_t), !is.na(chol), !is.na(age), !is.na(gender), !is.na(edu))

m_met_t_crude <- lm(chol ~ met_t, data = met_t_complete)
m_met_t_adj <- lm(chol ~ met_t + age + gender + edu, data = met_t_complete)

tidy(m_met_t_crude, conf.int = TRUE) |> filter(term == 'met_t')
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>   <dbl>    <dbl>     <dbl>
1 met_t -0.000772  0.000426     -1.81  0.0700 -0.00161 0.0000632
```

```
tidy(m_met_t_adj, conf.int = TRUE) |> filter(term == 'met_t')
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic  p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 met_t  0.00166  0.000443      3.74 0.000182 0.000790   0.00253
```

Crude, the slope on `met_t` is about -0.00077 mmol/L per MET-hour/day (95% CI: -0.00161 to 0.00006, p = 0.07). The interval includes zero, so on its own this says very little: these data are consistent with no association between total activity and cholesterol, and they don't rule one out either.
Adjusted for age, gender, and education, the slope changes sign and becomes clearly non-zero, at about 0.00166 mmol/L per MET-hour/day (95% CI: 0.00079 to 0.00253, $p < 0.001$). Once those three are held constant, more total activity is associated with slightly *higher* cholesterol in this sample. That's a reminder that adjustment can change a result's direction, not just its size, and that a crude estimate near zero can hide an association rather than rule one out. Both models are fitted on `met_t_complete`, the same participants, so the difference between them is the adjustment and nothing else.

```
levels(met_t_complete$edu)
```

```
[1] "Less than high school" "High school graduate"  "Above high school"
```

`'Less than high school'` is listed first, so that's the reference this model used: both `edu` rows in the adjusted output above are differences from that group. We don't read them further than that here, for the same reason the chapter gave: `edu` wasn't the exposure this model was built to estimate.
Exercise 2

Null hypothesis. In the population, the adjusted odds ratio for `vpa_r` is 1: no association between vigorous activity outside work and ten-year mortality, once age, gender, and education are held constant.
Alternative hypothesis. The adjusted odds ratio is not 1. The null sits at 1, not 0, since this is a ratio.

```r
vpa_r_dead_complete <- nhanes |>
  filter(!is.na(vpa_r), !is.na(dead_10yr), !is.na(age), !is.na(gender), !is.na(edu))

m_vpa_r_crude <- glm(dead_10yr ~ vpa_r, data = vpa_r_dead_complete, family = binomial)
m_vpa_r_adj <- glm(dead_10yr ~ vpa_r + age + gender + edu,
  data = vpa_r_dead_complete, family = binomial)

tidy(m_vpa_r_crude, exponentiate = TRUE, conf.int = TRUE) |> filter(term == 'vpa_r')
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic  p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 vpa_r   0.0650     0.196     -14.0 2.49e-44   0.0437    0.0941
```

```r
tidy(m_vpa_r_adj, exponentiate = TRUE, conf.int = TRUE) |> filter(term == 'vpa_r')
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic      p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>        <dbl>    <dbl>     <dbl>
1 vpa_r    0.376     0.172     -5.68 0.0000000137    0.264     0.520
```

```r
tbl_regression(m_vpa_r_adj, exponentiate = TRUE)
```

Characteristic	**OR**	**95% CI**	**p-value**
vpa_r	0.38	0.26, 0.52	<0.001
age	1.10	1.09, 1.10	<0.001
gender			
Male	—	—	
Female	0.58	0.52, 0.65	<0.001
edu			
Less than high school	—	—	
High school graduate	0.94	0.82, 1.07	0.4
Above high school	0.79	0.70, 0.89	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

Adjusted for age, gender, and education, each additional hour/day of vigorous activity outside work is associated with an odds ratio of about 0.376 for ten-year mortality (95% CI: 0.264 to 0.520, $p < 0.001$): participants reporting more vigorous activity had substantially lower odds of dying within ten years, holding age, gender, and education constant.

Now compare that with the crude model, because the gap is the biggest one in this chapter. Crude, the odds ratio is 0.065, which would say that an extra hour a day of vigorous activity cuts the odds of dying by something like 94%. Adjusted, it is 0.376: still a substantial association, but a fraction of the size. Almost all of that difference is age. Older participants do far less vigorous activity, and are far more likely to die within ten years, for reasons that have nothing to do with the activity itself. So a crude comparison of the active against the inactive is largely a comparison of the young against the old. This is the same confounding story the chapter told with `sed`, in a much louder version: there the adjusted estimate moved from 1.040 to 1.020, here it moves by a factor of about 6. Reporting the crude figure on its own would badly overstate what vigorous activity does.

Exercise 3

```
vpa_r_cvd_complete <- vpa_r_dead_complete |> filter(!is.na(blm.cvd))

m_vpa_r_without <- glm(dead_10yr ~ vpa_r + age + gender + edu,
  data = vpa_r_cvd_complete, family = binomial)
m_vpa_r_with <- glm(dead_10yr ~ vpa_r + age + gender + edu + blm.cvd,
  data = vpa_r_cvd_complete, family = binomial)

tbl_vpa_without <- tbl_regression(m_vpa_r_without, exponentiate = TRUE)
tbl_vpa_with <- tbl_regression(m_vpa_r_with, exponentiate = TRUE)

tbl_merge(list(tbl_vpa_without, tbl_vpa_with),
  tab_spanner = c('Without baseline CVD', 'With baseline CVD'), quiet = TRUE)
```

Characteristic	Without baseline CVD OR	95% CI	p-value	With baseline CVD OR	95% CI	p-value
vpa_r	0.38	0.26, 0.52	<0.001	0.41	0.29, 0.56	<0.001
age	1.10	1.09, 1.10	<0.001	1.09	1.09, 1.09	<0.001
gender						
Male	—	—		—	—	
Female	0.58	0.52, 0.65	<0.001	0.61	0.55, 0.68	<0.001
edu						
Less than high school	—	—		—	—	
High school graduate	0.94	0.82, 1.07	0.4	0.94	0.82, 1.08	0.4
Above high school	0.79	0.70, 0.89	<0.001	0.82	0.73, 0.93	0.002
blm.cvd						
No				—	—	
Yes				2.34	2.05, 2.68	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

The odds ratio for `vpa_r` moves a little between the two models: 0.376 (95% CI 0.264 to 0.520, $p < 0.001$) without `blm.cvd`, and 0.409 (95% CI 0.289 to 0.564, $p < 0.001$) with it. That's a smaller shift than the main adjustment step but still a real one. Without `blm.cvd`, the model assumes that any relationship between baseline CVD and mortality operates entirely through the predictors already in the model; with it, the model assumes `blm.cvd` is a separate cause of death, worth adjusting for directly. Both are defensible readings of a single measurement taken at one study visit, and nothing in this dataset can say which one is correct.

Writing exercise (sample answer)

Sedentary time was associated with total cholesterol after adjustment for age, gender, and education (slope -0.00113 mmol/L per hour/day, 95% CI -0.00224 to -0.00002, $p = 0.047$), and with higher odds of ten-year all-cause mortality (adjusted odds ratio 1.020, 95% CI 1.013 to 1.028, $p < 0.001$). Both estimates adjust only for the variables included in this model; they cannot rule out confounding by factors that were never measured, such as overall frailty or comorbidity not captured here.

In a sensitivity analysis, the adjusted odds ratio for sedentary time and ten-year mortality was 1.020 (95% CI 1.013 to 1.028) without baseline cardiovascular disease in the model, and 1.019 (95% CI 1.012 to 1.026) with it included. The first model assumes baseline CVD's influence on mortality runs entirely through the other adjustment variables; the second treats it as an independent contributor to mortality risk worth adjusting for directly. Because sedentary behaviour and baseline CVD were both measured at the same study visit, this dataset cannot establish which assumption is correct, and both estimates are reported for that reason.

Summary

This is the end of the course, and let's summarise what we have done here:

We communicated with R using text commands, and learnt what to do when we see R producing messages, warnings, and errors. We organised data so that they are 'tidy' and analysable. We described variables in the data based on their type, using plots and summary statistics. We distinguished a sample from a population, and the meaning of a sampling distribution. We attached a confidence interval to a summary statistic (e.g. mean, proportion), and the relationship of a CI with a classical hypothesis test. We also tested for any differences in summary statistics in the population using appropriate tests. We examined the linear association between two continuous variables using correlation coefficients, and built a simple linear regression model around that. Lastly, we adjusted for potential confounders in a multiple linear regression, and explored doing similar things for binary variables using logistic regression. These are most of the common statistical techniques in public health, but by no means are they the only things that you will see in the future.

Common confusions

Terminology can be the most confusing thing about epidemiology and statistics, and what we use here might be different from what you see elsewhere. We call the regression model with one outcome and several predictors 'multiple linear and logistic regression', but some describe it as *multivariable regression*. Both mean the same thing. However, *multivariate regression* is different and often misused. Formally, a multivariate model includes several outcomes together. This course doesn't cover it, and it's not common in public health research generally.

The other confusion could stem from how we describe variables: what this book calls the **outcome** is sometimes called the **dependent variable**, and what it calls a **predictor** is sometimes called an **independent variable**. Note also that an **exposure** is a **predictor**, but a **predictor** isn't necessarily an **exposure**.

The second confusion could be around modelling strategy. Every adjustment set in Chapter 7's models, `age`, `gender`, and `edu`, then `blm.cvd`, was picked from subject-matter reasoning about what could confound the association in question, decided before any model was fitted. That's the strategy for an aetiological question, one asking what causes the outcome. In contrast there is also prognostic questions, where one asking how well we can predict who gets the outcome. Prognostic modelling calls for a different strategy: selecting variables to minimise prediction error, regardless of confounders, mediators, colliders, or anything related to causality. These are outside the scope of this course. You may occasionally see people putting all available variables into the model, or selecting variables by p-value alone. Both are poor practice, generally. For an aetiological question these strategies risk adjusting for a mediator or a collider, either masking a true association or manufacturing a spurious one. For a prognostic question these strategies risk *multicollinearity* and *overfitting*, which are covered in Advanced Statistics. P-values also mean very little in these sequential testing, because of *multiplicity*, another concept we can't cover here.

What this course has not covered

Below is a non-exhaustive list of statistical techniques that you may see in public health literature. Firstly, those that will be taught in Advanced Statistics:

- **Survival analysis.** Signposted already: Chapter 7 threw away `fu_month` and excluded early-ending follow-up. That could be better modelled using Kaplan-Meier estimator and Cox regression.
- **Generalised linear models.** The logistic regression Chapter 7 taught is basic. We haven't covered model fit statistics, pseudo-R^2, diagnostics and prediction specific to logistic models. Logistic regression also belongs to a wider class of models called generalised linear models, hence the function name `glm()`. That family can accommodate other types of outcome too, for example a count variable. There are also more advanced topics like interaction and non-linear predictor-outcome association inm GLM.
- **Multi-level modelling**, another modelling class where observations could be correlated with each other.
- **Mediation analysis.** Chapter 7 raised the confounder-versus-mediator ambiguity and stopped at reporting both models. Formally decomposing an effect into direct and indirect parts is a separate method.
- **Meta-analysis**, combining estimates across studies rather than one from a single dataset.

The topics below aren't covered anywhere in the MPH curriculum at the moment:

- **Prognostic modelling.** A prognostic model aims to maximise prediction accuracy. It's built, judged, and reported differently, and require different techniques.
- **Missing data handling.** Every regression model built across Chapters 6 and 7 is *complete-case*: anyone missing a value on any variable in the model was dropped. Chapter 7's logistic model, for instance, ran on 10,696 of 41,765 participants. Complete-case analysis and reporting the exclusion is the minimum, but could be subject to biases if the missing data is not completely random. It requires a different class of methods.
- **Sample size calculation based on regression.** Chapter 4 sized a two-group comparison; sizing a study around an adjusted coefficient is a different calculation.

- **Time-varying confounding**, where a confounder changes over follow-up and standard adjustment stops working.
- **Survey weighting**, from Chapter 3: we've treated a complex survey as a simple cohort throughout, so nothing here is a population-representative estimate.

Statistics is a huge topic and no one can know everything. But the ability to differentiate when a method is good for the data, and when it is not, is one of the most important skills to have if you continue to do quantitative data analysis. With the foundation you built here and the wealth of material available for R online, you should be able to pick up new skills relatively easily.

Glossary

This appendix is a reference: packages, functions, R syntax, and statistical terms used across the ebook, each with a short, simplified description and the chapter where the fuller explanation lives.

Packages

Package	What it's for	First appearance
base R	Comes with R itself, no `library()` needed: objects, vectors, arithmetic, comparisons, `data.frame()`, basic tests and models.	Chapter 1
`dplyr`	Manipulating data: filtering rows, building new columns, grouping and summarising.	Chapter 1
`readr`	Reading data files (e.g. CSV) into R. Faster than base R's own version, and doesn't silently guess wrong.	Chapter 2
`here`	Builds a file path from the project folder down to a file, so code runs on any machine.	Chapter 2
`ggplot2`	All the plotting in this book.	Chapter 2
`gtsummary`	Publication-style summary and regression tables.	Chapter 3
`medicaldata`	Supplies `strep_tb`, the 1948 MRC streptomycin trial data.	Chapter 4
`epitools`	Risk ratios and odds ratios with confidence intervals, from a contingency table.	Chapter 5

Package	What it's for	First appearance
`broom`	Tidies a fitted model's coefficients and fitted values into a data frame.	Chapter 6
`broom.helpers`	Not loaded directly. `gtsummary`'s `tbl_regression()` depends on it and finds it automatically once it's installed.	Chapter 7

Functions

Base R: objects, vectors, and data frames

Function	What it does	First appearance
`c()`	Combines single values ('scalars') into a vector.	Chapter 1
`class()`	Reports what kind of value an object holds.	Chapter 1
`data.frame()`	Builds a data frame, columns supplied by name.	Chapter 1
`factor()`	Converts a variable into a set of named categories, with levels in a stated order.	Chapter 1
`library()`	Loads an installed package for the current session.	Chapter 1
`install.packages()`	Downloads a package onto this computer. It only needs to be run once per computer.	Chapter 1
`read.csv()`	Base R's own function for reading a CSV file. Superseded by `readr`'s `read_csv()` from Chapter 2 onward.	Chapter 1
`write.csv()`	Writes an object out as a real CSV file on disk. `row.names = FALSE` leaves out R's own row-number column.	Chapter 1

Function	What it does	First appearance
`is.na()`	Tests whether each value is missing.	Chapter 2
`names()`	Lists a data frame's column names.	Chapter 2
`dim()`	Reports the number of rows and columns.	Chapter 2
`head()`	Prints the first few rows of a data frame.	Chapter 2
`substr()`	Extracts characters from a fixed position in a text string.	Chapter 2
`levels()`	Lists a factor's categories, in their current order.	Chapter 3
`relevel()`	Reassigns a factor's reference level to the one named.	Chapter 3
`list()`	Bundles several (different) objects together.	Chapter 3
`nrow()`	Counts the rows in a data frame.	Chapter 4
`set.seed()`	Fixes the sequence of 'random' numbers R uses, so a random draw repeats the same way every time.	Chapter 4
`as.numeric()`	Converts a value to a continuous number.	Chapter 6

Base R: summaries and maths

Function	What it does	First appearance
`mean()`	Calculates the arithmetic mean. `na.rm = TRUE` drops missing values before calculation.	Chapter 1
`round()`	Rounds a number to a stated number of decimal places.	Chapter 1
`sum()`	Adds values, or counts the number of `TRUE`s when given a logical condition.	Chapter 1

Function	What it does	First appearance
`summary()`	Prints a quick five-number-plus-mean summary of a variable, or an overview of a data frame.	Chapter 2
`median()`	Calculates the median.	Chapter 3
`sd()`	Calculates the standard deviation.	Chapter 3
`IQR()`	Calculates the interquartile range (3rd quartile - 1st quartile).	Chapter 3
`quantile()`	Reports the value at a stated percentile.	Chapter 3
`table()`	Cross-tabulates one or more categorical variables into counts.	Chapter 3
`prop.table()`	Converts a table of counts into proportions. `margin = 1` divides by row totals, `margin = 2` by column totals.	Chapter 3
`sqrt()`	Calculates a square root.	Chapter 4
`exp()`	Undoes a natural logarithm, used to convert a log-odds coefficient back to an odds ratio.	Chapter 7

Base R: tests and models

Function	What it does	First appearance
`t.test()`	A t-test: a confidence interval and p-value for a mean, or for the difference between two means. `paired = TRUE` runs a paired test; `var.equal = TRUE` assumes the two groups share one variance, the same assumption `lm()` makes of a binary predictor.	Chapter 4

Function	What it does	First appearance
`prop.test()`	A confidence interval and p-value for a proportion, or for the difference between two proportions. Uses a continuity correction by default; `correct = FALSE` switches it off.	Chapter 4
`power.t.test()`	Calculates the sample size needed to detect a given difference in means, or the power a given sample size has.	Chapter 4
`power.prop.test()`	The same calculation as `power.t.test()`, for a difference in proportions.	Chapter 4
`wilcox.test()`	The Wilcoxon signed-rank test, a non-parametric alternative to a one-sample or paired t-test. `mu` sets the reference value for a one-sample test; `paired = TRUE` runs the paired version.	Chapter 5
`chisq.test()`	The chi-squared test of association between two categorical variables. `correct = FALSE` switches off its default continuity correction.	Chapter 5
`fisher.test()`	Fisher's exact test.	Chapter 5
`cor()`	Calculates a correlation coefficient.	Chapter 6
`cor.test()`	A correlation coefficient with a confidence interval and p-value attached.	Chapter 6
`lm()`	Fits a linear regression model.	Chapter 6
`coef()`	Pulls a fitted model's coefficients.	Chapter 6

Function	What it does	First appearance
`confint()`	Pulls a fitted model's confidence intervals. For `glm()`, this refits the model repeatedly ('profiling') rather than using a shortcut formula.	Chapter 6
`predict()`	Uses a fitted model to generate a predicted value for a new set of inputs, supplied as a data frame.	Chapter 6
`glm()`	Fits a generalised linear model. `family = binomial` fits a logistic regression, for a 0/1 outcome.	Chapter 7

`readr`

Function	What it does	First appearance
`read_csv()`	Reads a CSV file into a data frame. Faster than base R's `read.csv()`, and doesn't silently convert text columns to factors.	Chapter 2
`write_csv()`	The `readr` equivalent of `write.csv()`. Named alongside `read_csv()` in Chapter 2, but never called in this book's taught code.	Chapter 2

`here`

Function	What it does	First appearance
`here()`	Builds a file path from the project folder, so the same code finds its files on any machine.	Chapter 2

dplyr

Function	What it does	First appearance
`select()`	Keeps only the named columns. Another package, `MASS`, defines a different function with the same name; this book never loads `MASS`, so `select()` always means `dplyr`'s version.	Chapter 1
`glimpse()`	Prints every column of a data frame, its type, and a preview of its values, one row per column.	Chapter 3
`filter()`	Keeps only the rows that match a condition.	Chapter 3
`mutate()`	Adds a new column, or changes an existing one.	Chapter 3
`arrange()`	Sorts rows by one or more columns. `desc()` reverses the order.	Chapter 3
`desc()`	Reverses the sort order inside `arrange()`, high to low.	Chapter 3
`slice()`	Keeps rows by their position.	Chapter 3
`group_by()`	Groups a data frame's rows by one or more variables, so the next step runs once per group.	Chapter 3
`summarise()`	Collapses each group (or the whole data frame, if ungrouped) into a row of summary statistics.	Chapter 3
`n()`	Counts rows, used inside `summarise()`.	Chapter 3
`slice_sample()`	Keeps a random subset of rows. `prop =` states the fraction to keep.	Chapter 4
`pull()`	Extracts one column as a plain vector, rather than a one-column data frame.	Chapter 4

Function	What it does	First appearance
`case_when()`	Builds a new column from a list of conditions, checked in order.	Chapter 7

`ggplot2`

Function	What it does	First appearance
`ggplot()`	Starts a plot, given a data frame.	Chapter 2
`aes()`	Maps a variable from the data onto a visual property in the plot, e.g. x, y, fill, or colour.	Chapter 2
`geom_histogram()`	Draws a histogram of a continuous variable. `binwidth =` sets the width of each bar.	Chapter 2
`geom_bar()`	Draws a bar chart of a categorical variable's counts.	Chapter 3
`geom_boxplot()`	Draws a boxplot.	Chapter 3
`geom_point()`	Draws a scatterplot.	Chapter 3
`labs()`	Sets a plot's axis labels and title.	Chapter 3
`ylim()`	Fixes the y-axis range shown.	Chapter 3
`geom_smooth()`	Adds a fitted line, with a shaded confidence band, over a scatterplot.	Chapter 6
`geom_hline()`	Draws a horizontal reference line at a stated `yintercept`.	Chapter 6
`stat_qq()`	Plots standardised residuals against what a normal distribution would produce, for checking a regression's normality assumption.	Chapter 6
`stat_qq_line()`	Adds the reference line to a `stat_qq()` plot.	Chapter 6

`gtsummary`

Function	What it does	First appearance
`tbl_summary()`	Builds a publication-style summary table ('Table 1'). `by =` stratifies it by a grouping variable.	Chapter 3
`all_continuous()`	Inside `list()` or `statistic =`, means 'every continuous variable in this table'.	Chapter 3
`add_p()`	Adds a column of hypothesis-test p-values to a `tbl_summary()` table.	Chapter 5
`tbl_regression()`	Builds a publication-style table directly from a fitted model.	Chapter 7
`tbl_merge()`	Combines two or more `gtsummary` tables side by side, for comparing crude and adjusted models.	Chapter 7

`broom`

Function	What it does	First appearance
`tidy()`	Turns a fitted model's coefficients into a data frame, one row per term. `exponentiate = TRUE` converts a logistic model's coefficients to odds ratios.	Chapter 6
`augment()`	Adds each observation's fitted value (`.fitted`) and residual (`.resid`) back onto the original data.	Chapter 6

`epitools`

Function	What it does	First appearance
`riskratio()`	Takes an r x 2 table and calculates the risk ratio, with a confidence interval. r is the number of levels in the exposure variable.	Chapter 5
`oddsratio()`	Takes an r x 2 table and calculates the odds ratio. `method = 'wald'` matches the $\pm$ 1.96 x SE formula behind a confidence interval built by hand.	Chapter 5

R syntax and operators

<table>
<tr><th>Symbol</th><th>What it does</th><th>First appearance</th></tr>
<tr><td><code><-</code></td><td>Assignment: stores the value on the right in the name on the left.</td><td>Chapter 1</td></tr>
<tr><td><code>|></code></td><td>The pipe. Takes what's on the left and feeds it in as the first input to what's on the right. Read aloud as 'and then'.</td><td>Chapter 1</td></tr>
<tr><td><code>%>%</code></td><td>The older pipe, from <code>dplyr</code>, before base R had its own. Both can be treated the same.</td><td>Chapter 1</td></tr>
<tr><td><code>$</code></td><td>Pulls one named column, or one named piece, out of a data frame or another R object.</td><td>Chapter 1</td></tr>
<tr><td><code>%in%</code></td><td>Tests whether each value appears anywhere in a set.</td><td>Chapter 1</td></tr>
<tr><td><code>== !=</code></td><td>Equal to, not equal to.</td><td>Chapter 1</td></tr>
<tr><td><code>< > <= >=</code></td><td>Less than, greater than, and their 'or equal to' versions.</td><td>Chapter 1</td></tr>
</table>

<table>
<tr><th>Symbol</th><th>What it does</th><th>First appearance</th></tr>
<tr><td>`''`</td><td>Marks a value as text. Numbers, `TRUE`/`FALSE`, and object or column names don't take quotes. `'20'` will be recognised as text in R.</td><td>Chapter 1</td></tr>
<tr><td>`` ` ``</td><td>A different kind of quoting mark, needed for a column name containing a space or other unusual character.</td><td>Chapter 1</td></tr>
<tr><td>`?`</td><td>Followed by a function name, opens that function's help page.</td><td>Chapter 1</td></tr>
<tr><td>`#`</td><td>Starts a comment in an R script: everything after it on the line is ignored when the code runs.</td><td>Chapter 1</td></tr>
<tr><td>`#|`</td><td>A chunk option, inside a code chunk: sets things like a chunk's label or whether its output shows.</td><td>Chapter 1</td></tr>
<tr><td>`NA`</td><td>R's code for a missing value.</td><td>Chapter 1</td></tr>
<tr><td>`TRUE` / `FALSE`</td><td>The two logical values. Always written in full in this book, never abbreviated to `T`/`F`.</td><td>Chapter 1</td></tr>
<tr><td>`!`</td><td>Negates a condition. `!TRUE` is `FALSE`.</td><td>Chapter 1</td></tr>
<tr><td>`&`</td><td>Combines two conditions, both must be true. `TRUE & TRUE` is `TRUE`; `TRUE & FALSE` is `FALSE`.</td><td>Chapter 1</td></tr>
<tr><td>`|`</td><td>Combines two conditions, either must be true. `TRUE | TRUE` is `TRUE`; `TRUE | FALSE` is `TRUE`.</td><td>Chapter 1</td></tr>
<tr><td>`::`</td><td>Calls a function from a named package directly, without loading the whole package with `library()` first, as in `dplyr::select()`.</td><td>Chapter 1</td></tr>
</table>

Symbol	What it does	First appearance
`+`	Inside `ggplot2`, joins plot layers together, not arithmetic addition.	Chapter 2
`~`	A formula: separates an outcome (left) from a predictor (right), as in `t.test(chol ~ gender)`.	Chapter 4

Statistical terms

Describing data

Term	What it means	First appearance
Categorical variable	A variable that puts each observation into one of a fixed set of groups. Nominal categories have no order (`gender`); ordinal categories do (`edu`); binary is a two-category special case (`blm.cvd`), even where R has stored it as the numbers 0 and 1.	Chapter 3
Numeric variable	A variable whose values are numbers, in one of two shapes: count (0, 1, 2, 3, …) or continuous (12.1, 13.5, …). A binary variable coded 0/1 is categorical, not numeric, whatever class R has stored it as.	Chapter 3

Term	What it means	First appearance
Factor	R's way of storing a categorical variable, as a set of named levels. What class R stores a variable as (`glimpse()`'s `<dbl>`, `<chr>`, or a factor) is a separate question from what the variable statistically is.	Chapter 3
Reference level	The category everything else in a table or model is compared against. In R the default is the first level in a factor.	Chapter 3
Mean, median	Two measures of a distribution's centre: the arithmetic average, and the middle value when sorted.	Chapter 3
Standard deviation (SD)	A measure of a distribution's spread, usually paired with the mean. When used without context an SD can be assumed to describe the population.	Chapter 3
Interquartile range (IQR)	Another measure of a distribution's spread, usually paired with the median.	Chapter 3
Skew	A distribution with a longer tail on one side, where the mean and median differ.	Chapter 3
Complete-case	An analysis that keeps only the rows with no missing value on any variable it uses.	Chapter 3
Table 1	The semi-standard first table of a paper: participant characteristics, often stratified by group.	Chapter 3

Estimation and uncertainty

Term	What it means	First appearance
Population	The entire group of individuals or items we want to draw conclusions about.	Chapter 4
Population distribution	How a specific measurement or characteristic is spread out across the entire population.	Chapter 4
Sample	A subset of individuals drawn from the population, which we actually observe and measure.	Chapter 4
Point estimate	A single number calculated from a sample (e.g. a sample mean or proportion), used as our best guess for the unknown population value.	Chapter 4
Sampling distribution	The theoretical distribution of point estimates we would get if we took infinite independent samples of the exact same size from the population.	Chapter 4
Standard error (SE)	The standard deviation of the sampling distribution. It measures the precision of our point estimate, smaller SE means more precise, and does not describe the variability of individuals in the population, which is what the standard deviation is for.	Chapter 4
Central limit theorem (CLT)	The rule stating that if our sample size is large enough, the sampling distribution of the mean will be approximately normal, regardless of the underlying population's shape.	Chapter 4

Term	What it means	First appearance
Confidence interval (CI)	A range of values, built around a point estimate and based on the standard error, that quantifies our uncertainty about the true population parameter. Also known as an 'interval estimate'.	Chapter 4
95% CI	If we repeated our sampling process indefinitely, 95% of the calculated intervals would contain the true population value. It is incorrect to say 'there is a 95% chance our specific interval holds the true value'.	Chapter 4
Power, sample size calculation	The chance a study detects a real effect of a given size, and the calculation that sizes a study to hit a target power.	Chapter 4

Hypothesis tests and measures of association

Term	What it means	First appearance
Null hypothesis	The baseline assumption in a statistical test, typically stating there is no effect, no difference, or no association between variables in the population.	Chapter 5
Alternative hypothesis	The statement a test weighs the null hypothesis against: that there is an effect, a difference, or an association. A test never proves the null hypothesis true; it only asks whether the sample is surprising enough to reject it in favour of the alternative.	Chapter 5

Term	What it means	First appearance
p-value	The probability of observing a result as extreme as, or more extreme than, the one calculated from our sample, assuming the null hypothesis is completely true. As good practice, we should always report point and interval estimates along with p-values.	Chapter 5
Parametric, non-parametric	Parametric tests assume the underlying population data follows a specific distribution (e.g. t-distribution, in t-tests). Non-parametric tests make no such assumptions and often rely on ranking the data (e.g. Wilcoxon tests).	Chapter 5
Paired, independent samples	Paired (or dependent) samples involve measurements linked to each other (e.g. pre/post tests on the same patient, or matched twins). Independent samples involve completely separate, unrelated groups.	Chapter 5
2x2 (contingency) table	A table summarising the relationship between two binary variables. In epidemiology, exposure is typically on the rows and the disease/outcome is on the columns.	Chapter 5
Risk, risk ratio	Risk is the probability of an event occurring: the number of cases divided by the total group. The risk ratio (or relative risk) divides the risk in the exposed group by the risk in the unexposed group.	Chapter 5

Term	What it means	First appearance
Odds, odds ratio	Odds compare the probability of an event happening to it not happening: the probability divided by 1 minus the probability. The odds ratio divides the odds of the outcome in the exposed group by the odds in the unexposed group.	Chapter 5
Null value	The mathematical value representing the null hypothesis. For differences (e.g. risk difference), the null is 0. For ratios (e.g. RR, OR), the null is 1.	Chapter 5

Regression

Term	What it means	First appearance
Correlation	A statistic ranging from -1 to +1 that measures the strength and direction of association between two continuous variables.	Chapter 6
Pearson's r	A parametric measure of the degree of a strictly linear relationship between two continuous variables. It is highly sensitive to outliers.	Chapter 6
Spearman's rho	A non-parametric measure of the degree of a monotonic relationship, where the variables move in the same direction but not necessarily at a constant rate. It is rank-based and more robust to outliers.	Chapter 6

Term	What it means	First appearance
Outcome, predictor	What a regression model explains (the outcome, also called the dependent variable), and what it explains it with (a predictor, also called an independent variable).	Chapter 6
Intercept	The fitted line's value at zero.	Chapter 6
Slope	How much the outcome changes per unit of a predictor.	Chapter 6
Coefficient	The general name for intercept and slope.	Chapter 6
Residual	The vertical distance between an observed value and the model's fitted line.	Chapter 6
Linearity, constant variance, normally distributed residuals	The three assumptions a fitted line's trustworthiness rests on, checked with diagnostic plots.	Chapter 6
Extrapolation	Using a model to predict outside the range of data it was actually fitted on.	Chapter 6
Multivariable	Several predictors, one outcome. Some writers say 'multiple regression' for the same thing. 'Multivariate regression' is a different, formal term for several outcomes modelled together, not used in this course.	Summary
Crude, adjusted	An association estimated on its own (crude), against one estimated alongside other variables that might confound it (adjusted).	Chapter 7

Study design and causal language

Term	What it means	First appearance
Confounder	A variable that's a common cause of both the exposure and the outcome, distorting the crude association between them unless properly adjusted for.	Chapter 7
Mediator	A variable that sits on the causal path between an exposure and an outcome. Adjusting for one, unlike a confounder, removes part of the real effect rather than a distortion, which is why the same variable can't be treated both ways.	Chapter 7
Adjustment set	The variables chosen, before fitting a model, to adjust an association for.	Chapter 7
Sensitivity analysis	Fitting a model more than one way, on purpose, to see how much a result depends on an assumption that can't be settled from the data.	Chapter 7
Collider bias	Distortion introduced by adjusting for a variable that's a common *effect* of the exposure and the outcome, rather than a common cause.	Chapter 7
Table 2 fallacy	Reading every coefficient in an adjusted regression table as if it had the same interpretation as the one the model was built to estimate.	Chapter 7

Where to look things up

`?` followed by a function name, typed into the console, opens that function's help page, covered properly in Chapter 1. It's usually faster than searching online, and it's always correct for the version of R actually installed. RStudio also keeps a set of cheat sheets under **Help →** **Cheatsheets**, one per major package.